

POLYTECH INTL
Master's Program in Data Sciences

MASTER'S THESIS

*submitted in partial fulfillment of the requirements for the degree of
Master's in Data Sciences*

The DAM AI Agent

A CRISP-DM Case Study

*Building a Grounded Conversational Agent over the African Development Bank's
Delegation of Authority Matrix*

Author

Adem Mrabet

Supervisors

Mr. Marouene Chaieb (*Academic Supervisor*)

Mr. Yasser Ahmad (*Professional Supervisor*)

Mr. Surya Joymungul (*Professional
Supervisor*)

Host Institution: Polytech Intl
Client Organization: African Development Bank Group
Internship Duration: 6 months

Academic Year 2025 – 2026

Acknowledgments

I would like to begin by thanking my professional supervisors, Mr. Yasser Ahmad and Mr. Surya Joymungul, for their guidance, availability, and the real, candid feedback they gave throughout this project. Their willingness to test the system directly, report exactly what broke, and push back on anything that felt merely plausible rather than genuinely correct shaped this project far more than a polite sign-off ever could have.

I am equally grateful to my academic supervisor, Mr. Marouene Chaieb, for his guidance throughout this internship and for the academic rigor he encouraged at every stage of this work.

Words feel too small for what I owe my family, but I want to try. To my father, thank you for teaching me, by example, what quiet determination looks like, and for believing in me on the days I found it hard to believe in myself; every value I have brought to this work traces back to something you showed me first. To my mother, thank you for a love that has never once wavered, for every sacrifice I only understood the size of much later, and for being the calm I could always return to no matter how demanding this project became. To my brother, thank you for being a constant, steady presence throughout my life. I would not be who I am, or have made it here, without the three of you.

To my girlfriend, thank you does not begin to cover it. Thank you for your patience on the nights this project took more of me than I had planned to give, for believing in me even when I doubted myself, for the quiet encouragement that kept me going through the hardest stretches, and for standing beside me with so much love and understanding throughout this entire journey. Your support has meant more to me than I can put into words.

I also want to thank my friends, who made this Master's program a genuinely good chapter of my life. To Ayoub and Alaa, whom I met during this Master's and could not be more grateful to have found: thank you for the countless conversations, the shared frustrations, and the support along the way. To Fayrouz and Houde, thank you for always being there. And to Brahim, my childhood friend, thank you for a friendship that has lasted far longer than this degree and that I know will continue long after it.

Finally, I want to acknowledge Polytech Intl for hosting and supervising this project, and the African Development Bank Group for providing the real, meaningful problem this thesis is built around.

Adem Mrabet

Contents

A brief, chapter-level overview is given below for quick orientation. The complete, detailed table of contents, listing every section and its page number, is provided at the very end of this document, following standard practice for this type of report.

General Introduction

Chapter 1	Host Institution: Polytech Intl
Chapter 2	Client Organization: African Development Bank Group
Chapter 3	Literature Review and Theoretical Background
Chapter 4	Business Understanding
Chapter 5	Requirements Specification
Chapter 6	Data Understanding
Chapter 7	Data Preparation
Chapter 8	Modeling
Chapter 9	Evaluation
Chapter 10	Deployment
Chapter 11	User Manual and Deployment Guide
Chapter 12	Project Management and Process
Chapter 13	Lessons Learned and Limitations

General Conclusion**Web References**

Annex A	Automated Test Suite Breakdown
Annex B	Project Directory Structure
Annex C	Sample Live-Demo Transcript
Annex D	Selected Decision-Log Excerpts
Annex E	Glossary of Terms Used in This Report
Annex F	Complete Project Decision Log (Verbatim)
Annex G	Key Code Excerpts
Annex H	API Reference

Table of Contents (detailed, with page numbers)

List of Figures

7.1	The eight-stage data preparation pipeline, from raw PDF extraction to a validated node set.	22
8.1	End-to-end system architecture: source document to user-facing answer.	28
8.2	A real excerpt of the knowledge graph around activity 3.111, showing the contains edge from its parent process and the responsible_for edges to its approving and informed roles — the exact structure validated against the real printed table.	29
8.3	The grounded-generation flow: an LLM may only rephrase already-verified facts, and any answer that drops or alters a fact is discarded in favor of the safe deterministic template.	32

List of Tables

List of Abbreviations

AfDB	African Development Bank Group
API	Application Programming Interface
CRISP-DM	Cross-Industry Standard Process for Data Mining
DAM	Delegation of Authority Matrix
DDG	Deputy Director-General
FR	Functional Requirement
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I / C / R / A	Initiate / Check-verify-or-consult / Review / Approve (the DAM's authority-code notation)
(i)	The DAM's informed-party marker
JSON	JavaScript Object Notation
LLM	Large Language Model
NFR	Non-Functional Requirement
NSO	Non-Sovereign Operations
OOM	Out Of Memory
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
TF-IDF	Term Frequency–Inverse Document Frequency
UI / UX	User Interface / User Experience

Abstract

The Delegation of Authority Matrix (DAM) is the African Development Bank’s internal reference document defining who may approve, check, initiate, or must be informed about each of the Bank’s operational activities. In its native form it is a 79-page PDF containing dozens of large, rotated-header tables, footnoted role names, and a bespoke authority-code notation (I, C, R, A, and the parenthetical informed marker) that is unreadable at speed even by trained staff, and completely opaque to any generic text-search or off-the-shelf AI tool.

This thesis builds an AI agent that answers natural-language questions about the DAM (*“who approves the quarterly mission program?”*, *“who needs to be informed about non-sovereign loan disbursement?”*) with concrete, sourced, deterministic answers, phrased conversationally by a large language model but never allowed to invent or omit a fact. The system is delivered as a small, hosted web application: a chat interface, a live analytics dashboard over the DAM’s own structure, and a REST API in between.

The engineering work follows the CRISP-DM methodology end to end. Business Understanding fixed the two founding example questions the whole project is judged against. Data Understanding spent the largest share of effort: the source PDF’s tables use rotated column headers, superscript footnote digits fused with authority codes, and cross-row text bleeding that pdfplumber’s own word extraction cannot resolve automatically – every one of these had to be discovered from real pages, not assumed, before it could be parsed correctly. Data Preparation turned this into a validated, machine-readable node set: 327 structured nodes (3 chapters, 35 processes, 138 tasks, 102 child tasks, 49 threshold variants) carrying 1,776 extracted responsibility records, with an unresolved-role rate of 0.6%. Modeling built a `networkx` knowledge graph (459 nodes, 2,108 edges) plus a TF-IDF lexical search layer and a hybrid local/cloud LLM phrasing layer whose output is verified, fact-by-fact, against the graph before ever being shown to a user – a grounding check that rejects and silently falls back to a deterministic template answer if the model drops or alters a single role name. Evaluation is continuous rather than a single held-out test: 374 automated tests across 23 files, each written in response to a real, confirmed bug, plus repeated live verification against actual screenshots of the source document. Deployment ships the system as a containerized, auto-deploying web service on a free hosting tier, with a chat UI, a filterable analytics dashboard, and support for five languages.

Keywords: knowledge graphs, retrieval-augmented generation, large language models, grounded generation, PDF information extraction, delegation of authority, CRISP-DM.

General Introduction

Context of the Internship

This thesis documents a six-month Master's internship project, carried out as part of the Master's Program in Data Sciences at Polytech Intl, addressing a concrete, everyday operational problem for a real client organization, the African Development Bank Group: the Bank's Delegation of Authority Matrix (DAM) – the internal document defining who may approve, check, initiate, or must be informed about each of the Bank's operational activities – is authoritative but practically unreadable at speed. It is distributed as a 79-page PDF of dense, rotated-header tables, footnoted role names, and a bespoke authority-code notation. Anyone who needs an answer from it – a new staff member, a manager double-checking a delegation, an auditor tracing accountability for a decision – must manually locate the right table, read across a 90-degree-rotated header row, and cross-reference footnotes scattered across the page. There is no search function smarter than literal string matching in a PDF reader, and no way to ask the document a question in plain language.

The internship's mandate was to determine whether this problem could be solved responsibly with an AI system: not a generic chatbot bolted onto the PDF, but a system whose answers could be trusted for a compliance-sensitive purpose, where a wrong answer about who has authority to approve a transaction is a real institutional risk, not a minor inconvenience. That framing – trustworthiness first, conversational convenience second – is the thread running through every technical decision documented in this thesis.

Report Structure and How to Read It

This thesis is organized to serve two audiences at once: a reader who wants the engineering narrative (what was built, what went wrong, how it was fixed, what remains open) and a reader who wants to verify specific claims against primary evidence (exact test counts, exact bug descriptions, exact source-code behavior). To serve both without forcing either to wade through the other's material, the thesis is split into a main body and a substantial appendix section.

Chapter 1 presents the host institution, Polytech Intl. Chapter 2 presents the client organization, the African Development Bank Group, and the working relationship

between the two. Chapter 3 provides the theoretical and methodological background needed to follow the technical decisions in later chapters – CRISP-DM itself, knowledge graphs, retrieval-augmented generation, and the specific extraction and search techniques used. Chapters 4 through 10 are the CRISP-DM phases proper: Business Understanding, Requirements Specification, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. Chapter 11 is a practical user manual for anyone who wants to run or extend the system themselves. Chapters 12 and 13 close the main body with project-management reflection and an honest accounting of limitations, followed by a General Conclusion and a Web References section.

The appendices are deliberately extensive and are not optional reading for anyone verifying this thesis’s claims rather than just following its narrative. Annex F reproduces the project’s complete, dated decision log in full – every bug found, every fix attempted (including the ones that failed and were reverted), every scope decision, in the order it actually happened. It is the single primary source this entire thesis is built from, and it is included in full rather than only excerpted, in the same spirit that a scientific report includes raw data rather than only summary statistics. Annex G reproduces the source code of the system’s most technically significant functions directly, so that a claim like “the grounding check normalizes whitespace and case before comparing” can be checked against the actual sixty lines of Python that implement it, not just a paraphrase of them.

A Note on Authorship and Process

This project was carried out by one graduate intern working in close, continuous collaboration with an AI coding assistant acting in a senior-engineer advisory role – reviewing proposed approaches, flagging risks before code was written, and, critically, insisting that every claimed fix be checked against real data before being trusted. That collaboration is itself part of the story this thesis tells: the discipline of “verify against a real screenshot or real characters before touching code, and honestly document a fix attempt that failed rather than silently discarding it” is visible throughout the decision log in Annex F, and this thesis treats that discipline as one of the project’s genuine deliverables, not merely a process footnote.

Chapter 1

Host Institution: Polytech Intl

1.1 Presentation

This project was carried out under the supervision of Polytech Intl, within its Master's Program in Data Sciences. In the arrangement covered by this thesis, Polytech Intl acts as both the academic institution awarding the degree and the hosting and supervising body for the internship itself, rather than the internship being hosted by a separate company – the program is structured so that students carry out a substantial, real-world capstone project under academic supervision, working directly with an external client organization on a genuine business problem rather than a simulated or purely academic exercise.

1.2 Academic Supervision

The project was supervised academically by Mr. Marouene Chaieb, who provided guidance throughout the internship on both the technical direction of the work and its presentation as a piece of academic and professional deliverable work. Academic supervision in this arrangement covered methodology (ensuring the project's engineering process could be defended and evaluated, not just its output), scope discipline (helping keep a six-month project bounded to what could genuinely be delivered and verified), and the overall framing of the final deliverable – this thesis.

1.3 Why This Arrangement

Structuring the internship this way – university as host, external organization as client – lets the program combine two things that are often in tension in a standard internship: genuine external accountability (the African Development Bank Group is a real client with a real problem, not a hypothetical case study) and the academic rigor of a supervised, methodologically grounded final thesis (CRISP-DM discipline, a running decision log, comprehensive automated testing) that a purely commercial internship placement does not always require or make time for. Chapter 2 describes the client side of this arrangement in detail.

Chapter 2

Client Organization: African Development Bank Group

2.1 The African Development Bank Group

The African Development Bank Group is a regional multilateral development finance institution whose mandate is to finance and support economic and social development across its African member countries, through sovereign and non-sovereign lending, grants, technical assistance, and policy engagement. Like any institution that commits and disburses significant financial resources across a large, geographically distributed organization, it depends on a clearly defined internal authority structure: a documented answer, for every category of decision, to the question of exactly which role or committee is permitted to take it, at what threshold, and who else must be consulted or informed. The Delegation of Authority Matrix is that documented answer, and its accuracy and accessibility are not administrative details – they are a control mechanism the institution’s financial governance depends on.

2.2 Project Context and Working Model

The project was carried out with domain access to the DAM document and direct feedback access to a professional supervisor within the Bank, Mr. Yasser Ahmad, with additional professional supervision from Mr. Surya Joymungul, working in continuous collaboration with an AI coding assistant on the technical implementation. The working model that emerged, and that is visible throughout the decision log in Annex F, combined domain knowledge from the client side (what a Check/Verify authority code actually means operationally, which questions a real user would actually ask, what a specific test question should honestly fail to answer) with the intern’s own implementation work, verified continuously against real data rather than accepted on faith.

2.3 Tools and Technologies

The project's technology choices were made under three real constraints: no budget for paid infrastructure, no labeled training data, and a hard academic deadline. The resulting stack – Python for the entire backend (`pdfplumber`, `scikit-learn`, `networkx`, `FastAPI`, `Pydantic`), React and Vite for the frontend, Docker for containerization, and Render for free-tier hosting – reflects those constraints directly rather than an unconstrained “best tool for the job” selection; each individual choice is explained and justified at the point in this thesis where it was made, most substantially in Chapter 8 (Modeling) and Chapter 10 (Deployment).

2.4 Data Confidentiality

The DAM source document itself is an internal African Development Bank document and is treated accordingly throughout this project: it is never exposed through any public repository or public-facing file download, and this thesis quotes only the specific, narrow examples (individual activity identifiers, individual role titles, individual authority codes) necessary to explain a technical point, never the document as a whole.

2.5 Project Governance: The Decision Log

Rather than a conventional project-management artifact (a Gantt chart, a sprint board), this project's actual governance mechanism was a single running, dated Markdown file (`docs/decisions.md`), appended to continuously as the project progressed, recording every scope decision, every confirmed bug and its fix, and every accepted trade-off, in the order it happened and with the real evidence that justified it. This thesis's author considers that file the project's single most valuable artifact, independent of the code itself, and Chapter 12 discusses why; the file is reproduced in its entirety as Annex F.

Chapter 3

Literature Review

This chapter summarizes the methodological and technical background that later chapters build on: the CRISP-DM process model that structures this entire report, and the four technical areas — PDF-based information extraction, knowledge graphs, lexical versus semantic text retrieval, and retrieval-augmented generation with large language models — that the system’s design draws on.

3.1 CRISP-DM as a process model

The Cross-Industry Standard Process for Data Mining (CRISP-DM) was published in 2000 by a consortium of companies (NCR, SPSS, Daimler-Benz, and OHRA) as a vendor-neutral methodology for data mining and, more broadly, data-driven system development. It defines six phases — Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment — explicitly organized as an iterative cycle rather than a strict waterfall: the standard’s own process diagram shows arrows looping back from later phases to earlier ones, reflecting the reality that data understanding routinely reveals gaps in the original business framing, and modeling routinely reveals gaps in data preparation that send a project back a step.

CRISP-DM was designed with classical predictive modeling in mind (a target variable, a training set, a model that scores held-out data), and this project’s system is not a predictive model in that sense — it is a hybrid of a deterministic knowledge base, a lexical search index, and a constrained generative-language layer. Applying CRISP-DM’s phase structure to a system of this shape required interpretation, documented explicitly at each phase boundary in this report, but the underlying discipline transfers directly and is arguably more important for a system like this one than for a classical model: understanding the data’s real structure and failure modes (Data Understanding) before building anything on top of it, and evaluating the system’s actual correctness — not just whether it runs — before calling it done (Evaluation), are exactly the two disciplines a project handling compliance-sensitive facts cannot afford to skip.

3.2 Knowledge graphs as an enterprise data structure

A knowledge graph represents entities as nodes and relationships between them as typed, directed edges, typically stored and queried as a graph rather than as normalized relational tables. The approach has a long history in enterprise information systems, gaining particular prominence for problems where the relationships between entities are as important as the entities themselves — exactly the shape of the DAM's own underlying structure, where the meaningful unit of information is not "this role exists" but "this role is responsible for this activity, with this specific authority code and this specific footnoted qualification." Representing the DAM as a property graph (networkx's `MultiDiGraph` implementation, in this project's case) makes relationship-shaped questions — "who is connected to this activity by an approval edge" — a direct graph traversal rather than a join across several relational tables or a re-parse of a flat list on every query.

3.3 Lexical versus semantic text retrieval

Text retrieval systems broadly split into lexical methods, which match documents to a query based on shared surface vocabulary (classically, TF-IDF — Term Frequency–Inverse Document Frequency — weighting combined with cosine similarity), and semantic methods, which match based on learned vector representations (embeddings) intended to capture meaning independent of exact wording. Semantic methods generally outperform lexical methods when there is enough training or fine-tuning data to produce embeddings well-suited to the target domain's vocabulary; lexical methods remain competitive, and are markedly cheaper and more transparent, when the corpus is small, static, and shares a controlled or specialized vocabulary — which is closer to this project's situation, as detailed in the resulting design decision in Section 8.3.

3.4 PDF-based information extraction

Extracting structured information from PDF documents is a well-known hard problem in document processing, precisely because PDF is a page-description format, not a structured-data format: it encodes where each character or word is drawn on a page, not what document structure that drawing represents. Tools like `pdfplumber` (built on `pdfminer.six`) recover words and characters with their pixel coordinates and font metrics, but reconstructing table structure, reading order, multi-line cell wrapping, and typographic conventions like superscript footnote markers is left entirely to the

consuming application's own heuristics. This project's Data Understanding and Data Preparation phases (Chapters 6 and 7) are, in large part, an extended case study in exactly this class of problem: the DAM's rotated column headers, its superscript authority-code decorations, and its cross-row text wrapping are all instances of visual conventions that carry real semantic meaning but are not explicitly encoded as such anywhere in the underlying PDF structure.

3.5 Retrieval-augmented generation and the hallucination problem

Large language models are well documented to "hallucinate" — to produce fluent, confident, and factually wrong output, including inventing entities, relationships, or citations that do not exist in any source they were given. Retrieval-augmented generation (RAG), as originally proposed, addresses this by retrieving relevant source passages at query time and conditioning the model's generation on them, rather than relying solely on knowledge implicit in the model's training weights. In practice, RAG substantially reduces but does not eliminate hallucination: a model can still contradict, omit, or subtly rephrase a retrieved fact in ways that change its meaning, particularly under an instruction to "phrase this naturally," which is precisely the instruction a conversational interface needs to give it. For a system whose factual claims concern who holds financial approval authority, "substantially reduces" is not a sufficient guarantee; Section 8.4 describes this project's response to that gap — an explicit, automated verification step that checks the model's output against the retrieved facts token-for-token before it is ever shown to a user, rather than trusting the retrieval step alone to make hallucination unlikely.

Chapter 4

Business Understanding

4.1 Business context

The African Development Bank's Delegation of Authority Matrix assigns, for every one of the Bank's operational activities, which role or committee has the authority to Initiate (I), Check/verify or Consult (C), Review (R), or Approve (A) that activity, and which roles must simply be informed of it (marked with a parenthetical "(i)"). The matrix spans three chapters — general operations, sovereign operations, and non-sovereign operations (NSO) — printed as a sequence of dense tables with rotated column headers (one column per role) and footnoted qualifications (seniority level, monetary thresholds, exceptions).

In practice, the matrix is looked up manually: a staff member has to find the right table, read across a rotated header row, and cross-reference footnotes at the bottom of the page. For anyone outside the specific team that maintains the DAM, this is slow and error-prone, and the document itself is not searchable by anything smarter than literal string matching in a PDF reader.

4.2 Objective

Build an AI agent that can answer plain-language questions about the DAM correctly, and communicate honestly when it cannot. The project's two founding example questions — used throughout development as an acceptance test, not just an illustration — are:

- "Who approves [task X]?"
- "Who are the informed parties for [task X]?" (equivalently, "who needs to be informed for [task title]?")

Both phrasings (by id and by title) had to resolve to the identical, correct answer before any later feature was considered safe to build on top of.

4.3 Success criteria

Three criteria were treated as non-negotiable from the outset, and every later architectural decision in this report — most importantly the "grounded generation" decision in Modeling — traces back to them:

1. **Never fabricate a fact.** An answer must never name a role, an authority code, or a footnote qualification that is not actually present in the source document for that activity. This is the central risk of using any LLM against a compliance-sensitive document, and it shaped the entire generation architecture (Section 8.4).
2. **Say "I don't know" rather than guess.** A question that does not resolve to a real activity, or a code that does not exist, must be reported as such — never silently matched to something unrelated because it happens to share vocabulary.
3. **Deployable for evaluation.** The system had to run as a live, clickable web application a professor could test directly, not a notebook or a local-only script — which is why hosting, containerization, and a real frontend (Sections 10.1–10.3) were treated as in-scope engineering work, not an afterthought.

4.4 Constraints

- A fixed initial build phase for getting a working end-to-end system in place (see Section 12.1), governed by a running decision log (`docs/decisions.md`) so that every scope trade-off is traceable after the fact rather than reconstructed from memory.
- No labeled training data of any kind — no query/answer pairs, no annotated tables — which directly ruled out training a custom model and shaped the Modeling phase's choice of TF-IDF over embeddings (Section 8.3).
- A free-tier-only hosting budget (no credit card, no paid infrastructure), which shaped Deployment (Section 10.1).
- A development sandbox with no PDF-rendering GUI and, for most of the project's later half, a network proxy that blocks direct calls to the LLM API being used — meaning every LLM-dependent feature had to be validated by mocking the provider and, separately, by the intern's own live testing on a real machine. This constraint is disclosed wherever it applies rather than glossed over, because it is a genuine limit on what could be independently verified from within the development environment itself.

Chapter 5

Requirements Specification

Formal requirements were not written as a separate up-front document in this project — the iterative, decision-log-driven working method described in Section 12.1 and Section 3.4 made a traditional requirements-freeze impractical, and arguably counterproductive, given how many real requirements only became visible after real data or real user testing exposed them (Section 9.6). This chapter reconstructs the requirements the delivered system actually satisfies, organized in the conventional functional/non-functional split, both as a record of what the system does and as a checklist a reader can verify the system against directly.

5.1 Functional requirements

FR1 — Direct identifier lookup. Given a question containing a valid DAM activity identifier (e.g. "2.126"), the system shall return the activity's recorded authority information with full confidence, without ambiguity.

FR2 — Natural-language activity lookup. Given a question describing an activity by name or description rather than by identifier, the system shall resolve it to the correct activity using lexical similarity, and shall report its confidence level rather than presenting a low-confidence match as certain.

FR3 — Authority-role queries. For a resolved activity, the system shall be able to answer who may approve it, who may review it, who must check or verify it, who may initiate it, and who must be informed of it, using the DAM's own I/C/R/A/(i) notation.

FR4 — Mandatory obligation surfacing. Independent of which specific authority question was asked, the system shall always surface an activity's recorded mandatory Check/Verify obligation and, when present, its informed parties, since a Check/Verify entry is a mandatory institutional obligation rather than optional context (Section 8.6, Section 10.6).

FR5 — Cross-reference resolution. Where an activity's own row is a redirect to another activity or document section, the system shall follow that redirect automatically rather than reporting no responsibilities recorded.

FR6 — Typo and phrasing tolerance. The system shall tolerate common misspellings and minor phrasing variation in a question without requiring exact vocabu-

lary match, using a deterministic method that does not depend on network access to a language model.

FR7 — Glossary and abbreviation lookup. The system shall answer direct questions about what a DAM abbreviation or acronym stands for, and shall expand acronyms inline within role names in its other answers.

FR8 — Authority-code legend explanation. The system shall answer direct questions about what the DAM's own I/C/R/A/(i) notation means, distinct from questions about who holds a specific code on a specific activity.

FR9 — Conversational context carryover. Within a single conversation, a clearly anaphoric follow-up question (e.g. "and who is informed?" immediately after resolving an activity) shall be answered about the same activity as the previous turn, without the user needing to repeat the activity's name or identifier.

FR10 — Honest refusal. For a question referencing an identifier or activity that does not exist in the DAM, or a question genuinely outside the DAM's subject matter, the system shall say so explicitly rather than returning a fabricated or unrelated answer.

FR11 — Conversational tone adaptation. The system shall detect signs of user frustration or confusion and adapt its response's opening tone accordingly, without altering the underlying facts of the answer.

FR12 — Multi-language support. The system shall accept questions and provide answers in English, French, Spanish, Portuguese, and Arabic, with an explicit user-selectable override independent of the question's own detected language.

FR13 — Grounded LLM phrasing with safe fallback. When a language-model mode is enabled, the system shall use the model only to rephrase already-retrieved facts, shall verify that every fact survives the rephrasing, and shall fall back to a deterministic, template-based answer whenever that verification fails or no model is available.

FR14 — Analytics dashboard. The system shall provide a live, filterable visualization of the DAM's own structure (node-type breakdown, authority-code distribution, role rankings, coverage metrics), scoped by chapter.

5.2 Non-functional requirements

NFR1 — Factual reliability. No answer shall state a role, authority code, or footnote qualification that is not genuinely present in the source document for that activity. This is the system's highest-priority non-functional requirement, ranked above response fluency or response time, and is the direct justification for the grounded-generation architecture in Section 8.4.

NFR2 — Deployability on free infrastructure. The system shall run within the memory and compute limits of a free-tier hosting platform, since the project has no infrastructure budget.

NFR3 — Resilience to LLM unavailability. Every feature that depends on a language model shall degrade gracefully to a deterministic equivalent (a template answer, an untranslated response, a neutral tone) rather than failing or blocking when the model is unreachable, over quota, or returns a malformed response.

NFR4 — Response latency suitable for live conversation. A typical query shall resolve and return within a small number of seconds under normal operating conditions (excluding a cold-start wake from the hosting platform’s inactivity sleep, which is disclosed to the user as a known, accepted limitation in Section 13.2).

NFR5 — Test coverage for every confirmed defect. Every confirmed real bug shall be captured by at least one automated regression test before being considered resolved (Section 9.1–9.2).

NFR6 — Traceability of design decisions. Every non-trivial design or scope decision shall be recorded, dated, and justified with the real evidence that motivated it, in a running project log (Section 3.4, reproduced in full as Annex F).

NFR7 — Mobile usability. The chat and dashboard interfaces shall remain usable on a small mobile viewport, not only on desktop.

NFR8 — Data-source privacy. The DAM source document itself, being an internal Bank document, shall not be exposed through any public repository or public-facing file download; only the system’s derived, structured answers are exposed to end users.

5.3 Use cases

Table 1 summarizes the primary use cases the system was designed against, each directly traceable to a functional requirement above and to a real, tested question in this report (Annex C).

Use case	Primary actor	Requirement(s)	Example
Look up an activity’s approver by id	Any user	FR1, FR3	"who approves 2.126"
Look up an activity’s approver by description	Any user	FR2, FR3	"who approves the quarterly mission program"

Use case	Primary actor	Requirement(s)	Example
Discover an activity's full obligation set	Any user	FR3, FR4	"who approves 2.126" (also surfaces Check/Verify and informed parties)
Follow a redirected activity	Any user	FR5	"who initiates 1.117.2"
Ask a follow-up without repeating context	Any user	FR9	"and who are the informed parties?"
Look up what an abbreviation means	Any user, esp. new staff	FR7	"what does DDG stand for"
Understand the authority-code notation itself	New staff, auditors	FR8	"what's I, A and (i)?"
Ask in a non-English language	Non-English-speaking staff	FR12	"qui approuve 3.111"
Probe the system with an invalid or out-of-scope question	Evaluators, auditors	FR10	"what happens with 9.999.999"
Visualize DAM structure and coverage	Managers, auditors	FR14	Dashboard chapter filter and linked charts

Chapter 6

Data Understanding

6.1 The source document

Two candidate PDFs existed: a 66-page "MVP" export used in early exploration, and a 79-page "updated" export. The updated file was chosen because it alone contains the glossary, the abbreviations list, and the authority-code legend — reference data the agent needs for its own glossary and authority-code explanation features later on. This decision also expanded scope to include Chapter 3 (Non-Sovereign Operations), a full additional chapter of client relations, appraisal, trade finance, approval, and portfolio-management tables that the original ~50-table MVP scope had not covered.

6.2 Initial data exploration: what pdfplumber sees

The project used `pdfplumber` for PDF text and character extraction. Early exploration established several facts about the document that shaped every downstream design decision:

- **Column headers are rotated 90 degrees.** Extracting them requires reading `pdfplumber`'s raw characters in descending-top order, not the usual left-to-right reading order, and re-clustering headers that wrap across two vertical text strips (e.g. "Task Manager" printed as two adjacent runs) — verified character-by-character against real header text before any clustering logic was trusted.
- **Authority codes carry superscript decorations that `pdfplumber` cannot always distinguish.** A cell reading "C13" (a Check/verify code with footnote 13) is rendered identically, in the underlying PDF, to a cell where "C" and "13" are two separate concepts — but `pdfplumber`'s own word extraction is **inconsistent**: the same visual pattern comes out as one fused token on one page and two separate tokens on the next. This was confirmed at the character level (comparing raw x-position, y-position, and font size of each digit) before concluding that no text-only regex could solve it reliably — only real character geometry could, which the project subsequently built (`parsing/action_geometry.py`).
- **The letter "C" is semantically overloaded.** C1/C2 (printed in red) mean "Check and Verify"; C3/C4 (printed in purple) mean "Consult" (of the Board, Governors,

or an independent function) — two unrelated authority concepts sharing one letter, distinguished only by level number and color. This was flagged early as a trap for the natural-language intent parser built later (Section 8.2), and is covered by a dedicated test ensuring a "consult" question can never surface Check/verify roles or vice versa.

- **A hidden fourth hierarchy level exists.** Beyond chapter → process → task → child task, some child tasks carry their own lettered sub-items keyed by monetary threshold (e.g. activity 2.513.3 splits into "(a) Up to UA 2,000,000 / (b) Over UA 2M up to UA 10M / (c) Over UA 10M"). This was found via a single real screenshot and then confirmed document-wide (51 genuine instances, 2 false positives correctly excluded) before being formalized as a new node type, `threshold_variant`.
- **The informed-party marker "(i)" is not what it visually appears to be.** In the underlying character stream the marker is five separate characters — open-parenthesis, space, "i", space, close-parenthesis — not three consecutive characters as it looks on the rendered page. A first-generation extractor that scanned for three consecutive characters therefore missed every single instance; this was the single highest-impact data-understanding bug in the project (Section 7.4).
- **Reference/redirect phrasing is inconsistent across the document.** At least four distinct real phrasings redirect a reader from one activity to another ("See DAM 16.100, 16.200...", "Refer to Activities X – Y in Section Z", "See 2.120: Organization of mission", and "(See 1.114.1 / 1.114.2)"), each discovered separately by hitting a real question that the agent answered wrongly, never assumed as a complete set in advance.

6.3 Data quality issues catalogued before parsing began

The project's discipline throughout was to confirm every anomaly against real page content (character coordinates, a screenshot, or both) before writing a fix — never to patch based on a hypothesis alone. This surfaced a long list of genuine data-quality issues in how pdfplumber represents the source PDF, distinct from any coding mistake:

- Superscript footnote digits are visually indistinguishable from level-indicator digits except by comparing each digit's own vertical alignment to the base letter's top/bottom edge — a rule only arrived at after an earlier, wrong hypothesis (comparing to cell-center) was tested against real coordinates and disproved.

- A wide table row's rightmost and leftmost cells can render up to several points apart vertically, enough to be bucketed as two separate physical rows by naive y-coordinate grouping — confirmed via a requested screenshot of a real table (NSO section, page 58) after two more theoretical fixes had already been tried and rejected as unsafe.
- Two adjacent but genuinely different columns (e.g. "Regional Implementation Support Manager (RISM)" and "Country Manager / DDG") can sit closer together in x-position than two wrapped lines of the *same* column elsewhere on the same page — meaning no single distance threshold can correctly separate "different columns" from "wrapped header lines" document-wide; a content-based signal (a closing parenthesis ending a balanced role name) had to be added alongside the geometric one.
- pdfplumber's word extraction genuinely merges characters from two unrelated, visually overlapping lines of text into corrupted single "words" on a handful of titles — not synthetic OCR noise, but a real limitation of the library's row-heuristic on this specific document's layout, confirmed by re-extracting the same region at the character level and finding the real, un-merged text underneath.
- One-off inconsistencies exist even within the source PDF itself: one footnote section header reads "Notes to" where every other instance in the document reads "Notes on" (a genuine typo in the DAM's own formatting), and it silently broke a section-boundary detector until checked page-by-page.

6.4 Scope decisions grounded in real data

Two scope-narrowing decisions were made only after checking the real content, not by default:

- The Abbreviations list (document pages 2–7) was extracted and wired into the agent, but the separate Glossary section (pages 8–11) was deliberately deferred: its longer, line-wrapping definitions break the simple row-pairing approach that works for the short Abbreviations entries, and it was judged not worth the additional geometry work within the time budget.
- The action-geometry extractor's per-task action counts disagreed with the (already-trusted) text-based counts on roughly 14% of tasks, with disagreements ranging from off-by-one to wildly wrong on individual cases. Rather than chase every remaining disagreement — the spread of error sizes suggested more than one uncatalogued cause — the project consciously shipped with the text-based count as the trusted source of truth and geometry as best-effort supplementary

detail only, a decision made explicitly with the internship supervisor rather than silently absorbed into the code.

6.5 Two deep-dive investigations worth narrating in full

Two findings during Data Understanding are worth walking through in more procedural detail than the summary above, because the process by which they were found is as instructive as the findings themselves.

The hidden fourth hierarchy level. The DAM's hierarchy was originally understood as three levels — chapter, process, task — with an optional child task beneath a task. A screenshot of document page 36 (activity 2.513.3) showed something the schema had no place for: a child task with its own further sub-items, lettered by monetary threshold rather than numbered sequentially — "(a) Up to UA 2,000,000", "(b) Over UA 2M up to UA 10M", "(c) Over UA 10M" — each carrying its own independent set of authority-code responsibilities. The natural first question was whether this was a one-off formatting quirk on this single page or a real, recurring structural feature. It was checked document-wide before any schema change was made: 53 lettered sub-item lines were found across the full 79 pages, 51 genuine and 2 false positives (letter-lists embedded inside footnote prose, which the existing Notes-section boundary detector already excluded correctly). Confirmed as real and recurring, the decision — made jointly with the internship supervisor rather than unilaterally — was to model this as a new, distinct node type (`threshold_variant`) with a dot-joined identifier built from whichever child task most recently opened (e.g. "2.513.3.a"), rather than treating it as just another child task, because it represents a conditional amount-band of one activity rather than a separate activity in its own right.

The OCR-looking title corruption, root-caused rather than patched. A small number of activity titles appeared to contain OCR-style letter-spacing artifacts (e.g. fragments resembling "ro", "je", "c" instead of readable words), and an early, reasonable-looking hypothesis was that this was scanning noise in the source PDF itself, to be handled with a cleanup pattern once convenient. Investigating properly instead of patching around the symptom revealed two distinct and unrelated root causes. The first, affecting one specific title, was a genuine parsing bug: the page's own footnote-section header read "Notes to NSO 3.510 – 3.520" where every other instance of that header in the entire document reads "Notes on" — a one-off typo in the DAM's own source formatting, confirmed by checking all 22 instances of the header document-wide (21 say "on", exactly 1 says "to") — and the section-boundary detector, which only recognized "on", consequently never closed that section and

kept absorbing every following line, including the page footer, into the activity's title. The second, affecting the remaining cases, was not OCR noise at all: it was pdfplumber's own word-extraction heuristic merging characters from two different, visually overlapping physical lines of real text into corrupted single tokens — confirmed by re-grouping the raw characters by row directly (bypassing pdfplumber's own word-boundary logic entirely) and finding the real, uncorrupted two-line title underneath. The fix built a character-level row reconstruction used only as a targeted fallback when a title is flagged as corrupted, deliberately scoped narrowly rather than replacing the word-based pipeline everywhere, and reduced the known corruption count from six titles to one, with the residual case and the reasoning behind not chasing it further documented honestly rather than claimed as fully resolved.

Chapter 7

Data Preparation

7.1 Parsing pipeline overview

The raw PDF is turned into structured data in three stages, implemented across the `parsing/` and `modeling/` packages: (1) group raw words/characters into physical rows and then into task blocks (`parsing/rows.py`, `parsing/task_blocks.py`); (2) enrich each task block with a cleaned title, extracted references, and geometry-resolved actions (`parsing/metadata.py`, `parsing/action_geometry.py`, `parsing/column_roles.py`); (3) assemble the enriched blocks into the project's own typed schema (`schema/schema.py`) and merge in the chapter/process skeleton (`modeling/build_nodes.py`, `modeling/hierarchy_skeleton.py`).

7.2 Title and boundary detection

A task's title can wrap onto a line that physically appears after its own action-code row, immediately before the `*next*` task's bare identifier — which a naive parser reads as belonging to the next task. The correct rule, arrived at only after two earlier attempts failed on real counter-examples, treats a task as "closed" (safe to forward-attach a following title line to the `*next*` task instead) only when one of three independent signals is present: a trailing period on the currently open task's text, or a second batch of action codes arriving while the current task already has some of its own. Each of the three signals was added in response to one specific, confirmed real failure (tasks 2.126, and 3.225/3.226), not designed upfront.

7.3 Reference and footnote cleaning

Cross-reference phrases ("See DAM 16.100, 16.200...") and footnote-qualification digits share the same superficial shape — bare numbers — as ordinary content, so an early version of the footnote-stripping regex was corrupting genuine reference ids and genuine dollar/UA amounts (e.g. "2,000,000" became ".,,"; "a 3-Year Rolling Business Plan" lost its "3"). The eventual, tested rule excludes a bare number from footnote-stripping whenever it is immediately preceded `*or*` followed by a comma or hyphen

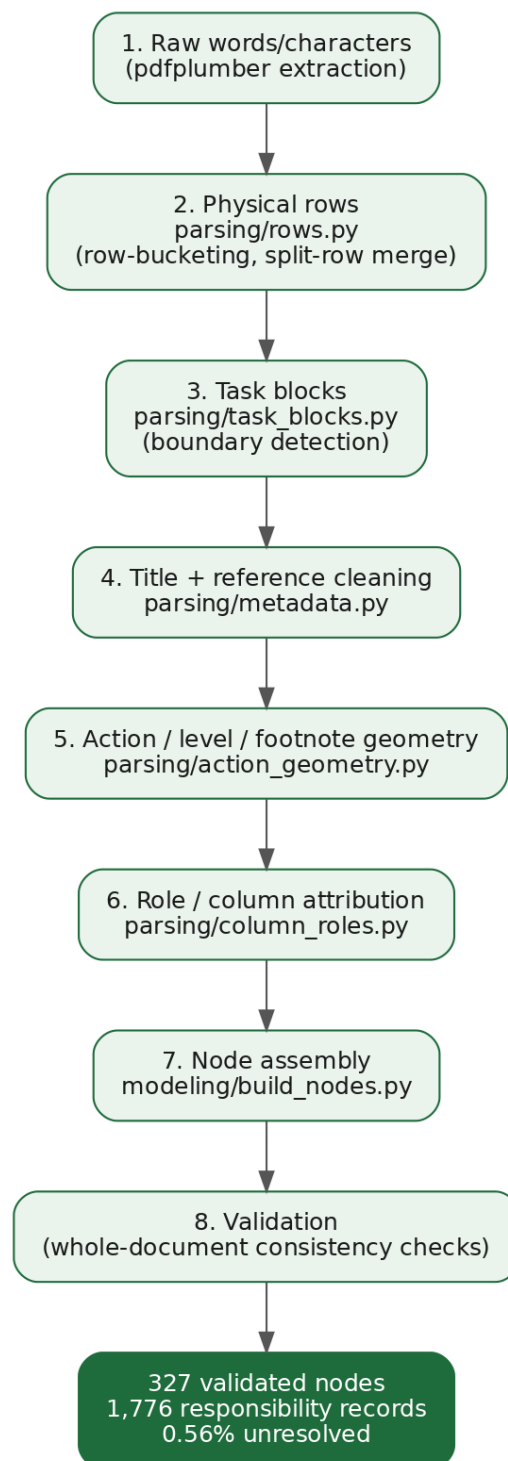


Figure 7.1: The eight-stage data preparation pipeline, from raw PDF extraction to a validated node set.

— arrived at incrementally as each new false-positive shape was discovered on real titles, and validated in `tests/test_metadata.py` against every previously-fixed case simultaneously so a new fix could never silently reintroduce an old bug.

References themselves are extracted through four distinct regex patterns, one per real phrasing found in the document (Section 6.2), each wired into both the extraction path (so the redirect is captured in `node.references`) and the stripping path (so the same phrase does not simultaneously get mangled by the footnote-digit stripper) — the same two-sided pattern applied consistently as each new phrasing was found.

7.4 Action, level, and footnote geometry

Building on the alignment rule established in Data Understanding (Section 6.2), `parsing/action_geometry.py` compares each small superscript character following an authority-code letter to that letter's own top and bottom edge: a digit aligned to the letter's bottom is a seniority-level indicator (at most one per code); consecutive digits aligned to the letter's top concatenate into a multi-digit footnote reference number. Anchor detection (finding the authority-code letter itself among all the small text on a page) went through three iterations to correctly exclude OCR-style letter-spaced prose ("A p p r a i s a l") and isolated single letters embedded in unrelated words, each iteration validated against a specific real false-positive found on document page 12.

The "(i)" informed-marker bug (Section 6.2) — a five-character sequence including internal spaces, originally scanned for as three consecutive non-space characters — was the single fix with the largest measured impact in the entire parsing pipeline: it had silently prevented 174 of 276 informed-party markers in the previous version of this project from ever resolving to a real role.

7.5 Role/column attribution

Rotated column headers are read in descending-top character order and clustered in two passes: a tight pass ($\text{gap} \leq 3\text{pt}$) groups each physical vertical text run in the correct reading order first, then a wider pass ($\text{gap} \leq 14\text{pt}$, calibrated against real measured gaps: wrapped lines of the same header sit roughly 10–12pt apart, while distinct columns sit roughly 19–27pt apart) concatenates runs into full header strings. Skipping the tight pass and going straight to the wide pass was tried and rejected, because it interleaves two headers character-by-character when their y-ranges happen to overlap (a real, confirmed failure mode, not a hypothetical one).

Once headers are extracted, each authority-code cell is matched to the nearest column header by x-position (maximum 20pt distance; no match within that distance

returns "unresolved" rather than a guessed wrong role). Across the full 79-page document this resolves 98.8% of actions to a role on pages where headers exist; 43 of 79 pages have no rotated header text at all, of which 42 are legitimately non-table pages and exactly one is a genuine multi-page table continuation, handled by carrying the previous page's header set forward.

Two further real bugs were found and fixed only by checking results across the *whole* document rather than a single sample page: character interleaving between two columns sitting closer together ($\approx 2\text{pt}$) than the tight-cluster threshold on one specific page, and a false merge of two genuinely different columns caused by that same page's unusually tight spacing defeating a purely gap-based merge rule. The second was fixed with a content-based signal — stop merging once accumulated header text already ends in a balanced closing parenthesis — checked against all 79 pages before being trusted, since it changed output on only the four pages that needed it and no others.

Footnote digits trailing directly on role-header strings (e.g. "Regional NSO Lead2") were stripped only after three attempts: the first never fired at all (it checked the literal last character, which is almost always a positioned trailing space, not the digit); the second stripped only one digit of a genuine multi-digit footnote number; the third correctly excludes legitimate department codes that also end in a digit ("Manager FIFC.4") by checking that the character immediately preceding the digit run is not a literal period.

7.6 Node assembly and validation

`modeling/build_nodes.py` performs one page-walk producing a fully enriched, typed `schema.Node` for every task, child task, and threshold variant, merged with a separate chapter/process skeleton (`modeling/hierarchy_skeleton.py`) for the two hierarchy levels the row-level parser never directly emits. Validation was not a one-time check but a repeated, whole-document consistency pass: a `get_children()` bug that silently returned wrong results for two of the four hierarchy levels (because it guessed parent/child relationships from id-string prefixes, which happens to work for two levels and is simply wrong for the other two) was caught only by a consistency check requiring every node's children to be of the expected type and every child's parent-pointer to point back correctly — not by manual inspection, which had missed it.

The final, current build produces **327 nodes** (3 chapters, 35 processes, 138 tasks, 102 child tasks, 49 threshold variants) and **1,776 extracted responsibility records**, with an unresolved-role rate of **0.56%** (10 of 1,786 gross action instances) and 3

residual exact-duplicate responsibilities, all localized to one page with unusually tight column spacing and left as a small, understood residual rather than chased further. Two pre-existing empty titles (activities 2.516 and 2.522) and one residual title-corruption case (2.222.1, down from six before the character-level title-recovery fix) remain, both explicitly documented as known, bounded data-quality limitations rather than silently hidden.

7.7 A chain of six real bugs from one scoping question

Before starting work on the knowledge graph, a scoping question was raised that could have been treated as a quick sanity check: role-header strings carry a trailing footnote digit ("Regional NSO Lead2"), and action cells carry their own footnote digits — is that the same footnote numbering pool, or two separate ones? Answering it properly, against real document evidence rather than by assumption, opened a chain of six further real bugs, each found and fixed only because the previous one was checked thoroughly enough to expose the next. They are narrated here in the order found, because the chain itself — not just the individual fixes — illustrates how deceptively deep a single "quick check" question could go in this document.

1. The footnote pools were confirmed to be the same one. Checking a specific real table (the NSO section) directly confirmed that a role header's trailing footnote digit and an action cell's trailing footnote digit reference the same numbered list at the bottom of the table — one shared numbering pool per table, not two independent ones.

2. A hard alignment-tolerance cutoff was silently dropping real footnote digits. While confirming point 1, one specific authority code's footnote digit was found to sit 0.004 points past the alignment-tolerance cutoff used to distinguish a footnote digit from a seniority-level digit (Section 6.2's alignment rule) — close enough that it satisfied neither rule and was silently dropped rather than misclassified. Checked document-wide rather than patched for the one case: 30 of 345 digit decorations were being dropped this way, 23 sharing the exact same near-miss signature and 7 genuinely unrelated. The tolerance was widened from 1.5 to 2.0 points, and the two original real examples that had calibrated the 1.5 figure in the first place were re-checked to confirm they remained correctly classified under the new, looser threshold.

3. A genuine PDF row-splitting quirk, confirmed by a requested screenshot. Investigating a related discrepancy required requesting and inspecting a screenshot of the real table, which revealed that a wide table row's own approval-code cells could render on a **different** row-bucket than its own identifier cell — a real PDF

layout quirk where the rightmost cells of a wide row land roughly a point away from the leftmost cells, enough to round into a different row grouping. This was systematic across the entire table, not an isolated case, cascading each row's action codes one position off from where they belonged. A first, broad fix (merge any two row-buckets within five points that share no overlapping word x-ranges) was tried and rejected after being checked against the whole document: it produced 1,614 candidate merges, several of them unsafe, including one case that would have spliced two unrelated activities' text together. The eventual fix used a narrower, safer signal instead — only merge a row-bucket into its neighbor if the row consists *entirely* of action-code content, since no genuine standalone table row is ever composed of nothing but authority codes.

4. A bug in the fix for point 3, caught while testing point 3. The new "is this row pure action-code content" check used a pattern-matching function that could not correctly recognize codes like "I2" or "I3" due to a subtle word-boundary mismatch between a letter and an immediately following digit — a normalization step that already existed elsewhere in the codebase for exactly this reason was not applied consistently in the new check, causing it to silently skip merging rows it should have merged. The fix was to reuse the existing, already-correct normalization function rather than write a second, independently-maintained version of the same logic — the same single-source-of-truth lesson the project had already learned once (Section 6.3), this time self-inflicted rather than found in the source PDF.

5. A second occurrence of the comma-adjacency footnote bug, this time via a hyphen. Fixing point 3 surfaced a real activity title for the first time that had previously been hidden behind the row-splitting bug, and it was corrupted: "Concerned Staff members below PL-2 level" had lost its digit, rendering as "...below PL-level". The cause was a near-miss of a bug already fixed once before (Section 6.3's comma-adjacency exclusion for footnote-digit stripping): the existing rule excluded a bare digit immediately preceded by a comma or hyphen, but did not exclude one immediately *followed* by a hyphen, and "PL-2" has exactly that shape. Checked document-wide before extending the rule (six affected blocks found, five genuinely fixed by the extension, one pre-existing unrelated garbling left untouched) rather than assumed to be an isolated case.

6. A geometry search-window bug, caught only while writing the regression test for point 3. After the row-merge fix from point 3, the merged rows' *text* was correct, but their *authority-code actions* had vanished entirely. The cause traced to a helper function that reported a row-bucket's vertical span as a single repeated point value rather than the true minimum and maximum vertical extent of the words actually inside it — harmless before the row-merge fix existed, since an unmerged

row's words were always within about a point of that single value anyway, but no longer harmless once a merged row could legitimately span several points, which then fell outside the small fixed search window the geometry-based action extractor used to look for authority codes near a given row. The fix made the helper report the row's genuine vertical span, correct for merged rows and a no-op for the unmerged case.

Following this chain, a full re-test confirmed all pre-existing tests still passed, five new regression tests were added pinning the exact values recovered from the requested screenshot, and a full document rebuild showed total extracted responsibilities rising from 937 to 1,776 — the row-splitting fix alone recovering a substantial number of real actions that had previously been silently dropped or misattributed across roughly two dozen affected rows document-wide.

Chapter 8

Modeling

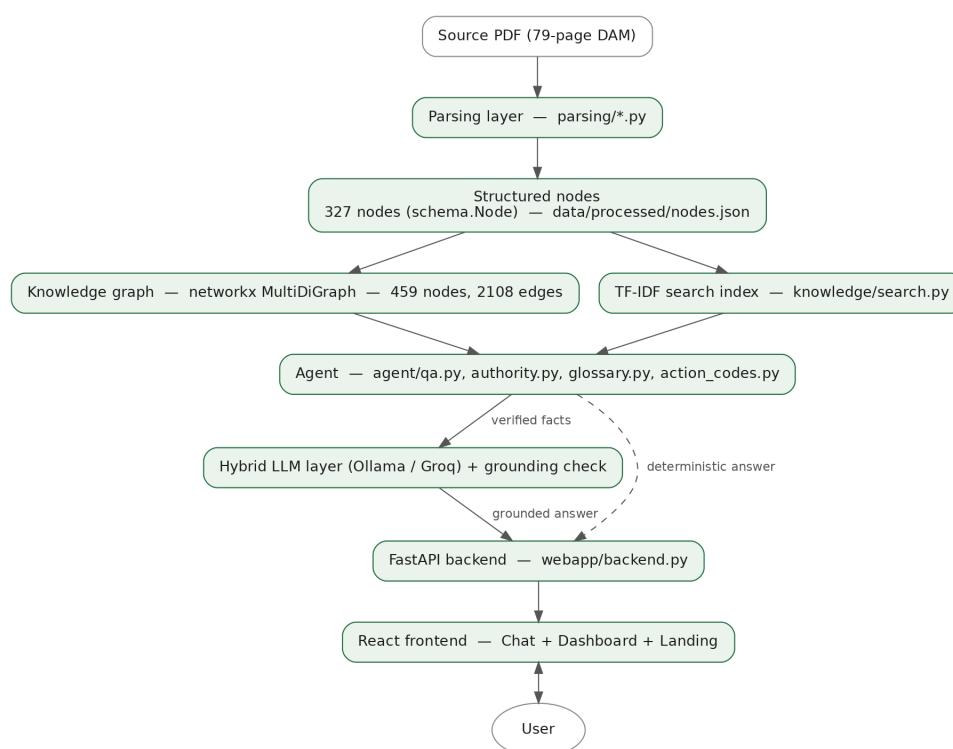


Figure 8.1: End-to-end system architecture: source document to user-facing answer.

8.1 Knowledge graph

The structured node set is loaded into a `networkx MultiDiGraph` (`modeling/graph.py`) rather than queried as a flat dictionary, because the project's own founding questions ("who approves X", "who is informed about X") are naturally graph queries — "who is connected to X by a particular labeled edge" — and because a role like "Country Manager / DDG" is a real, recurring entity referenced from dozens of different activities and deserves to be one graph node with one identity, not a string repeated independently in 168 different responsibility lists. A `MultiDiGraph` (allowing parallel edges) was chosen specifically because one role can hold two different responsibilities on the same activity (e.g. it both checks and later approves), which is two parallel edges, not one edge carrying two labels.

Every DAM node keeps its own id; every distinct canonical role string becomes its own node, namespaced (`role::...`) so it can never collide with a real DAM id; unresolved responsibilities are attached to one shared placeholder node (`role::__unresolved__`) rather than silently dropped, so a query can still surface that something could not be attributed. Three edge types connect the graph: `contains` (hierarchy), `references` (cross-activity redirects, wired only when the target id actually exists inside this document's scope — several real redirects point outside the 79-page document and are correctly left unwired rather than pointed at nothing), and `responsible_for` (role $\rightarrow$ activity, carrying the authority code, seniority level, and footnote reference as edge attributes).

The current graph has **459 nodes** (327 DAM nodes plus 131 distinct role nodes, including the unresolved placeholder) and **2,108 edges**. It was validated not just by edge count but by reproducing a specific real screenshot exactly: querying the graph for activity 3.111's approvers returns precisely "Origination Sector Manager" (footnote 3) and "Supporting Dept. Division Manager" (footnote 4) — the same two names, in the same order, with the same footnote numbers, as the actual printed table.

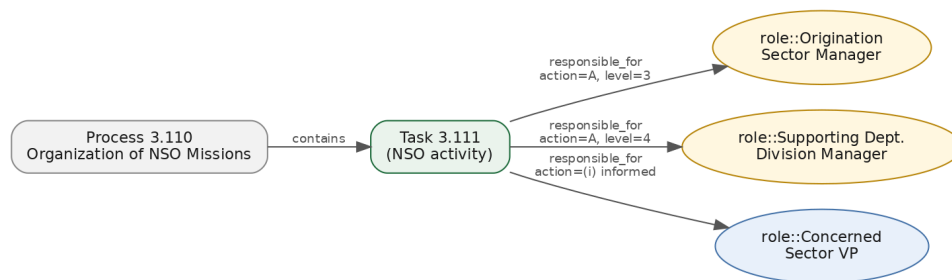


Figure 8.2: A real excerpt of the knowledge graph around activity 3.111, showing the `contains` edge from its parent process and the `responsible_for` edges to its approving and informed roles — the exact structure validated against the real printed table.

8.2 Natural-language understanding

Two independent resolution paths handle an incoming question (`knowledge/search.py`): an id fast-path that looks for a literal DAM identifier already present in the question text (100% accurate when it fires, and covers the project's own founding example questions, which are typically phrased with an explicit id), and a TF-IDF lexical fallback for questions phrased by title or description rather than by code.

A separate module, `agent/authority.py`, maps how people actually phrase authority questions ("who signs off on X", "who checks X") onto the DAM's own

authority-code structure, reading the code/level definitions directly from the extracted reference data (`data/reference/authority_codes.json`) rather than a handwritten vocabulary that could silently drift out of sync with it. The "C" ambiguity found in Data Understanding (Section 6.2) is encoded directly into each intent's matching predicate and covered by a dedicated regression test, so a "consult" question can never be answered with Check/verify facts or vice versa.

On top of this core resolution layer, several real user-experience gaps were found and fixed only through direct testing against real (and sometimes intentionally malformed) questions: plain conversational messages ("hi", "thanks") are recognized and answered directly rather than run through DAM lookup at all; a question containing an id-shaped but non-existent code is explicitly reported as invalid (with real, relevant suggestions) rather than silently falling through to a full-text search that can coincidentally match an unrelated real activity; genuinely vague questions ("mission", "approve") trigger a clarification response listing the top real candidates rather than guessing; and a deliberately conservative deterministic typo-correction layer (`knowledge/typo_correct.py`, threshold-tuned against real observed typos and real observed false positives, e.g. "unrelated" almost incorrectly matching "related") tolerates ordinary misspellings without ever touching the underlying lexical-matching logic itself.

8.3 Why TF-IDF over trained embeddings

With only 327 short titles and no labeled query/title pairs available, training a custom embedding model was ruled out outright: there is nowhere near enough data to beat a general-purpose pretrained embedding model, and doing so anyway would risk visible overfitting rather than a real capability gain — "there was not enough data to justify it" is judged the more defensible answer to give a reviewer than attempting it regardless. Installing a pretrained sentence-embedding library (`sentence-transformers`, which requires PyTorch) was attempted as the alternative and failed to complete twice in the development sandbox due to the size of the dependency — a real, if environment-specific, signal about deployment weight. TF-IDF via `scikit-learn` was chosen for reliability and portability (installs in roughly 17 seconds, no heavy dependency, runs anywhere).

This choice is explicitly disclosed as lexical (shared-word) rather than semantic (shared-meaning) matching throughout the system: every resolved answer carries a `method` field ("id" or "text_search") so this distinction is visible to any caller, rather than the system silently implying an understanding of meaning it does not have. The consequence — a question phrased with different words than the actual title (e.g.

"sign off on" versus "Signature of") scores lower than an exact phrase match — is documented as a known, accepted limitation of this design choice, not treated as a bug to be patched away.

8.4 Grounded generation: how the LLM is allowed to touch the answer

This is the architectural decision most directly traceable back to the Business Understanding success criteria (Section 4.3), and the one most worth explaining carefully, because "integrate an LLM" or "add RAG" can mean two very different things. One option — the textbook interpretation of retrieval-augmented generation — would chunk the source document, embed it, and let an LLM freely reason over whatever text gets retrieved. This was explicitly rejected: it would have quietly undone the entire reason the project exists, since the whole point of building a structured knowledge graph in the first place was to avoid an LLM hallucinating who approves a financial transaction by grounding every answer in a verified data structure instead of raw document search.

The chosen design keeps retrieval exactly as validated in Sections 8.1–8.2 (deterministic, checked against real screenshots) and restricts the LLM to a strictly narrower job: rephrasing facts that retrieval has already found, never deciding what those facts are. `agent/generate.py` builds a system prompt instructing the model to use only the facts supplied and to say so plainly if the fact list is empty, then checks its output against those same facts with a grounding function (`_mentions_expected_facts`) that requires every role name to survive, verbatim (normalized only for whitespace and case, never abbreviated or paraphrased), in the model's rephrasing. If any name is dropped or altered, the system discards the LLM's answer entirely and falls back to the original deterministic template answer — never a partially-trusted hybrid. Two cases never reach the LLM at all: no provider configured, and no real activity resolved (nothing to phrase, which would only invite the model to invent something plausible-sounding).

This is still legitimately retrieval-augmented generation — the retrieval step existed, was validated, and remains completely unaffected by the LLM's presence; only the generation half of RAG changed.

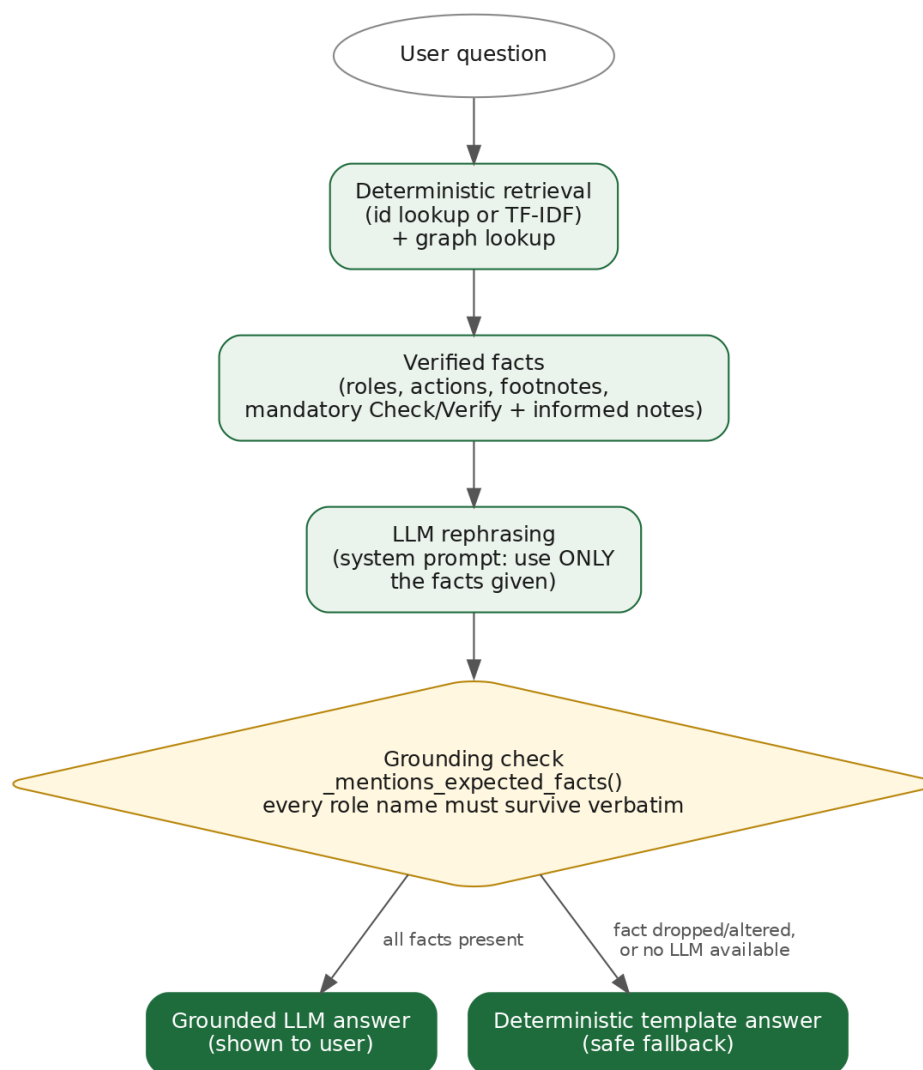


Figure 8.3: The grounded-generation flow: an LLM may only rephrase already-verified facts, and any answer that drops or alters a fact is discarded in favor of the safe deterministic template.

8.5 Hybrid local/cloud LLM layer

A common interface (`llm/base.py`, one `chat()` method and one shared failure exception) sits in front of two interchangeable providers: a local Ollama server (default model `llama3.1`, no network dependency, no cost) and Groq's cloud API (originally `llama-3.3-70b-versatile`, migrated in September 2026 to `openai/gpt-oss-120b` after the original model was moved to enterprise-only pricing — see Section 10.5). A router resolves a requested mode ("`off`" / "`ollama`" / "`groq`" / "`auto`") to a concrete provider; "`auto`" mode was initially set to prefer the free local option and fall back to the cloud API only if the local server was unreachable, then later reversed at the intern's request to prefer the cloud API first (faster, no local setup required for a demo audience) with local as the fallback if the API key is unavailable — a real example of a design choice being revisited once actual usage patterns (a live demo, not a development machine) made the original assumption wrong.

8.6 Recent modeling extensions: multi-language, mandatory notes, tone

Three further modeling-layer capabilities were added in direct response to explicit requests after the initial system was demonstrated (see Section 10.6 for full detail on each, since they were shipped as part of deployment-stage iteration): translating questions and answers across five languages while keeping the underlying English retrieval and grounding check completely unchanged; treating a DAM row's mandatory Check/Verify authority as a fact the agent must always surface even when not directly asked, rather than optional supplementary detail, following an explicit domain-knowledge correction from the intern; and a cheap, pre-filtered LLM classification step that detects a frustrated or confused user and prepends a short, deterministic, language-matched empathetic opener to any response type — never a second free-form generation, so the added warmth carries none of the factual risk the core grounding check exists to prevent.

Chapter 9

Evaluation

9.1 Evaluation philosophy

The project treats "the answer is factually correct" and "the code behaves as intended" as two separate, both mandatory, evaluation questions, and pursues both continuously rather than as a single end-of-project test pass. Data-layer correctness (Sections 6–7) was evaluated primarily against the real source document itself — a specific screenshot, a specific page's raw characters, or a specific confirmed real user question — never against a synthetic or assumed example. Code-layer correctness is evaluated with a large, continuously growing automated test suite that mirrors that same discipline: a new test is written to reproduce a real confirmed bug before it is fixed, not written speculatively.

9.2 Automated test suite

As of this report, the project has **374 automated tests across 23 test files**, spanning parsing (`test_task_blocks.py`, `test_row_merge.py`, `test_column_roles.py`, `test_title_corruption.py`, `test_metadata.py`, `test_glossary_parsing.py`), the built data model (`test_build_nodes.py`, `test_graph.py`, `test_nodes_cache.py`), the natural-language layer (`test_search.py`, `test_typo_correct.py`, `test_smalltalk.py`, `test_action_codes.py`, `test_glossary_agent.py`, `test_agent.py`), the LLM and generation layer (`test_llm.py`, `test_generate.py`, `test_translate.py`, `test_tone.py`), and the web application layer (`test_backend.py`, `test_dashboard_data.py`, `test_frontend_source.py`). The whole codebase (excluding the retained v1 backup) is roughly 7,700 lines of Python.

Every test file traces back to a real, dated entry in the project's decision log explaining exactly what real-world case it exists to pin down — several tests are named directly after the specific real question or screenshot that exposed the bug they now guard against (for example, a regression test verifying that the follow-up question "what's I, A and (i)?" is never swallowed by conversation-context carryover again).

9.3 Data-model evaluation metrics

Measured directly against the current build (data/processed/nodes.json, live-recomputed for this report rather than quoted from memory):

Metric	Value
Total nodes	327 (3 chapters, 35 processes, 138 tasks, 102 child tasks, 49 threshold variants)
Total extracted responsibility records	1,776
Unresolved-role rate	0.56% (10 records)
Distinct canonical roles	131
Average responsibilities per node	5.48
Nodes with no direct responsibilities	96 (29.6% — mostly process-level nodes, whose responsibilities live on their child tasks by design)
Knowledge graph nodes	459 (327 DAM + 131 role + 1 unresolved placeholder)
Knowledge graph edges	2,108
Known residual data-quality issues	2 empty titles, 1 residual title-corruption case, 3 duplicate responsibilities (all understood, bounded, and documented rather than hidden)

9.4 Correctness under generation: the grounding check in practice

The grounding check described in Section 8.4 is itself evaluated, not just trusted: dedicated tests confirm it correctly accepts an LLM rephrasing that preserves every fact, correctly rejects one that drops or alters a role name, tolerates purely cosmetic differences (extra whitespace, different capitalization) without weakening the underlying guarantee, and correctly falls back to the deterministic template on a provider exception or an empty response. A late-project investigation into why the "LLM unavailable" fallback was firing more often than expected found the true cause to be a genuine tension between two features shipped the same day — a new domain rule

that increased the average number of facts an answer must state, and a sentence-count limit that had never been revisited to account for it — and fixed it by relaxing the sentence limit specifically for many-fact answers while leaving the verbatim-accuracy requirement completely untouched. A separate, unrelated candidate fix attempted during the same investigation (a parsing-layer change intended to fix one specific garbled role name) was tested, found not to fix its target case, found to introduce a real regression elsewhere, and was fully reverted — documented in the decision log as an honest record of an investigated-and-abandoned path rather than silently discarded.

9.5 Live verification against the real system

Because the development sandbox cannot run a persistent background server across separate tool calls, and cannot reach the live LLM API network endpoint at all, two complementary verification strategies were used throughout: FastAPI’s in-process `TestClient` (used for all continuous, repeatable testing, including every test in `test_backend.py`), and, for milestones that needed a genuine end-to-end confirmation, starting a real server and exercising it with real HTTP requests within a single shell invocation. Both of the project’s founding example questions were confirmed to return identical, correct answers through the real HTTP API, one resolved via the `id` fast-path and one via TF-IDF text search — the specific proof that the two-path resolution design in Section 8.2 actually delivers on its purpose, not just that it compiles.

Genuine external verification (a real Ollama server, a real Groq API key, a real Docker build, a real Render deployment) could not be performed from within the development sandbox at all in several cases — no Docker binary was ever available in the sandbox, and outbound requests to the Groq API endpoint were proxy-blocked for most of the project’s second half. Every such gap is explicitly flagged in the decision log rather than silently assumed to be fine, and was instead independently confirmed later by the intern on his own machine (Docker Compose, for example, worked correctly on first attempt once actually tested outside the sandbox).

9.6 Recurring bug pattern: found by real testing, not by inspection

A consistent pattern across the whole project is that the most consequential bugs were found by testing against real data or a real screenshot, not by reading the

code. Notable examples, spanning the full project timeline: the "(i)" informed-marker bug (missed by every prior version of this project, found only by scanning real raw characters); the `get_children()` hierarchy bug (invisible to inspection, caught only by an automated whole-document consistency check); a chat follow-up question resolving to the wrong activity (defeated an initial phrase-based fix within one further message from the intern, and was only correctly solved once the actual TF-IDF confidence score — not the wording of the question — was identified as the real signal); and the Groq 404 error on the deployed instance (traced to a model deprecation the intern discovered by hitting a genuine production error, not by proactive monitoring). Each of these is documented in this report's earlier sections at the point where it occurred, and together they are the strongest evidence that the "verify against real data before trusting a fix" discipline stated at the start of this report was actually followed in practice rather than merely stated as a principle.

Chapter 10

Deployment

10.1 Hosting

The system is deployed as a Docker container on Render's free tier (750 hours/-month, 512MB RAM, no credit card required, roughly 30–60 second cold-start after 15 minutes of inactivity). This platform was chosen only after a genuinely thorough comparison, redone twice as circumstances changed: Railway, Fly.io, Hugging Face Spaces, Back4App, and Google Cloud Run were each evaluated and ruled out for concrete reasons (a gutted free-credit model, a mandatory credit card, Docker support behind a paywall, insufficient free RAM, and mandatory account/billing setup, respectively); Vercel, Netlify, GitHub Pages, and Cloudflare were separately ruled out because they are serverless/static platforms that cannot run this project's Dockerized, stateful backend at all; and Koyeb, initially a strong secondary candidate, was dropped after its mid-acquisition by Mistral AI was confirmed via its own public announcement — judged too great a stability risk to build on immediately before a graded deadline.

The first real deployment attempt failed with an out-of-memory kill on Render's 512MB limit. The root cause was that the backend re-parsed the entire 79-page PDF from scratch (a roughly 12.7-second, memory-heavy operation) on every process startup. The fix moves this work out of the request path entirely: a pre-built, validated node cache (`data/processed/nodes.json`) is baked into the Docker image at build time, and the backend loads that cache at startup, falling back to a live parse only if no cache is present — verified to be perfectly lossless (zero mismatches across all 327 nodes) before being trusted.

Because the local Ollama option requires a locally-running model with real RAM that no free hosting tier provides, the hosted deployment runs Groq-only; the local option remains fully available and tested for anyone running the project on their own machine via Docker Compose.

10.2 Containerization

The project ships as a two-stage Docker build: a `node:20-slim` stage builds the React frontend, and the final `python:3.11-slim` stage contains only the built static assets and the Python backend — Node itself never ships in the final image. `docker-compose.yml` defines an app service and, for local development, an ollama service on a shared network; the Groq option needs no local container at all. Configuration is entirely environment-variable driven (`.env`, excluded from both git and the Docker build context) with a documented `.env.example` template, so no API key is ever at risk of being committed or baked into an image layer.

10.3 Web application

A thin FastAPI backend (`webapp/backend.py`) wraps the agent's `answer_question()` function with no independent business logic of its own — every real decision already lives in the modeling and agent layers described in Section 8. The frontend was rewritten partway through the project from static HTML/CSS/JS to React (via Vite) plus GSAP animation, an explicit trade-off the intern chose after being shown both options, accepting a new build-toolchain dependency in exchange for a more maintainable, componentized codebase. It ships as three independent single-page applications (landing, chat, dashboard) rather than one combined app, matching the backend's existing separate routes for each page.

The chat interface supports full multi-conversation history (a sidebar of past conversations persisted in the browser, in the style of a typical AI chat product), a live model-status indicator, a picker for which LLM mode to use, and a toggle to compare the LLM-phrased answer against the original deterministic template answer side by side — a deliberate transparency feature, not a debugging leftover, letting a user see exactly what changed between the verified facts and their conversational rephrasing. A companion analytics dashboard visualizes the DAM's own structure (node-type breakdown, authority-code distribution, most-referenced roles, unresolved-role rate) with a chapter filter and linked charts, built specifically to showcase the underlying data model rather than agent usage statistics.

10.4 Multi-language support

Following direct feedback that non-English speakers could not use the system, the agent now supports French, Spanish, Portuguese, and Arabic alongside English, via an explicit language picker in the chat interface (in addition to automatic detection

of the question’s own language). The architecture deliberately does not attempt to rebuild the underlying retrieval pipeline per language — doing so would mean re-validating years of accumulated, hard-won English-language parsing logic in four more languages with no additional labeled data. Instead, a non-English question is translated to English before it reaches retrieval (which remains completely unchanged and unaffected), and the final answer is translated into the requested language afterward. The grounding check described in Section 8.4 is extended, not weakened, for this case: it still requires the literal English role names to appear verbatim in the model’s output regardless of what language the surrounding sentence is written in, and any answer that fails this check falls back to the safe English template rather than showing an unverified translation. Arabic additionally triggers right-to-left layout in the chat interface. A small, fixed set of interface labels (roughly 15 strings) is hand-translated once into each language rather than translated live by the LLM on every use — deliberately, for speed, cost, and consistency, following the same reasoning applied elsewhere in the project to small, fixed vocabularies (Section 8.6’s tone-response prefixes use the identical pattern).

10.5 Operational incident: a live production bug and its resolution

In September 2026 the deployed instance began returning HTTP 404 errors from the Groq API on every LLM-mode request. Root-causing this against Groq’s own current documentation (rather than assumed prior knowledge, since API model availability changes over time) confirmed that the model in use, `llama-3.3-70b-versatile`, had been moved to enterprise-only pricing on 16 August 2026 and was no longer available on the free tier being used. The fix — migrating the default model to `openai/gpt-oss-120b`, Groq’s own documented free-tier replacement — was applied across the code, environment configuration, documentation, and tests. This incident is a useful, honest illustration of a real operational risk in any system that depends on a third-party model API: model availability and pricing tiers are not static, and a system built against one can silently break with no code change of its own.

10.6 Chatbot feature iteration following professor and user feedback

Following review by the internship supervisor, four further improvements were requested and delivered, each following the same discipline of confirming a real gap before building the fix:

- **Vague-question and out-of-scope detection.** A question too short or generic to confidently resolve (measured, for example, by content-word count after removing stopwords and by the score gap between the top match and its nearest competitor) now returns a clarification request listing real candidate activities rather than a low-confidence guess; a question genuinely unrelated to the DAM's subject matter is reported as out of scope rather than matched to an unrelated activity by incidental word overlap. Both are explicitly disclosed as best-effort heuristics, not true topic understanding — a small number of edge cases (e.g. "tell me a joke") are known to be misclassified, but the system's core honesty guarantee (never fabricating a false DAM answer) holds regardless.
- **Mandatory Check/Verify and informed-party notes.** Following an explicit domain-knowledge correction from the intern (drawn from direct internship experience, not derivable from the PDF's formatting alone) — that a Check/Verify entity on an activity's row is mandatory information, not optional supplementary detail — every intent-specific answer now automatically appends a note naming the mandatory checking/verifying role and, when present and relevant, the informed parties, regardless of which specific authority action was originally asked about. This was scoped, after discussion, to apply to every intent uniformly (a property of the activity itself, not of the question asked) and to be applied deterministically and consistently rather than only "when it seems useful" — chosen specifically because a consistent, testable rule is more trustworthy than a heuristic that might silently omit the note on an unusual phrasing.
- **Action-code legend explanation.** A question asking what the authority-code letters themselves mean (e.g. "what's I, A and (i)?") was, for a period, incorrectly swallowed by the conversational-context-carryover logic (a short question containing only single letters looked, by an internal word-count heuristic, indistinguishable from a genuinely vague follow-up). The fix adds a dedicated detection path that runs before context-carryover is ever considered, reusing the DAM's own already-extracted authority-code reference data rather than a separately hand-maintained explanation.

- **Emotional tone detection.** Every response, regardless of whether it is a factual DAM lookup, a glossary answer, or an honest refusal, is now checked against a cheap, deterministic pre-filter for signs of user frustration or confusion (repeated punctuation, all-caps words, characteristic phrases); only messages that pass this filter trigger a single, cheap LLM classification call, and only that call's result (never free-form LLM text) determines whether a short, deterministic, language-matched empathetic opener is prepended to the answer. This keeps the added warmth completely outside the factual grounding-check boundary — it can never itself introduce an ungrounded claim, by construction.

10.7 Live status and reliability signaling

The web interface polls the backend's health endpoint continuously and shows a real-time status indicator (rather than assuming the backend is always reachable), and every answer's metadata (which LLM mode actually produced it, whether translation succeeded, whether the answer fell back to the deterministic template) is surfaced directly in the UI rather than hidden — a deliberate transparency choice consistent with the project's overall stance that an honest "this didn't fully work" is always preferable to a confident but unverifiable answer.

10.8 The hosting-platform search, narrated in full

Section 10.1 summarizes the final hosting decision; the process of reaching it is worth narrating because it was genuinely re-done twice as circumstances changed, not settled once and left alone. The first pass ruled out Railway (a free tier gutted to roughly one dollar of monthly credit, no longer practically free), Fly.io (requires a credit card even for its smallest tier), Hugging Face Spaces (Docker support is a paid feature), Google Cloud Run (requires a card and more account setup than the project's scope warranted), and Back4App (256MB of RAM, judged too tight for this application's needs). Render was selected from what remained.

A second pass was triggered later by further research into alternatives before the first Render deployment attempt had even been retried after its initial out-of-memory failure (Section 10.1). This pass considered Vercel (a serverless-functions platform whose "FastAPI preset" ignores a repository's own Dockerfile entirely, meaning neither the frontend build step nor the node-cache build step this project depends on would ever run, and whose cold-start-per-invocation model has no equivalent for this application's single in-memory server state), GitHub Pages (static files only, by design incapable of running any backend code at all), Netlify (the same serverless/Dockerfile

mismatch as Vercel), and Cloudflare (whose Containers product would actually fit, but sits behind a five-dollar-a-month paid plan with no free equivalent). Koyeb was, for a period, considered the strongest secondary candidate — architecturally the same Docker-native model as Render — until its dashboard was found to display a banner announcing its acquisition by Mistral AI and a mid-transition "stay tuned for a revamped experience" state instead of a normal service-creation flow; judged too great a stability risk to build a graded deliverable on top of during an active platform transition, this was set aside rather than pursued further. The search concluded, a second time, back at Render — the platform had been correct on the first pass, and the out-of-memory failure that had briefly cast doubt on it was a fixable bug in this project's own startup behavior (Section 10.1), not a genuine limitation of the platform itself.

10.9 Glossary and abbreviation lookup

Following direct testing feedback that questions like "what does DDG mean" returned a generic failure, the project extracted the DAM's own Abbreviations list (document pages 2–7, deliberately scoped separately from the longer, line-wrapping Glossary section on pages 8–11, which was set aside as a harder extraction problem not worth the time budget at that stage) into a structured reference file of 170 entries, and built a dedicated glossary-lookup path (`agent/glossary.py`) alongside inline acronym expansion within role names elsewhere in the system's answers — 19 of the DAM's 131 distinct canonical role names are themselves entries in this same abbreviations list. Two real parsing bugs were found and fixed while building the extractor: rows containing only a definition continuation (no new acronym on that line) were incorrectly being treated as starting a new entry, and page-footer roman numerals were leaking into the tail of a definition's text. A same-day follow-up round of direct testing found the glossary trigger phrase "what does X stand for" did not recognize the common variant "what does X stands for" (a minor grammatical slip a real user actually typed), and separately that "RDNG" — an acronym that appears throughout the DAM's own role names but, unlike "RDG", is never separately defined in the DAM's own printed Abbreviations pages — could not be resolved at all. Both were fixed: the trigger regex was widened, and a small, explicitly separate `MANUAL_ALIASES` dictionary was introduced for the handful of acronyms like this one, kept deliberately apart from the file that stores only what the DAM document itself actually defines, so the system's own added knowledge is never silently mixed with, or mistaken for, verified source-document content.

10.10 The React frontend rewrite, and a real sandbox environment limitation

The original frontend was static, build-free HTML/CSS/JavaScript. After showing both options and their trade-offs, the project was rewritten to React (via Vite) with GSAP animation — accepting a new Node-based build toolchain in exchange for a more maintainable, componentized codebase, an explicit trade-off decision rather than a default choice. The rewrite ships as three independent single-page applications (landing, chat, dashboard), matching the backend's existing separate routes for each page rather than consolidating into one combined app.

Building this inside the development sandbox surfaced a genuine, non-obvious environment problem: running `npm install` inside a folder synchronized by OneDrive was unworkably slow, because the cloud-sync layer intercepts and processes every one of the thousands of small files `node_modules` produces. The practical workaround, used for every subsequent frontend build in this project, was to run the install and build steps in an unsynchronized temporary directory and copy back only the finished, built output — never the intermediate `node_modules` tree itself. A second, related limitation was that browser-based UI testing (via Playwright) was attempted but blocked by the sandbox's outbound network allowlist, which prevented downloading a Chromium binary; the project's automated test coverage for the frontend was consequently redesigned around source-level structural checks (`tests/test_frontend_source.py`) rather than rendered-browser assertions, an honest adaptation to a real tooling constraint rather than an attempt to work around it invisibly.

Direct testing of the freshly rebuilt frontend surfaced five further real, user-visible gaps in a single round, each confirmed against a real screenshot before being fixed: a widened glossary-trigger regex was still missing the specific phrasing "what does RDG stands for" on the new build; "RDNG" needed the same manual-alias treatment as above; the chat page rendered its message column at the full page width with an ungainly gray gap on wide screens, fixed with an explicit centered, max-width inner container; there was no way to see or return to a previous conversation, addressed by building a full multi-conversation sidebar backed by browser local storage; and the agent's replies, while factually sound, read as flat and mechanical, addressed with varied smalltalk phrasing and a warmer (but still fact-grounded) system prompt for LLM-phrased answers.

10.11 Deterministic typo tolerance

Real testing surfaced a cluster of ordinary human typing mistakes the system could not handle: a misspelled "aproves", a misspelled "qaurterly mision", a truncated "helo", and a garbled "waht does DDG men" — each causing an otherwise-correct question to fail to resolve. Rather than route typo correction through the language model (which would make even this basic layer of the system dependent on network access and cost), a deterministic correction utility (`knowledge/typo_correct.py`) was built on Python's standard-library `difflib` similarity scoring, with two different similarity thresholds calibrated for two different situations: a looser threshold for small, curated vocabularies (smalltalk trigger words, glossary trigger words, intent-detection keywords), and a stricter threshold for the large, organic vocabulary of real DAM title words used in text search, where a looser threshold risks false corrections. That false-correction risk was not hypothetical: an early threshold setting was found to incorrectly "correct" the genuinely different word "unrelated" into "related" due to their high surface similarity, caught and fixed by raising the large-vocabulary threshold until real typo cases remained correctly matched while this specific false positive no longer was. Words shorter than four letters and all-capitalized words (likely to be genuine acronyms rather than misspellings) are deliberately excluded from correction, a conscious, documented trade-off rather than an oversight.

10.12 Conversational context carryover: a two-round fix

Section 8.6 references this fix; the two rounds it took to get right are worth narrating because the first, reasonable-seeming fix was defeated by a real follow-up message within the same testing session, which is itself a useful illustration of why every fix in this project was tested against real, adversarial-feeling input rather than a single happy-path example. Direct testing found that after resolving one activity and asking a follow-up question referencing "that activity" without repeating its name, the system resolved the follow-up to a completely different, unrelated activity that merely happened to share enough surface vocabulary to score above zero in text search. The first fix recognized a small set of explicit anaphoric trigger phrases ("that activity", "it", "this") and carried the previous activity's identity forward whenever one was detected. This fix was defeated on the very next real message: a follow-up phrased with a typo and no recognized trigger phrase ("and who are the informed partie?") landed on the exact same wrong, unrelated activity at an almost identical confidence score to the original failure — proof that the confidence score itself, not the specific wording of the follow-up, was the real signal that should have been used from the

start. The second, final fix discarded the phrase-list approach entirely and instead always resolves the fresh question first, only overriding with the previous turn's activity when the fresh resolution's own confidence score falls below a threshold calibrated against real measured examples of genuine matches versus real measured examples of coincidental low-confidence mismatches. This is explicitly documented as a sharper but still imperfect trade-off: a genuinely new question that happens to be weakly worded can still be swept into carryover, accepted because such a question would have been a low-confidence, borderline answer on its own regardless.

10.13 Mobile responsiveness and the dashboard rework

Following the React rewrite, the interface was adapted for small mobile viewports: a hamburger-triggered off-canvas conversation sidebar below a defined breakpoint, a responsive header that wraps its controls rather than overflowing, and message bubbles that use a percentage-based width on mobile instead of the fixed desktop width. Separately, following direct feedback from the internship supervisor requesting a more dynamic, interactive, and clearly relevant dashboard, the analytics page was substantially reworked: a chapter filter re-scopes every card and chart to a single chapter of the DAM (graph-level totals remain whole-document, since cross-chapter reference edges cannot be meaningfully attributed to one chapter alone), two new coverage metrics were added (average responsibilities per node, and the percentage of nodes with no directly recorded responsibilities — a genuine, previously invisible data-quality signal about the extraction itself, not a cosmetic addition), and clicking a bar on the authority-code distribution chart now filters the roles chart above it to show who is most active under that specific code. Building this rework caught one further real bug: a chart-rendering component had an empty effect-dependency array, meaning its charts silently never updated after their very first render regardless of new data — fixed by correcting the dependency list to include the actual data and options the chart depends on.

Chapter 11

User Manual and Deployment Guide

This chapter is a practical reference for anyone who wants to run, extend, or evaluate the system directly, rather than only read about it. It consolidates material from the project's own `docs/llm_setup.md`, `docs/hosting.md`, and `docs/docker_setup.md`.

11.1 Running the system with no setup at all

The system works with zero configuration: the "No LLM (template answer)" mode is the default, and it returns the same deterministic, graph-backed answers the project has used since its first working version. No API key, no local model, and no internet access beyond loading the page are required for this mode. Every other mode described below is an enhancement to answer phrasing and language support, never a requirement for correct, factual answers.

11.2 Local setup (bare metal)

1. Clone the repository and install Python dependencies: `pip install -r requirements.txt`.
2. Build the frontend once (and again after any frontend source change): `cd webapp/frontend && npm install && npm run build`.
3. Optionally copy `.env.example` to `.env` and fill in `GROQ_API_KEY` and/or `OLLAMA_HOST/OLLAMA_MODEL` to enable LLM-phrased answers.
4. Start the server: `python -m uvicorn webapp.backend:app -host 127.0.0.1 -port 8000`.
5. Open `http://127.0.0.1:8000/` for the landing page, `/chat` for the chat interface, and `/dashboard` for the analytics dashboard.

11.3 Local setup (Docker)

`docker compose up --build` builds and starts both the app service (this project's own code, including the frontend build step) and an `ollama` service for local LLM

phrasing, on a shared Docker network. No manual frontend build step is needed — the two-stage Dockerfile (Section 10.2) handles it as part of the image build. The Groq cloud option needs no additional container; setting `GROQ_API_KEY` in `.env` is sufficient.

11.4 Environment variables reference

Variable	Purpose	Default
<code>GROQ_API_KEY</code>	Authenticates requests to Groq's cloud API	none (Groq mode disabled without it)
<code>GROQ_MODEL</code>	Overrides the Groq model used	<code>openai/gpt-oss-120b</code>
<code>OLLAMA_HOST</code>	Base URL of a local Ollama server	<code>http://localhost:11434</code>
<code>OLLAMA_MODEL</code>	Overrides the local Ollama model used	<code>llama3.1</code>
<code>PORT</code>	Overrides the port uvicorn binds to (used by hosting platforms)	8000

11.5 Choosing an LLM mode

"Off" always returns the deterministic template answer with no phrasing pass. "Ollama (local)" requires a local Ollama installation with a pulled model, has no network dependency once the model is downloaded, and incurs no API cost. "Groq (API)" requires a free Groq account and API key, and responds in well under a second on the free tier. "Auto" prefers Groq first (as of the September 2026 reordering described in Section 8.5) and falls back to Ollama if no API key is configured, or to the deterministic template if neither is available — this mode never fails outright, only degrades.

11.6 Deploying to Render (free tier)

1. Push the repository to GitHub.
2. On Render: New → Web Service → connect the repository.
3. Environment: Docker (auto-detected from the root `Dockerfile`).

4. Health Check Path: `/api/health`.
5. Add the `GROQ_API_KEY` environment variable under the service's Environment tab.
6. Leave the port field untouched — Render injects its own `PORT`, which the Dockerfile's shell-form `CMD` already respects.
7. If redeploying after an earlier failed build, use Manual Deploy → Clear build cache and deploy, not a plain retry, since a plain retry can reuse cached layers from a previous failing build.
8. Allow a few minutes for the first build (npm install, Vite build, pip install, and the nodes-cache build step, per the Dockerfile's layer order).
9. Because the free tier sleeps after 15 minutes of inactivity, open the deployed URL a minute or two before a live demonstration to avoid the 30–60 second cold-start delay occurring in front of an audience.

11.7 API reference

See Annex H for the complete endpoint reference with example requests and responses.

11.8 Verifying an LLM mode is actually working

Ask the identical question under two different LLM modes and confirm the underlying facts (role names, authority codes, footnote numbers) never change, only the phrasing does. This is the direct, hands-on demonstration of the grounded-generation guarantee described in Section 8.4: retrieval is fixed and deterministic regardless of which LLM mode is active, or whether one is active at all.

Chapter 12

Project Management and Process

12.1 Timeline

The project ran over the course of the six-month internship, organized into two main phases rather than a fixed day-by-day schedule. The initial build phase established the system's foundation, in order: source selection and scope, schema design, parsing, normalization and validation, node construction, knowledge graph construction, the NLP/search layer, the agent, and the web backend. This was followed by a substantially longer second phase — driven by direct feedback from both the intern and the internship supervisors after the initial system was demonstrated — that added the hybrid LLM layer, grounded generation, the analytics dashboard, the React frontend rewrite, containerized deployment, hosting, multi-language support, and the four chatbot-quality features detailed in Section 10.6. In total, the system took a bit more than four months of active development to reach the state described in this report, with the iterative, decision-log-driven working method described below applied consistently across both phases.

12.2 Decision-logging discipline

Every scope decision, every confirmed bug, and every accepted trade-off in this project is recorded, dated, in a running decision log (`docs/decisions.md`) specifically so the reasoning behind the current system is traceable without needing to reconstruct it from memory or chat history — the primary source for this report itself, and the artifact this report's author considers the single most valuable piece of documentation the project produced, independent of the code.

12.3 Working method

Two working principles were applied consistently enough, across a genuinely large number of individually-documented bugs, to be considered defining characteristics of how this project was executed rather than incidental good habits: first, no fix was trusted until it had been checked against real data — a real screenshot, real

page characters, a real confirmed user question, or a whole-document consistency scan — rather than reasoned about in the abstract; second, when a fix attempt was tried and found not to work, or found to introduce a new regression, it was reverted and documented as an honest, explained dead end rather than kept because it looked plausible or silently discarded without a record. Section 9.4’s account of the abandoned parsing fix during the grounding-fallback investigation is one concrete instance of the second principle in practice, not the only one.

Chapter 13

Lessons Learned and Limitations

13.1 Technical lessons

- **PDF layout parsing is a geometry problem wearing a text-processing costume.** Nearly every hard bug in Sections 6–7 was, underneath, a question about character position and size that a text regex could only ever approximate — and every regex-only attempt at these problems eventually hit a real counter-example that only geometry could resolve correctly.
- **A single source of truth prevents entire classes of self-inflicted bugs.** Several fixes were only necessary because two different parts of the code implemented equivalent logic slightly differently (e.g. one action-code normalization step existing in two places, one of which was missed when the other was updated) — consolidating to one shared function, referenced from everywhere, was consistently the more durable fix.
- **Whole-document validation catches what sampling misses.** Bugs affecting 1–2% of activities were repeatedly invisible on a single sample page and only surfaced by checking every one of the 79 pages, or all 327 built nodes, at once.
- **Restricting an LLM’s role to verified rephrasing, rather than open-ended reasoning, is what makes an LLM usable at all in a compliance-sensitive context.** The grounding check (Section 8.4) is, in the author’s assessment, the single most important architectural decision in the project — every later feature (multi-language, mandatory notes, tone) was designed to extend the system’s capabilities without ever weakening that one guarantee.

13.2 Known limitations

- Two hierarchy-4 (`threshold_variant`) parsing gaps remain unresolved by design: fused level-and-footnote superscript digits that pdfplumber tokenizes inconsistently, and plain space-separated amounts inside titles (as opposed to comma-formatted ones), both flagged as needing true geometry-based digit detection rather than another text-pattern patch.

- Two garbled role-name titles (2 of 327 nodes) remain, after one candidate fix was tried, found ineffective, found to cause a regression, and reverted; the true, mid-run character-shrinking root cause is understood but not yet fixed.
- The action-geometry extractor's per-task action counts disagree with the trusted text-based counts on approximately 14% of activities; the text-based count is used as ground truth throughout the rest of the system, and this gap is accepted rather than chased further within the project's time budget.
- Text search is lexical, not semantic; a question phrased with entirely different vocabulary from the actual activity title will score lower than a closer phrase match, regardless of whether the underlying meaning matches.
- Vague-question and out-of-scope detection are heuristic, not a true topic classifier, and a small number of edge cases are known to be misclassified — though never in a way that produces a fabricated DAM answer.
- Several LLM-dependent features (the Groq model migration, the grounding-fallback-frequency fix, tone detection) could not be independently verified against a live call to the Groq API from within the development sandbox, due to a persistent outbound network restriction, and instead rest on mocked-provider testing plus the intern's own live verification outside the sandbox.
- The five-language interface translations were produced by the AI collaborator itself and have not yet been reviewed by a native speaker of French, Spanish, Portuguese, or Arabic — acceptable for a first release of interface labels, but flagged explicitly for review before any formal multilingual demonstration. The underlying DAM facts themselves are unaffected by this, since they remain subject to the same English-language grounding check regardless of display language.
- Render's free hosting tier sleeps after 15 minutes of inactivity, producing a 30–60 second cold-start delay on the first request after idle — a known and accepted trade-off of using a free hosting tier for a project of this scope.

General Conclusion

This thesis delivers a working, deployed, tested AI agent that answers questions about a real, complex compliance document with verified factual accuracy, built through a disciplined, evidence-driven engineering process from raw PDF extraction through to a hosted multilingual chat application. Every phase of the CRISP-DM cycle is represented not as a formality but as real, substantial engineering work: Data Understanding and Data Preparation in particular consumed the largest share of total effort, which is itself an honest and instructive finding about where the real difficulty lies in building a trustworthy AI system over a messy, real-world source document – not in the LLM integration, which by comparison was the more straightforward half of the project once the underlying data and retrieval layer could be fully trusted.

Natural next steps, in rough priority order based on where the project's own decision log flags the largest remaining gaps: resolving the two remaining `threshold_variant` geometry limitations with true character-level superscript detection; independently verifying the LLM-dependent fixes made in September 2026 against a real, unblocked Groq API connection; having the five-language interface translations reviewed by native speakers; and, longer-term, revisiting whether the accumulated real-query corpus (now large enough, after months of live testing, to plausibly constitute a small labeled dataset in its own right) changes the earlier conclusion that there was not enough data to justify a custom-trained retrieval model.

Beyond its immediate technical deliverable, this project is offered as a case study in a broader argument: that a large language model can be safely deployed against a compliance-sensitive institutional document, without accepting the hallucination risk normally associated with generative AI, provided the model is architecturally restricted to rephrasing already-verified facts rather than ever being trusted to retrieve or decide what those facts are. Every design decision in this thesis that mattered – the knowledge graph, the grounding check, the deterministic fallback – follows directly from taking that constraint seriously from the very first day of the project rather than treating it as an afterthought to be bolted on once a first version was already generating plausible-sounding answers.

Web References

All sources cited in this thesis are freely accessible online; they are listed together below as web references rather than split into a separate print bibliography, since no source used required access beyond a public URL.

1. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc. – the methodology framework structuring this thesis.
2. African Development Bank Group. *Delegation of Authority Matrix* (internal reference document, “updated dam file.pdf”, 79 pages). The project’s source data; an internal Bank document, not publicly published, referenced here only for the intern’s own project documentation.
3. pdfplumber documentation. *jsvine/pdfplumber* – PDF text, character, and table extraction library. <https://github.com/jsvine/pdfplumber>
4. scikit-learn documentation. *TfidfVectorizer* and cosine-similarity utilities. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
5. NetworkX documentation. *MultiDiGraph* – Directed graphs with self loops and parallel edges. <https://networkx.org/documentation/stable/reference/classes/multidigraph.html>
6. FastAPI documentation. <https://fastapi.tiangolo.com>
7. React documentation. <https://react.dev>
8. Vite documentation. <https://vitejs.dev>
9. GSAP (GreenSock Animation Platform) documentation. <https://gsap.com/docs/v3/>
10. Groq Inc. *GroqCloud Developer Documentation* – Rate Limits and Model Deprecations pages, consulted directly (not from training knowledge) to confirm current free-tier model availability. <https://console.groq.com/docs/rate-limits> and <https://console.groq.com/docs/deprecations>
11. Ollama documentation. *Local large language model runtime*. <https://ollama.com>

12. Docker documentation. *Dockerfile reference* and *Compose file reference*. <https://docs.docker.com>
13. Render documentation. *Free instance types* and *Deploying a Docker web service*. <https://render.com/docs>
14. Python difflib module documentation (standard library) – used for deterministic typo-tolerance matching. <https://docs.python.org/3/library/difflib.html>
15. Pydantic documentation. *Data validation using Python type hints*, used for the project's typed schema layer. <https://docs.pydantic.dev>

Appendix A

Automated Test Suite Breakdown

Test file	Tests	Covers
test_task_blocks.py	25	Task-boundary detection, title attachment, threshold-variant hierarchy
test_row_merge.py	5	Split-row merge fix (NSO table row-bucketing)
test_column_roles.py	8	Rotated-header extraction, role/column clustering
test_title_corruption.py	10	Character-level title recovery for cross-row word merging
test_metadata.py	11	Reference extraction, footnote-digit stripping rules
test_glossary_parsing.py	7	Abbreviations-list extraction
test_build_nodes.py	7	Node assembly, get_children() hierarchy correctness
test_graph.py	7	Knowledge graph construction and edge validation
test_nodes_cache.py	2	Lossless node-cache save/load round trip
test_search.py	8	Id fast-path and TF-IDF resolution

Test file	Tests	Covers
test_typo_correct.py	18	Deterministic typo-tolerance thresholds
test_smalltalk.py	40	Greeting/farewell/help detection
test_action_codes.py	20	Authority-code legend explanation, context-carryover exclusion
test_glossary_agent.py	32	Glossary lookup and in-line acronym expansion
test_agent.py	29	Core answer_question() orchestration, mandatory notes, context carry-over
test_llm.py	15	Provider mocking, router mode resolution, failure handling
test_generate.py	19	Grounding check correctness, sentence-cap scaling
test_translate.py	26	Multi-language detection and translation shim
test_tone.py	25	Tone pre-filter, classification parsing, prefix application
test_backend.py	29	End-to-end API behavior via TestClient
test_dashboard_data.py	15	Dashboard summary metrics and chapter scoping
test_frontend_source.py	16	React source-level structural checks
Total	374	across 23 files

Appendix B

Project Directory Structure

```

agent/          action_codes.py, authority.py, generate.py, glossary.py, qa.py,
    smalltalk.py
knowledge/      search.py, typo_correct.py
llm/           base.py, groq_provider.py, ollama_provider.py, router.py, tone.
    py, translate.py
modeling/       build_nodes.py, graph.py, hierarchy_skeleton.py, nodes_cache.py
parsing/       action_geometry.py, column_roles.py, glossary.py, hierarchy.py,
    metadata.py, rows.py, task_blocks.py, text_cleaning.py
schema/        schema.py
scripts/       build_nodes_cache.py
webapp/        backend.py, dashboard_data.py, static/ (built frontend),
    frontend/ (React source)
data/          raw/ (source PDF), processed/ (nodes.json cache), reference/ (
    authority codes, abbreviations)
docs/          decisions.md, code_notes.md, demo_script.md, hosting.md,
    docker_setup.md, llm_setup.md
tests/         23 files, 374 tests (see Annex A)
Dockerfile, docker-compose.yml, requirements.txt, .env.example

```

Approximate size: 7,700 lines of Python across the current (v2) codebase, excluding the retained v1 backup kept for reference.

Appendix C

Sample Live-Demo Transcript

The following question sequence is drawn directly from docs/demo_script.md and was run against the real, current build before being documented — not a hypothetical illustration.

1. Direct id lookup. `"who approves 2.126"` → resolves with certainty (method: id, confidence 1.0) to activity 2.126 ("Quarterly Mission program"), naming RDG / Director RDNG as approver, and automatically appending the mandatory Check/Verify note (Sector Manager, Supporting Dept. Division Manager) and the informed-party note (Concerned Sector VP, RDVP, Task Manager/Project Team) — without being asked for either.

2. Natural language, no id. `"who needs to be informed for quarterly mission program"` → resolves to the identical node 2.126 purely from the activity's name, now listing the informed parties.

3. Typo tolerance. `"who aproves the qaurterly mision program"` (two deliberate typos) → still resolves correctly to 2.126, via deterministic similarity matching, not an LLM guess.

4. Domain glossary. `"what does DDG stand for"` → "DDG stands for: Deputy Director-General." `"what does RDNG stand for"` → resolved via a maintained alias, with the agent transparent that this came from a curated alias rather than the DAM's own printed glossary.

5. Conversation follow-up. `"who approves of Communication with Co-Financiers of projects"` → resolves to 2.118 (Country Manager / DDG). Then, without naming the subject again, `"and who are the informed parties?"` → correctly stays on 2.118, labeled "carried over from previous question" in the UI.

6. Automatic redirect-following. `"who initiates 1.117.2"` → 1.117.2's own row is a pointer ("See 2.120: Organization of mission"); the agent follows it automatically and surfaces 2.120's real responsibilities instead of dead-ending.

7. Honest refusal. `"what happens with 9.999.999"` → "'9.999.999' doesn't exist in the DAM... Double-check the id, or describe the activity in words instead." No fabricated answer — presented in the demo script explicitly as "the feature, not a bug."

8. Personality. `"hi"`, later `"thanks"` → warm, varied replies, bypassing the DAM lookup pipeline entirely.

Appendix D

Selected Decision-Log Excerpts

Verbatim excerpts from docs/decisions.md, illustrating the project's evidence-before-fix discipline described in Section 12.3.

2026-07-28. *“(i)’ informed-marker characters were never matched. `extract_informed_instances` scanned raw characters for a consecutive ‘(’, ‘i’, ‘)’ run, but the real PDF characters for this marker are five separate characters... Confirmed directly against 2.126’s real characters (page 24) before fixing... 174/276 v1 nodes had ‘(i)’ with no role ever attached to it — 2.126 now resolves both of its informed markers to real roles.”*

2026-07-28. *“`get_children()` was silently wrong for two of four hierarchy levels... This was invisible until `threshold_variant ids` existed in the `id` set... caught by a consistency check (every node’s children must be the correct `node_type`, and every child’s own parent pointer must point back), not by inspection.”*

2026-09-03. *“Tried a blanket fix first... and rejected it after checking: 1614 candidates document-wide, several unsafe... Narrowed to a safe signal instead: only merge a row-bucket if it’s ENTIRELY action-code content.” *(on the NSO row-splitting bug — illustrating a rejected first fix, not just a successful one)**

2026-09-03. *“A candidate parsing fix... was tried, verified NOT to fix its target case, found to cause a real regression elsewhere, and was fully reverted... left as a known, honestly-documented limitation (2 of 327 nodes) rather than risking another blind fix.” *(on the grounding-fallback-frequency investigation, Section 9.4)**

Appendix E

Glossary of Terms Used in This Report

Term	Meaning
DAM	Delegation of Authority Matrix — the source document this project makes queryable
I / C / R / A	Initiate / Check-verify-or-consult / Review / Approve — the DAM's authority-code notation
(i)	The DAM's informed-party marker (a role that must simply be told about an activity)
Node	One structured record in this project's data model (a chapter, process, task, child task, or threshold variant)
Grounding check	The verification step ensuring an LLM's rephrased answer still contains every real fact, verbatim
RAG	Retrieval-Augmented Generation — here, deterministic retrieval feeding a fact-constrained LLM rephrasing step
TF-IDF	Term Frequency–Inverse Document Frequency, the lexical text-matching method used for free-text search
MultiDiGraph	A directed graph type (networkx) permitting multiple parallel edges between the same two nodes
Threshold variant	A DAM sub-item keyed by a monetary threshold band rather than a sequential number

Term	Meaning
OOM	Out of memory — the cause of the first failed Render deployment

Appendix F

Complete Project Decision Log (Verbatim)

This annex reproduces `docs/decisions.md` in its entirety, unedited beyond reformatting its own heading levels so it does not fragment this report's table of contents. It is the project's single primary source: a running, dated record of every scope decision, every confirmed bug and its fix, and every accepted trade-off, in the order it actually happened. Every claim made in the main body of this report about a specific bug, a specific test count, or a specific design decision traces back to an entry in this log. It is included here in full, not only excerpted, in the same spirit that a scientific report includes its raw data rather than only summary statistics — a reader who wants to verify this report's narrative against primary evidence, rather than take it on trust, should start here.

2026-07-21 - Source document

Switched source PDF from `dam_mvp_chapters.pdf` (66 pages, the original MVP-scoped export) to updated `dam file.pdf` (79 pages). Reason: the updated file includes the glossary, abbreviations, and authority-code (levels, colors) legend that the MVP export didn't have, and those are needed as reference data for the agent's `explain_authority / glossary-lookup` capability. Old file kept in `OLD/` for reference only.

2026-07-21 - Scope: chapter 3 (NSO) included

The updated file contains a full chapter 3 - Non-Sovereign Operations (client relations, identification, appraisal, trade finance, approval, portfolio management, equity investment) - that wasn't part of the original ~50-table MVP scope (chapters 1-2 only). Decision: chapter 3 is in scope for v2.

2026-07-21 - Two parsing bugs found and fixed (with tests)

1. **Title misattribution:** a task's title line that wraps to appear after its own actions row, immediately before the next task's bare identifier row, was being stolen by

that next task (v1). Fixed in `parsing/task_blocks.py` - forward-attachment to the next identifier only happens when no task is currently open.

2. **Reference-text corruption:** "See DAM 16.100, 16.200..." cross-reference phrases were being digit-stripped into titles as noise instead of extracted. Fixed by centralizing reference-stripping into one function (`parsing/metadata.py: remove_references`) that every title/notes path calls, instead of two divergent implementations.

Both verified against real pages of the actual PDF, not synthetic examples - see `tests/fixtures/known_cases.py` and `tests/test_task_blocks.py`.

2026-07-21 - Second real bug found via screenshot: 3.225/3.226

Confirmed against the actual table (page 48): 3.225 is a self-contained "redirect" row ("Refer to Activities 2.114 - 2.117 in Section 2."), period-terminated, and 3.226's title starts on the next physical line before 3.226's own identifier appears. The v1-style fix (only forward-attach when nothing is open) wasn't enough, since 3.225 was still "open" by bookkeeping despite being finished. Fixed by treating a trailing period on the open task's text as a "this is finished" signal see `parsing/task_blocks.py`. Also added a second reference pattern, "Refer to Activities X - Y in Section Z" (a range, expanded to every id in it), alongside the existing "See DAM ..." list pattern.

2026-07-21 - Authority code legend transcribed, "C" found to be overloaded

Full legend transcribed from the real page (`data/reference/authority_codes.json`, validated against the `AuthorityCode` schema - 17/17 load cleanly). Found: the letter "C" covers two unrelated concepts distinguished only by level and color - C1/C2 (red) = "Check and Verify", C3/C4 (purple) = "Consult" (Board/Governors or Independent Functions). Not a bug in the schema (each full code like "C3" is stored separately with its own meaning), but a trap for the agent's intent parser later: mapping the word "check" or "consult" to the bare letter "C" would wrongly merge two different authority concepts. Noted for the agent stage, not fixed yet.

2026-07-21 - Task 2.126 resolved (screenshot corrected an earlier theory)

Originally diagnosed (from coordinates alone) as needing geometry: two PDF lines ~0.15pt apart wrongly merged into one row. A real screenshot of page 12 showed this was wrong - title+actions are genuinely one row, textbook title-above-identifier,

and it failed for the same reason as 3.225/3.226: `current_task` (2.125) was still open with no period to signal it was finished. Added a third signal to `task_blocks.py`: a task is also treated as finished if an incoming line carries action codes while the open task already has some of its own - a second batch of actions is a strong sign it belongs to the next task, not the current one. All 9 tests pass, 0 xfail. Full 79-page smoke test: 240 task/child blocks, 0 exceptions, 2 remaining empty titles (2.516, 2.522, both on the new source's pages 49-50) -logged for the node-validation stage, not chased now.

2026-07-21 - New hierarchy level found: `threshold_variant`

Screenshot of page 36 (2.513.3) showed a `child_task` can have its OWN children, letter-labeled by amount threshold rather than numbered (e.g. 2.513.3 -> (a) Up to UA 2,000,000 / (b) Over UA 2M up to UA 10M / (c) Over UA 10M). Confirmed recurring, not a one-off: 53 lettered sub-item lines found document-wide (51 real, 2 false positives from letter-lists inside footnote prose, already safely excluded by the existing Notes-on boundary). Decided with Adem: id = dot-joined ("2.513.3.a"), `node_type` = new value "threshold_variant" (`schema.py`), distinct from `child_task` since it represents a condition/amount band of one activity rather than a separate activity. `task_blocks.py` now reconstructs the id from whichever `child_task` opened most recently, since the PDF only ever prints the bare "(a)". Also found and fixed a related bug while testing this: `NOTE_PATTERN`'s footnote-digit stripper was corrupting titles containing comma-formatted amounts ("2,000,000" -> ",,,") - fixed for comma-adjacent digits. Plain space-separated amounts ("2 million") are NOT fixed by this and still get corrupted -left as a documented xfail, genuinely needs superscript/geometry detection (v1's original approach), not another text-pattern patch. Full 79-page re-run: 291 blocks (138 task, 102 `child_task`, 51 `threshold_variant`), 0 errors, same 2 pre-existing empty titles.

2026-07-21 - Confirmed (not assumed): fused level+footnote digits need real geometry

Screenshot of page 29 (2.223.2/2.223.3) showed cells like "C13" -level digit and footnote digit both superscript, no separator. Checked at the character level: both digits are genuinely size 6 vs the letter's size 9 (confirms superscript), BUT `pdfplumber`'s own `extract_words()` is inconsistent about grouping them - the identical visual pattern comes out as one fused word "C13" in one cell and as two separate words "C" + "13" in the next cell on the same row. Text-pattern logic gives different, disagreeing answers depending on which way `pdfplumber` happened to tokenize it (fused: level=1/footnote=3; split: level=none/footnote=13) - not a regex problem, a tokeniza-

tion problem underneath the regex. Real fix needs raw character x-position und size, not `extract_words()`. Confirms the remaining geometry backlog (role attribution, informed-role gap, fused footnote digits) is correctly scoped as geometry work, not more text heuristics.

CORRECTED below - the "base" isn't the cell center, it's the letter's own top/bottom (Adem's original v1 idea, refined).

2026-07-21 - Level vs. footnote geometry rule, validated with real coordinates

Compared two real cases character-by-character:

- 2.224.4 "A1" with footnote 21: level digit "1" is BOTTOM-aligned with the letter (bottom 440.12 vs letter's 440.28); footnote digits "2","1" are TOP-aligned with the letter (top 430.64 vs letter's 431.28) and share identical height with each other.
- 2.223.2/2.223.3 "C13"/"C14": both trailing digits in every instance share identical top AND bottom with each other, matching the FOOTNOTE alignment, not the level alignment. Corrected an earlier wrong guess (level=1/footnote=3) - there's no level digit in these cells at all, it's bare "C" + two-digit footnote "13"/"14".

Rule: compare each small (size 6) character after the action letter to the LETTER's own top/bottom (not a cell-center estimate). Bottom-aligned = the level digit (at most one). Top-aligned = a footnote digit; consecutive top-aligned digits at the same height concatenate into one multi-digit footnote number. This is the basis for the geometry-based action/footnote extractor, next to build.

2026-07-21 - Geometry action/footnote extractor built (parsing/action_geometry.py) - partially validated

Built on the confirmed level/footnote alignment rule above. Anchor detection went through 3 rounds against real false positives, all found on one title (2.223.2, "Project Appraisal Report..."): raw character scans caught OCR-spaced letters ("A" from "A p p r a i s a l"); requiring the whole word to match `^[ICRA]\d*$` fixed embedded letters ("PAR") but not isolated single-letter spaced words; final version also checks word adjacency to catch that case.

Cross-checked geometry's per-task action count against the already-trusted text-based count (`extract_actions`) across all 79 pages: 228 tasks with actions, 197 match, 31 don't (~14%), with mismatches ranging from off-by-one to wildly off (2.412.1:

text=3/geo=0; 3.642: text=1/geo=9). Spread suggests more than one remaining cause, not a single last bug. NOT wired into node-building yet - status is "core alignment rule proven correct, anchor detection still has real, uncatalogued gaps." Needs either more real examples to diagnose against, or a conscious decision to ship with text-based counts as the trusted source and geometry as best-effort supplementary detail only.

Decision with Adem: move on to role/column attribution, treat the text-based action count as the trusted source for the rest of the build (don't keep chasing the 14%).

2026-07-21 - Column headers extracted (parsing/column_roles.py), role attribution built

Confirmed via real characters: column headers are rotated 90 degrees (pdfplumber's upright=False). Reading order is DESCENDING top, not left-to-right - checked "Task" letter by letter, each next character has a smaller top. x0 is constant per column, same role x-clustering plays for rows on the y-axis.

Two-line-wrapped headers ("Task Manager" / "Team Members" as adjacent vertical strips) needed a two-pass cluster: a tight pass (gap<=3pt) groups each physical text run in correct reading order FIRST, then a wider pass (gap<=14pt, calibrated against real gaps - wrapped lines ~10-12pt apart, distinct columns ~19-27pt apart) concatenates runs in x0 order. Doing one wide pass without the tight pass first interleaves the two runs character-by-character (their y-ranges can overlap).

Extracted headers for page 29 match the real table exactly (21 columns, "Task Manager1 & Project Team Members2" through "BOARD OF DIRECTORS"). Combined with action_geometry's x0 per action + nearest-neighbor matching (max 20pt distance, no match = None rather than a guessed wrong role): 940 actions checked across all 79 pages, 929 resolved to a role (98.8%) on pages where headers were found.

43/79 pages had no rotated header text at all - checked each one: 42 are legitimately non-table pages (glossary, notes, cover pages) -correct to have no headers. Exactly 1 (page 45) is a genuine multi-page table continuation - "2.434 Misprocurement: (a) Declaration of misprocurement..." starts the page directly, header was only printed on the previous page. Needs the eventual page-walking pipeline to carry forward the last successfully-found header set to any page that has none, rather than treating a missing header as page-specific -that's an integration detail for the orchestration/node-building stage, not a bug in column_roles.py itself.

2026-07-28 - Nodes built end to end (modeling/build_nodes.py) - two real bugs found and fixed

Single page-walk that builds task blocks, then enriches each one into a real schema. Node: title/references (parsing/metadata.py), actions with level/footnote geometry (parsing/action_geometry.py), and role attribution (parsing/column_roles.py, with the page-45 header carry-forward from the previous entry now actually wired in). Merged with the Pass-1 skeleton (modeling/hierarchy_skeleton.py) to add chapter and process nodes, which task_blocks.py never emits (chapter titles left blank - chasing chapter cover-page titles wasn't worth the time against this phase's schedule, flagged here rather than silently skipped). Full 79-page run: 327 nodes (3 chapter, 35 process, 138 task, 102 child_task, 49 threshold_variant), 937 responsibilities extracted, only 11 (1.2%) with an unresolved role. 2 pre-existing empty titles unchanged (2.516, 2.522 - still not chased, same as the 2026-07-21 entry).

Two real bugs found by testing against the full document, not assumed away:

1. **get_children() was silently wrong for two of four hierarchy levels.** It matched children by string prefix ("does this id start with task_id + '.'"), which only happens to be true for child_task->threshold_variant and task->child_task, because those ids are literally longer strings built from their parent's id. It's false for process->task ("2.513" does not start with "2.510." - tasks are grouped into a process by get_process_id()'s round-down-to-10 math, not by string containment) and for chapter->process (prefix matching returns every descendant at every depth flattened together, not just the chapter's own processes). This was invisible until threshold_variant ids existed in the id set and a task's grandchildren started colliding with its direct children under the same prefix - caught by a consistency check (every node's children must be the correct node_type, and every child's own parent pointer must point back), not by inspection. Fixed by deriving children from each candidate's own parent-pointer field (process_id / parent_task_id / chapter), read in the correct direction instead of guessed from string shape. Regression tests in tests/test_build_nodes.py check every node's children against its expected node_type and specifically pin process 2.510's children (previously always []) and chapter 2's children (previously a 211-item blob, now its 15 direct processes).
1. **"(i)" informed-marker characters were never matched.** extract_informed_instances scanned raw characters for a consecutive "(", "i", ")" run, but the real PDF characters for this marker are five separate characters - "(", " ", "i", " ", ")" - the space between the parenthesis and the "i" is a genuine PDF character, not just visual spacing. The 3-in-a-row scan always missed because band[i+1] was

a space, never "i". Confirmed directly against 2.126's real characters (page 24) before fixing. Fix: filter out whitespace-only characters before scanning. This is the actual fix for the v1 gap noted throughout this project -174/276 v1 nodes had "(i)" with no role ever attached to it -2.126 now resolves both of its informed markers to real roles.

Decision (my own, not yet run past Adem): both bugs fixed and covered by new tests rather than flagged and deferred, since both were cheap, well-understood fixes with a clear real-data repro, not open-ended geometry work like the still-deferred 14% action-geometry mismatch. Flagging here for him to review, per the "explain everything, don't silently change things" agreement - the `get_children()` fix in particular was surfaced as a decision point before being implemented.

2026-07-28 - Five more real bugs found and fixed, chasing "is the header footnote the same pool as the action footnote"

Adem asked a scoping question about the graph plan (role nodes need canonical role strings, and headers carry a trailing footnote digit -is that the same footnote numbering as action-level footnote_refs?). Answering it with real evidence, not a guess, opened a chain of five more real bugs - each found and fixed with the same discipline as before (real coordinates or a real screenshot before touching code), documented here in the order they were found.

1. Confirmed: same footnote pool. Page 58 (NSO 3.110, printed page 46) - header "Regional NSO Lead2" and Notes item 2 ("Regional NSO Lead: PL-2 or PL-1 staff...") are the same footnote; header "Origination Sector Manager3" and Notes item 3 match the same way. One numbered list per table, referenced from both role headers and action cells.

2. ALIGNMENT_TOLERANCE was dropping real footnote digits at the boundary. Task 3.111's "A3" digit has a top-diff of 1.50432pt from its letter - 0.004pt past the old ≤ 1.5 cutoff - while its bottom-diff (4.02pt) was nowhere near level-aligned either, so it matched neither rule and got silently dropped. Checked document-wide before changing anything: 30/345 digit decorations were being dropped, 23 with this exact near-miss signature (top-diff 1.504, bottom-diff 4.024) and 7 genuinely unrelated (diffs of 10.4-21.2pt). Raised ALIGNMENT_TOLERANCE from 1.5 to 2.0 - re-checked against the two examples that originally calibrated 1.5 (diffs of 0.16 and 0.64) to confirm they're unaffected.

3. Real row-splitting bug, confirmed via screenshot. Requested and got a screenshot of page 58's real table. It proved task 3.111's own "A3"/"A4" approval codes render on a row-bucket ABOVE 3.111's identifier row - a genuine PDF quirk

where a wide table row's rightmost cells land ~1pt off from its leftmost cells, enough to round into a different bucket in `build_rows()`. This was systematic across the whole 3.110 table (3.112's I2/I3/I4/A5/A6 were landing on 3.111, 3.113's I5/I6 were landing on 3.112 - a one-row cascade), not an isolated case.

Tried a blanket fix first (merge any two row-buckets within 5pt that have zero overlapping word x-ranges) and rejected it after checking: 1614 candidates document-wide, several unsafe - rotated header fragments, footnote list markers, and one case ("Directors:17" immediately before the next task's bare identifier "2.224") that would have spliced two different tasks together. Narrowed to a safe signal instead: only merge a row-bucket if it's ENTIRELY action-code content (no real standalone row can be just action codes with nothing else) - added `_merge_split_action_rows()` to `parsing/rows.py`, called from `build_rows()`.

4. Bug in my own new code, found testing #3. The purity check used `raw ACTION_PATTERN.sub()`, which can't match "I2"/"I3"/"I4" - `ACTION_PATTERN`'s I-branch is `\bI\b`, and there's no word boundary between a letter and an immediately-following digit. `metadata.extract_actions()` already normalizes this (`\bI(\d+)\b -> I`) before matching; my check didn't, so it silently skipped merging rows that were genuinely pure action content. Applied the same normalization instead of inventing a second, disagreeing check for the same thing - exactly the single-source-of-truth lesson from earlier in this project, just self-inflicted this time.

5. Second instance of the comma-adjacency footnote bug, via hyphens. Fixing #3 surfaced 3.111's real title for the first time - "Concerned Staff members below PL-2 level" was coming out as "... below PL- level". `NOTE_PATTERN`'s comma exclusion (see the 2026-07-21 `threshold_variant` entry) doesn't cover hyphens - "PL-2" has a word boundary between "-" and "2", so the bare digit matched and got stripped as a false footnote number. Checked document-wide (6 affected blocks, 5 clean "PL-N" cases, one - 3.524 - with unrelated pre-existing garbling not touched here) before extending the exclusion to `(?![, -])`.

6. One more bug, found writing the regression test for #3. After the row-merge fix, 3.111/3.112/3.113's merged TEXT was correct, but their extracted ACTIONS vanished entirely. Root cause: `ordered_lines()` reported both "top" and "bottom" as the row-bucket's single key y, never the words' real min/max span - harmless before, since every word in an unmerged row was within ~1pt of that key anyway. After `_merge_split_action_rows()` can combine words up to 5pt apart into one bucket, that single point plus `action_geometry.py`'s +/-1pt search pad was too narrow to still reach the merged content - 3.111's genuinely-merged "A3"/"A4" text existed, but geometry extraction's search band didn't cover its real y-position. Fixed by having `ordered_lines()` report the row's actual `min(top)/max(bottom)` across its words

instead of repeating the bucket key - correct for merged rows and a no-op for ordinary ones.

Full re-test after all six: 19/19 existing tests pass, plus 5 new regression tests in `tests/test_row_merge.py` pinning the exact (action, level, footnote) values from the screenshot for 3.111/ 3.112/3.113 and the two "PL-2"/"PL-1" titles. Full 79-page rebuild: 327 nodes (unchanged), 1772 responsibilities (up from 937 - the row-split fix recovers real actions that were previously dropped or misattributed across ~25 affected rows document-wide), 2.5% unresolved role.

Column-header duplicate bug, fixed (two separate root causes). The 50 exact-duplicate responsibilities found above traced to `parsing/column_roles.py`, not today's earlier changes - and turned out to be two distinct bugs, not one, both only visible once headers were checked across the whole document rather than spot-checked on one page (29, as before).

Bug A - character interleaving. On page index 43, two adjacent but genuinely DIFFERENT columns ("Regional Implementation Support Manager (RISM)" and "Country Manager / DDG") sit only ~2.04pt apart in x0 - closer than `TIGHT_CLUSTER_GAP` (3.0), so the tight-cluster pass merged their characters into one cluster. Since both are full header strings spanning the same y-range, sorting by top alone interleaved them letter-by-letter: "...Support Manager (CRoLuSntMr) y Manager / DDG". Tried detecting this via y-range overlap between adjacent characters first and rejected it - checked document-wide, 838/841 clusters showed overlap (rotated character bounding boxes routinely overlap due to font metrics, nothing to do with interleaving), so it doesn't distinguish anything. Found a real signal instead: the corrupted cluster's characters split cleanly into two x0 values with zero internal variance and a real gap between them (bimodal), not smeared/continuous jitter like a genuine single run. Checked this detector document-wide before using it: it flags exactly the one real corrupted cluster and nothing else. Added `_split_if_interleaved()` to `parsing/column_roles.py`.

Bug B - false merge of different columns. Fixing Bug A revealed the SAME two columns still merging into one header string ("...Support Manager (RISM) Country Manager / DDG") - a second, separate problem: `COLUMN_MERGE_GAP`'s gap-only logic can't distinguish them, because the WRONG merge gap on this page (~2pt, between two different columns) is smaller than the RIGHT merge gap elsewhere on the same page (~9pt, between wrapped lines of one column). No threshold value can separate a 2pt "wrong" case from a 9pt "right" case when the wrong one is smaller. Added a second, non-geometric signal: stop merging once the accumulated text already ends in a balanced closing paren (e.g. "...(RISM)"), regardless of gap. Checked this document-wide against the old gap-only merge before keeping it: changed only

4 pages out of 79 - the 3 with the known bug, plus one genuine additional find (page 26: four separate Director roles, "Director, Safeguards & Compliance (SNSC) Director, Resources Mobilisation & Partnerships (FIRM) Director, Syndications & Client Solutions (FIST) Manager – Programming (SNPB.1)", were merged into one blob and are now correctly split into four). No other page changed - not introducing new false splits elsewhere.

Result: exact-duplicate responsibilities 50 -> 3 (all three on page 43, the page with the tightest column spacing in the document - a small, localized, understood residual, not chased further), 19/19 + new tests pass, full 79-page rebuild still 327 nodes, unresolved role rate 3.2% -> 0.5%.

2026-07-28 - Role-header footnote digit stripped (blocker for graph role-node identity)

Before starting the knowledge graph, went back to the original scoping question that kicked off this whole chain: role headers carry a trailing footnote digit ("Regional NSO Lead2"), which fragments one real role into several different strings and would fragment the graph's role nodes the same way. Confirmed still present after all the fixes above (62/172 role strings ending in a digit) and fixed it in `parsing/column_roles.py` - three rounds, each caught by testing against real data before trusting it:

1. First attempt checked `ordered_chars[-1]` directly and never fired at all - the run's actual last character is almost always a trailing space (a real positioned character, not just a join artifact), not the digit. Fixed to find the last NON-space character instead.
2. Second attempt only stripped ONE trailing digit - multi-digit footnote numbers ("Specialist14" -> footnote 14, "FIFC Officer 11" -> footnote 11) only lost their last digit ("...14" -> "...1"). Fixed to walk back over every consecutive small trailing digit, the same way `action_geometry.py` already concatenates multi-digit footnote numbers.
3. Found and excluded a real false-positive risk before it caused damage: "Manager FIFC.4" / "Manager FITR.2" / "Manager PGCL.1" are real department codes, not footnote references, but also end in a digit. Excluded by checking the character immediately before the digit run isn't a literal "." - every confirmed real footnote case is preceded by a letter or closing paren, never a period. Also found while checking this: the exact same code ("FIFC.4") renders at NORMAL digit size on pages 33/48 but SHRUNK (footnote-style) size on page 49 - a rendering inconsistency size alone can't resolve, which is exactly why the

period-check matters as an independent second signal rather than relying on size alone.

4. Also handled: a footnote digit can be an entire run by itself with nothing else in it (e.g. a bare "1 " run split off on its own) -confirmed document-wide that an isolated small-digit-only run is always a stray footnote, safe to drop entirely.

Result: 172 -> 132 distinct role strings, all remaining trailing-digit cases are the 5 legitimate department codes. Full re-test: all existing + 3 new tests pass. Full rebuild: 327 nodes (unchanged), 1772 responsibilities, 0.5% unresolved, 3 duplicates (unchanged - the page-43 residual is unrelated to role-header footnotes).

Test suite is getting slow (~60-90s total across files, several of which do their own full 79-page rebuild with no shared caching) -noted, not fixed now; would be worth a session-scoped fixture shared across test files if this keeps growing.

2026-07-28 - Knowledge graph built (modeling/graph.py)

Chose networkx (MultiDiGraph, not DiGraph - a role can have two separate responsibilities on the same task, e.g. checks AND later approves, which is two parallel edges between the same pair of nodes, not one edge with two labels) over continuing to query the flat {id: Node} dict directly. Reasoning: the two founding example questions from the very first project brief - "who approves task X" and "who needs to be informed for task X" - are the same shape of question (who's connected to X by a particular kind of edge, filtered by action), and a role like "Country Manager / DDG" is a real, recurring entity referenced from dozens of tasks - worth being a first-class node with its own identity rather than a string repeated across 168 different Responsibility lists.

Schema: every schema.Node becomes a graph node (kept keyed by its own DAM id); one node per canonical role string, prefixed `role::` so a role name can never collide with a real DAM id; one shared `role::__unresolved__` node for responsibilities whose role never resolved to a header (kept visible rather than silently dropped, so a "who's responsible for X" query still surfaces that something couldn't be attributed). Three edge types: contains (parent->child, straight from the now-fixed `get_children()`), references (task->task, only wired if the target id actually exists in the node set - schema.py's own documented caution, confirmed real: task 2.312.2's "See DAM 16.100/16.200/16.300/16.400" references point outside this document's ~79-page scope and are correctly skipped, not silently wired to nothing), responsible_for (role->task, action/ level/ footnote_refs as edge attributes).

Full build: 459 graph nodes (327 DAM nodes + 132 distinct roles, including the unresolved placeholder), 2098 edges. Validated against the same real screenshot used earlier (NSO 3.110, page 58): task 3.111's approvers via `responsible_roles(graph,`

"3.111", action="A") returns exactly "Origination Sector Manager" (footnote 3) and "Supporting Dept. Division Manager" (footnote 4), matching the screenshot exactly - not just count-of-edges, the specific roles and footnote numbers. 7 new tests in `tests/test_graph.py`, all passing.

Added `requirements.txt` entry for `networkx`.

2026-07-28 - NLP component (`knowledge/search.py`) - TF-IDF chosen over pretrained embeddings

Closed the loop on the embeddings question raised in the very first project conversation ("do I need embeddings and normalization"). Ruled out training a custom embedding model outright: 327 short titles and no labeled query-title pairs is nowhere near enough data to train an embedding model that beats a general-purpose pretrained one - that would risk visible overfitting, not a real capability gain, and "there isn't enough data" is the honest, correct answer if a jury asks why one wasn't trained.

That left two real options: TF-IDF (classical, lexical, via `scikit-learn`) vs. pretrained sentence embeddings (`sentence-transformers`, semantic, needs `PyTorch`). Tried installing `sentence-transformers` first to see if it was practical - it didn't finish even after two attempts in this dev sandbox (`PyTorch` is a large dependency) - a real signal about deployment weight, though not necessarily disqualifying on a better-resourced machine. Presented both options with their tradeoffs to Adem; he chose TF-IDF for reliability/portability (installs in ~17s, no heavy dependency, works anywhere).

Built `knowledge/search.py`: a two-path resolver.

1. **ID fast-path** (`find_ids_in_query`): if the question already contains a literal DAM id, look it up directly - 100% accurate, no model involved. Covers most of this project's own founding example questions ("who approves task X"), which are phrased with an explicit id.
2. **TF-IDF fallback** (`search_by_text`): vectorizes every task/ child_task/threshold_variant/pro title and ranks by cosine similarity to the query, for questions phrased by title/description instead of id.

One real gap found testing this before trusting it: process titles ("3.110 ORGANIZATION OF NSO MISSIONS...") were originally excluded from the searchable corpus (reasoned that processes have no responsibilities of their own, since `build_nodes.py` only extracts those for task/child_task/threshold_variant) - but a query for "organization of nso missions" then matched nothing meaningful, because the one node whose title actually says that wasn't indexed at all. Fixed by including process titles too - what the agent does with a process-type match (e.g. aggregate its child tasks) is

the agent's concern, not this module's; the search module's only job is finding the right node.

Explicitly labeled: TF-IDF is LEXICAL (shared words), not semantic (shared meaning) - `resolve_query()` always returns a "method" field ("id" or "text_search") precisely so this distinction stays visible to callers and to the report, rather than being quietly overstated as "the agent understands your question" when what's actually happening is word-overlap ranking. A query using different words than the title (e.g. "sign off on" vs. "Signature of") will score lower than an exact phrase match - a known, documented limitation of this choice, not a bug.

5 new tests in `tests/test_search.py`, all passing. Added `requirements.txt` entry for `scikit-learn`.

2026-07-28 - Agent built (`agent/qa.py`, `agent/authority.py`)

Ties `knowledge/search.py` (resolve a question to a node) and `modeling/graph.py` (look up who's responsible) together into `answer_question()` - one function, UI-agnostic, that the web backend just has to wrap. `agent/authority.py` maps how people actually phrase questions ("who signs off on X", "who checks X") to the DAM's own authority codes, built to read the code/level structure from `data/reference/authority_codes.json` rather than a hand-written vocabulary that could drift from it. The "C" trap (documented back on 2026-07-21 - C1/C2 = Check and Verify, C3/C4 = Consult, two unrelated concepts sharing a letter, with bare "C" defaulting to Check per the reference data's own note) is encoded directly in each intent's match predicate and covered by a dedicated test - a "consult" question can never surface Check and Verify responsibilities or vice versa.

Confirmed both phrasings of this project's own founding example question from the very first conversation - "who are the informed parties for [Task_ID]" and "who needs to be informed for [Task_title]" - resolve to the identical answer, one via the id fast-path, one via TF-IDF, proving the id/text-search split in `knowledge/search.py` actually delivers what it was designed for.

One real UX gap found testing this: a question matching a process-type node (e.g. "organization of nso missions" -> process 3.110) used to answer "has no recorded responsibilities" - technically true (processes don't carry responsibilities directly, only their child tasks do) but a dead end. Fixed by pointing to the node's children instead when it has no direct responsibilities.

OCR title corruption - root cause found and fixed (not just patched)

Testing the agent's own output surfaced the "OCR-spacing" titles flagged as known debt back on 2026-07-21 in a way that made them impossible to ignore - they're

user-facing now. Investigated properly rather than re-applying the existing patch-list approach:

Bug 1 - a real parsing bug, not OCR noise. Task 3.524's title had swallowed its entire footnote section AND the page footer ("Page 57"). Root cause: page index 69's footnote header reads "Notes to NSO 3.510 - 3.520" - a one-off typo in the source PDF (checked document-wide: 21 pages say "Notes on", exactly 1 says "Notes to"). `task_blocks.py`'s `NOTES_PATTERN` only matched "on", so it never recognized the section boundary and kept attaching every following line to task 3.524. Fixed by matching both "on" and "to".

Bug 2 - the real root cause of the other 5. Checked the raw characters for 2.223.2's title and found this was never actually "OCR spacing" - it's pdfplumber's `OWN extract_words()` merging characters from TWO overlapping lines of real text ("Project Appraisal Report (PAR) for a non-" / "exceptional operation11", an ordinary wrapped two-line title) into garbled single words like 'ro', 'je', 'c' - corruption that exists in the word list handed to this project's code, not introduced by it. Confirmed the fix by grouping raw `CHARACTERS` by rounded top (exactly like `build_rows()` already does for words) and concatenating directly within each row - real space characters are already present in the character stream, so no word-boundary guessing is needed at all once cross-row contamination is eliminated. Added `build_char_rows()` / `char_lines()` to `parsing/rows.py` as a character-level parallel to `build_rows()/ordered_lines()`, used ONLY as a fallback in `modeling/build_nodes.py` when `parsing.metadata.looks_word_boundary_corrupted()` flags a title (a lowercase letter immediately followed by an uppercase letter, checked document-wide before using it - found exactly the 6 known cases, 0 false positives among the other ~320) - deliberately scoped to title recovery, not a replacement for the word-based pipeline everything else depends on.

Trade-off found and accepted, not silently hidden: character concatenation is exactly right for prose, but the actions/footnote row within the same block has no real spacing between adjacent footnote digits from different columns - they land glued together ("13131316"), which `NOTE_PATTERN`'s `\{b\}d{1,3}\{b\}` can't split (no word boundary exists inside an unbroken digit run). Added a narrow cleanup (strip digit runs of 4+, strip stray standalone periods) for the character-reconstructed path specifically, rather than reimplementing action/footnote-column separation at the character level - `task_blocks.py/action_geometry.py` already do that job well from words/geometry. Result: 1/6 titles (2.222.1) still trips the corruption detector after recovery (a stray "aI" adjacency from a superscript digit landing in an unexpected row-bucket) - down from 6/6, not a claim of 100% resolution. Documented via a

regression test (`test_corruption_rate_stays_low`) that fails loudly if this regresses further, rather than silently accepting any level of corruption.

16 new tests across `tests/test_title_corruption.py` and `tests/test_agent.py`, all passing. Full test suite re-run with no regressions.

2026-08-03 - Web page + backend (`webapp/backend.py`, `webapp/static/index.html`)

Thin FastAPI wrapper around `agent.qa.answer_question()` - deliberately no new logic in the web layer, every real decision already lives in the agent module. Builds `nodes/graph/search-index` ONCE at startup (`@app.on_event("startup")`, stored in a module-level state dict), not per-request: a full 79-page rebuild takes ~15-20s, and the DAM doesn't change between requests, so rebuilding per-question would be both slow and pointless. Exposes `POST /api/ask`, `GET /api/health`, serves the frontend at `/` and its static assets at `/static`.

Frontend color palette per Adem's explicit instruction: white background, #228b22 (forest green) as primary, #c80815 (red) used sparingly as an accent - specifically tied to low-confidence/no-match answers (small header dot, plus a red left-border + "[!] low confidence" flag on any agent message where `!node_id` or `method == 'text_search' && score < 0.3`), not decorative. The color itself carries the same meaning the agent already reports via its own `method/score` fields, rather than being arbitrary styling.

Verification hit a real environment constraint, not a code bug. Starting the server in one shell call, then trying to `curl` it from a separate, later shell call, failed every time (connection refused) -this dev sandbox tears down each shell call's background processes before the next call starts, the same constraint already hit earlier in this project with a backgrounded `pip install`. Confirmed by starting the server AND running every `curl` check within a single shell call: `/api/health -> {"status": "ok", "nodes_loaded": 327}`; both phrasings of the founding example question ("who are the informed parties for 2.126" / "who needs to be informed for quarterly mission program") returned the identical correct answer through the actual HTTP endpoint, one via the `id` path and one via `text-search`, matching the agent's own test; the unresolvable-query case returned the honest "couldn't find" answer, not a fabricated one; / served the real HTML with the correct title, `/static/index.html` served directly too.

That manual check isn't a repeatable test though, so also added `tests/test_backend.py` using FastAPI's `TestClient` (in-process, fires the real startup event itself) instead of a real `uvicorn` process - sidesteps the shell-call-isolation problem entirely rather than working around it, and is the correct way to test an API layer regardless. 5 tests: `health`, both `id/title` phrasings agreeing through HTTP, honest no-match,

frontend served at /, static mount. All pass (~17s, dominated by the one real PDF rebuild the module-scoped fixture does). Needed `httpx` as a test-only dependency for `TestClient` (not added to `requirements.txt` - it's not a runtime dependency of the app itself, only of the test).

Added `fastapi/uvicorn` to `requirements.txt`. Noted, not fixed: `@app.on_event("startup")` is deprecated in favor of lifespan handlers in current FastAPI - cosmetic, not worth the churn mid-project, flagged here so it's a visible, intentional choice rather than an oversight.

2026-08-03 - Comments pulled out of the codebase into docs/code_notes.md

Ahead of showing the code to the professor: moved every inline `#` comment across all 25 Python source/test files (plus the one CSS comment in `webapp/static/index.html`) into a new `docs/code_notes.md`, organized by file, so the reasoning stays fully available without cluttering the code itself. Docstrings were left in place - they're the code's own API reference, not the running commentary this was about consolidating.

Mechanical, not manual: used Python's `tokenize` module to locate every `COMMENT` token precisely (never mistakes a `#` inside a string literal for a real comment, unlike a regex approach would), grouped consecutive comment-only lines into blocks, recorded each block's line range and nearest enclosing `def/class` for context, then blanked each comment from its source line and dropped any line left empty as a result. Every rewritten file was `compile()`-checked before being written, and the full test suite (every file in `tests/`, run in small batches to fit the sandbox's per-call time limit) was re-run afterward end to end - all passing, confirming the strip was a true no-op against behavior, not just "the file still parses."

160 comment blocks moved in total, heaviest in `parsing/task_blocks.py` (19), `parsing/column_roles.py` (18), and `modeling/build_nodes.py` (13) - unsurprising, those are exactly the files with the most hard-won, non-obvious fixes documented earlier in this log.

2026-08-05 - Post-meeting additions: dashboard, hybrid LLM (Ollama + Groq), grounded RAG

The professor asked for three things after seeing the working agent: a dashboard, "an AI module," and RAG with hybrid local/API LLMs (Ollama for local, Groq for the API - confirmed with Adem it was Groq, not Grok/xAI, since "hybrid local + API"

tutorials almost always pair Ollama with Groq, and the two names get mixed up constantly).

Scoping decision made before writing any code, and worth recording because it reopens the project's founding premise. "Integrate RAG" could mean two very different things: (a) textbook RAG - chunk the PDF, embed it, let an LLM freely reason over whatever gets retrieved, or (b) grounded generation - keep retrieval exactly as it already is (the graph + TF-IDF, deterministic, validated against real screenshots from the earlier build phase), and only let the LLM rephrase the facts that retrieval already found, never decide what the facts are. Option (a) would have quietly undone the entire reason this project exists -it was built specifically to avoid an LLM hallucinating who approves a transaction, by using a structured knowledge graph instead of raw document search. Went with (b), confirmed with Adem before building: retrieval is unchanged, the LLM only touches phrasing, and a grounding check (`agent/generate.py: _mentions_expected_facts`) verifies every role name from the retrieved facts is still present, verbatim, in the LLM's output - if it isn't, the app falls back to the deterministic template answer instead of showing the LLM's version. This is still legitimately RAG (retrieval-augmented generation), just the retrieval was already correct and doesn't need an LLM anywhere near it.

Hybrid LLM layer (llm/)

`llm/base.py` defines one interface (`LLMProvider.chat(system, user)`) and one exception (`LLMUnavailableError`) that both providers raise for every kind of failure - missing config, network error, timeout, unexpected response shape - so callers only ever need to catch one thing. `llm/ollama_provider.py` talks to a local Ollama server's `/api/chat` (default `http://localhost:11434`, model `llama3.1`, both overridable via env vars). `llm/groq_provider.py` talks to Groq's OpenAI-compatible `/chat/completions` endpoint (default model `llama-3.3-70b-versatile`, confirmed current via Groq's own docs rather than assumed from training knowledge, since API model names change). `llm/router.py` resolves a requested mode ("`off`" / "`ollama`" / "`groq`" / "`auto`") to a concrete provider; "`auto`" prefers Ollama (free, no network dependency) and only falls back to Groq if Ollama isn't reachable and a `GROQ_API_KEY` is actually set - a real, useful demo point (the agent keeps answering if the venue's wifi drops).

Config is env-var-only (`GROQ_API_KEY`, `GROQ_MODEL`, `OLLAMA_HOST`, `OLLAMA_MODEL`), loaded from an optional `.env` via `python-dotenv`. Added `.env.example` documenting each var and added `.env` to `.gitignore` (it wasn't there before - checked and fixed, since a leaked API key is a real, avoidable risk, not a hypothetical one).

Grounded generation (agent/generate.py)

agent/qa.py: `answer_question()` was extended (additively - existing keys unchanged, existing tests unaffected) to also return the exact structured facts behind its answer: roles (the filtered role list if there's an intent, otherwise the full list - literally what the template string was built from, not re-derived) plus `node_title/`
`node_type/intent`. agent/generate.py: `humanize_answer()` builds a strict system prompt ("use ONLY the facts given, never add a role/ action/footnote not listed, if the fact list is empty say so plainly"), calls the resolved provider, and applies the grounding check described above. Two cases never even reach the LLM: no provider configured, and no node resolved at all (nothing to phrase, and handing an LLM a near-empty prompt just invites it to invent something plausible-sounding).

Dashboard (webapp/dashboard_data.py, webapp/static/dashboard.html)

Scoped to DAM structure visualization per Adem's call, not agent usage/monitoring - shows off the actual data model (327 nodes, 1776 responsibilities, 131 distinct roles, 459 graph nodes / 2102 edges) rather than synthetic example data. `build_summary()` computes every number live from the current nodes/graph at request time (same "one source of truth, queried live" reasoning as reading authority codes from their own reference file rather than a hand-copied vocabulary) - node counts by type, action-code distribution, top 15 roles by responsibility count, unresolved-role rate. One honest finding surfaced by building this: the action-code distribution has a handful of single-digit-count stray entries ("F", "o", " ", "O") -pre-existing extraction noise, ~5 occurrences out of 1776, not introduced by this work and too small to be worth chasing right now; noted here rather than silently smoothed out of the chart data (the frontend does roll small categories into an "other" bucket for chart readability, but the API itself returns the real counts, unrounded).

New endpoints and UI

GET `/api/dashboard/summary` (backend). POST `/api/ask` gained an optional `llm` field ("off" default / "ollama" / "groq" / "auto"); response gained `used_llm`, `llm_provider`, `llm_error`, and `deterministic_answer` (the original template answer, always present, so the chat UI can show "phrased by Ollama" alongside a "show structured (template) answer" toggle - useful for exactly the demo point above, showing the same facts phrased two ways side by side). `webapp/static/dashboard.html` (new page, same palette, Chart.js from cdnjs) and `webapp/static/index.html` (added the LLM mode dropdown, a nav link to the dashboard, and the `used-llm/ fallback/toggle` UI) round out the frontend.

Testing

All new logic is tested without needing a real Ollama install or a real Groq key: `tests/test_llm.py` mocks `requests.post/requests.get` at the provider level (15 tests - success, network failure, missing key, malformed response, router mode resolution). `tests/test_generate.py` uses a fake in-test provider to check the grounding check itself (8 tests - passthrough when no provider/no match, success, dropped-role fallback, exception fallback, empty-response fallback). `tests/test_backend.py` gained 6 more tests including one that patches `llm.ollama_provider.requests.post` to raise a real `ConnectionError` and confirms the live HTTP endpoint falls back cleanly - and one that does NOT mock anything, sending `llm: "ollama"` against this sandbox's real (nonexistent) Ollama server, to prove the fallback path works against a genuine failure, not just a mocked one. `tests/test_dashboard_data.py` (6 tests) checks the summary against the known 327-node / 1776-responsibility build. All 36 new/changed tests pass; full existing suite re-run with no regressions.

Also live-tested the whole stack end to end in one shell call (server start + curl checks together, the same pattern needed since separate shell calls don't share a running process): dashboard summary, dashboard page, chat page's new nav/toggle markup, a plain `/api/ask` call, and an `/api/ask` call with `llm: "ollama"` against this sandbox's real absence of an Ollama server - confirmed the response came back `used_llm: false` with a real connection-refused error message and the correct deterministic answer, not a crash or a stall.

Not yet done, flagged for Adem: real end-to-end verification against an actual running Ollama and a real Groq key can't happen in this sandbox (no GPU/model weights here, and I won't ask for or handle a real API key) - needs to happen on his own machine before the meeting, instructions in `docs/llm_setup.md`.

2026-08-06 - Small talk handling + fixed a real "wrong task" bug in invalid-code queries

Adem tested the agent and found two real gaps: plain greetings ("hi", "hello") got run through the DAM lookup pipeline instead of a normal reply, and asking about a task/code that doesn't exist could come back answering about a completely different, unrelated task instead of saying the code was wrong.

Small talk (`agent/smalltalk.py`). `detect_smalltalk(query)` checks the whole normalized query against a small set of anchored patterns (greetings, farewells, thanks, "how are you", help/capability questions) and returns a canned reply, short-circuiting before `resolve_query` ever runs. Anchored with `^...$` against the full message, not a substring search - deliberately, since a substring check would false-

positive on real questions containing "hi" (e.g. "history of the DAM") or starting with a greeting word before a real question ("hi, who approves 3.111"). Wired into `answer_question()` as the very first check. Node id, roles, score all come back `None` with method: "smalltalk" - the frontend's low-confidence flag had to be updated too (`data.method != 'smalltalk' && ...`), since a null node id used to always mean "uncertain match" and would otherwise have painted a friendly "hello" reply with the same red low-confidence border as a failed lookup.

Invalid codes (`knowledge/search.py`, `agent/qa.py`) - **a real bug, not just a missing feature.** Root cause: `resolve_query()` only ever checked whether an id-shaped substring existed in nodes; if it didn't, the code silently fell through to `search_by_text()` on the ENTIRE query, including the bogus id. That's dangerous, not just unhelpful - the leftover words in the query (e.g. "task", "who", "approves") can share enough vocabulary with some unrelated real task's title to score above zero and get returned as a confident answer, with nothing telling the user the code they actually asked about doesn't exist. Confirmed this was reachable, not hypothetical, before fixing it (see 2026-07-28's `test_unresolvable_query_is_honest_not_fabricated`, which used exactly this kind of query and got lucky that "what happens with 9.999.999" had too little shared vocabulary to match anything - a different invalid-code phrasing would not have been so lucky).

Fixed by adding a third path to `resolve_query()`: `_all_id_candidates()` finds every id-SHAPED substring regardless of validity; if one exists and none of them are real, the method is "invalid_id" (not "text_search") and the whole-query text-search fallback never runs. Suggestions come from two sources, tried in order: `suggest_ids_near()` (real siblings under the same chapter number, ranked by longest shared dotted prefix - simple and explainable, not fuzzy matching) if the chapter itself is real; otherwise the query with the bogus id stripped out is run through the normal text search, so genuinely descriptive words in the query (e.g. "quarterly mission program 9.999") still surface a real, relevant suggestion even though the code itself was wrong. `agent/qa.py`: `_format_invalid_id_answer()` turns this into a message that says plainly the code doesn't exist and never mentions a different task as if it were the answer.

Had to update three existing tests whose queries happened to contain an id-shaped-but-fake number and asserted on the old, less-safe message ("couldn't find" instead of "doesn't exist") - `tests/test_agent.py` and `tests/test_backend.py`'s versions of `test_unresolvable_query...`. Renamed and re-asserted rather than deleted, plus added a companion test with a genuinely non-id free-text query so the original "I couldn't find a task" path still has direct coverage of its own.

24 new tests total (tests/test_smalltalk.py: 37 cases across 6 categories; tests/test_search.py: 3 new resolve_query cases; tests/test_agent.py/tests/test_backend.py: smalltalk + invalid-id integration cases) - all passing, plus the full existing suite re-run with no regressions. Live-tested against the real running server: "hi"/"thanks" get canned replies, "who approves 3.999" suggests real chapter-3 tasks, "who approves the quarterly mission program 9.999" recovers 2.126 from the left-over words despite the bogus code, and "who approves 3.111" (a real question) is unaffected.

2026-08-06 (later) - "See X" reference formats were silently corrupting titles AND dead-ending real questions

Adem reported a specific case: "who initiates 1.117.2" answered "no one is recorded" even though the real DAM page shows 1.117.1/1.117.2 both say "See 2.120: Organization of mission" and 2.120's own table (screenshot provided) clearly has an initiator. Investigated with the raw PDF text rather than guessing, and found a chain of three real bugs, not one.

Root cause. `extract_references()` only recognized two reference phrasings: "See DAM X, Y, Z" and "Refer to Activities X - Y in Section Z". Two more real phrasings exist in the document and were never recognized: "See <id>: <description>" (a redirect to a whole process, e.g. 1.117.1/1.117.2 -> 2.120) and "(See <id> / <id>)" (a redirect to one of two alternative activities, e.g. 1.115.1/1.115.2 -> 1.114.1 or 1.114.2). Because neither was recognized, the id inside them was never protected from NOTE_PATTERN's bare-digit stripper -`extract_title` calls `remove_references` BEFORE `remove_note_references`, specifically so a reference's digits can't be mistaken for footnote noise (see the 2026-07-21 entry), but that protection only covers phrasings the code knows to strip. The result: "See 2.120: Organization of mission" had its "2" and "120" stripped individually as two separate footnote-looking digit runs, leaving the garbled "See .: Organization of mission" sitting inside 1.117.1's and 1.117.2's titles - and, separately, the reference itself was just gone, never making it into `node.references`.

Checked document-wide before fixing (the project's standing rule): exactly 6 nodes affected, all in chapter 1 - 1.115.1/1.115.2 (the slash form), 1.117.1/1.117.2 (the colon form). Fixed by adding `SEE_ID_COLON_PATTERN` and `SEE_ID_SLASH_PATTERN` to `parsing/metadata.py`, wired into both `extract_references()` (captures the id(s)) and `remove_references()` (strips the phrase before the footnote-digit stripper ever sees it) - same pattern the two existing reference types already use, not a new mechanism.

A second, unrelated bug found in the same 6 nodes. 1.114.1/ 1.114.2's titles read "...with a -Year Rolling Business Plan" -missing digit again, but a different cause: NOTE_PATTERN's hyphen exclusion `((?<![, -])\b\d{1,3}\b(?:, ))` only excluded a digit PRECEDED by a hyphen ("PL-2", fixed 2026-07-28) - it never excluded a digit FOLLOWED by one. The real text is "a 3-Year Rolling Business Plan" (confirmed against the raw page text before touching the regex), and "3-Year" is the mirror-image case the original fix didn't cover. Extended the lookahead to match the lookbehind: `((?<![, -])\b\d{1,3}\b(?:, )?)`. Checked this doesn't regress the already-covered cases (footnote digits, comma-formatted amounts, PL-2) via `tests/test_metadata.py` before trusting it.

Third: even with the reference now captured correctly, the agent still had no way to USE it. `responsible_roles()` only looks up direct `responsible_for` edges on the queried node itself; a node whose only real content is a redirect has none, by design - that's what a redirect means. Added `agent/qa.py: _format_reference_pointer()`, tried as a second fallback (after the existing `children-pointer`, which already handles the analogous "process node with no direct responsibilities" case) whenever a node has no responsibilities and no children but does have references. It follows the reference and: if the referenced node itself carries responsibilities, surfaces those (filtered by intent, so "who initiates" only shows initiators, not every role on the target); if the referenced node has no responsibilities of its own but has children (2.120's actual shape -a process whose child tasks 2.121-2.126 carry the real responsibilities), points to those instead, reusing the same child-listing logic as the existing `children-pointer` rather than a second copy of it (extracted into `_children_list_text()` so both call sites share one implementation). Verified against the exact reported case: "who initiates 1.117.2" now reads "1.117.2 (...) redirects to 2.120 (...), a process that doesn't carry responsibilities directly. Its activities: 2.121 (...), ..." - and "who initiates 1.115.1" (a redirect to a task that DOES carry its own responsibilities, not a process) correctly surfaces "Country Economist" directly rather than another children list.

Grounding gap found and closed while wiring this up, before it became a real problem. A pointer/redirect answer has a real `node_id` but an empty roles list (the facts live on the referenced node, not the queried one). `agent/generate.py: humanize_answer()` previously only skipped the LLM when `node_id` was `None` - it would have handed a pointer answer to the LLM with an empty facts list, and `_mentions_expected_facts's` check (`all(... for r in roles)`) is vacuously true over an empty list, so the grounding check would have passed regardless of what the LLM said. Closed by also skipping whenever roles is falsy, not just when `node_id` is `None` - caught by reasoning through the new

code's interaction with the existing LLM layer, not by observing a live failure, and added a dedicated test (`test_pointer_answer_with_a_resolved_node_still_never_calls_the_provider`) to keep it caught.

12 new tests (`tests/test_metadata.py`: 11 unit tests on the regex functions directly; `tests/test_task_blocks.py`: 6 new integration cases against the real PDF pages; `tests/test_agent.py`: 2 reference-redirect cases; `tests/test_generate.py`: 1 grounding-gap case) plus 6 new real cases added to `tests/fixtures/known_cases.py`. Full existing suite re-run with no regressions; full 79-page rebuild still 327 nodes, 1776 responsibilities (unchanged - this was a title/reference fix, not an action-extraction one), corruption count still 1/327 (the same already-documented 2.222.1 residual, untouched by this fix), 14 references now captured document-wide (up from 8), graph edges 2108 (up from 2102, the 6 new valid reference edges). Live-verified against the real running server with the exact query Adem reported.

2026-08-06 (later still) - Dockerized for portability, live status dot, LLM picker defaults + model names

Three asks aimed at making the professor demo more self-contained and transparent.

Docker. Two services, not one: `app` (this project's own code) and `ollama` (the official `ollama/ollama` image), on a shared Compose network - not because splitting them is required, but because that's genuinely what they are: two independent services with independent lifecycles (the `app` rebuilds on every code change; Ollama never needs to, it just needs a model pulled once into a persistent named volume). Groq needs no container at all - it's a cloud API, reached the same way from inside Docker as outside it. `Dockerfile` binds `uvicorn` to `0.0.0.0` (not `127.0.0.1` - a container bound to `localhost` is unreachable from outside itself, a common first-time Docker mistake). `.dockerignore` explicitly excludes `.env - COPY` . . would otherwise bake a real Groq key straight into an image layer, which is worse than a plaintext file lying around since image layers tend to get shared/pushed without a second thought.

One real problem caught before it could bite: Compose's `env_file`: - `.env` fails the whole stack immediately if `.env` doesn't exist, and `.env` is gitignored on purpose (2026-08-06 entry) - meaning a fresh checkout, exactly the "hand this to the professor" scenario this was built for, would refuse to start at all with no real explanation. Fixed with Compose's `env_file`: - `path`: `.env` / `required`: `false` form, plus docs/`docker_setup.md` telling people to `cp .env.example .env` as the first step regardless, so it works whether or not their Compose version is new enough to support `required: false`.

Couldn't build or run the containers here - no docker binary in this sandbox (docker -version returns "command not found"). Validated what's checkable without it: docker-compose.yml parses as valid YAML with the exact structure intended (checked via PyYAML), Dockerfile's CMD matches the same uvicorn invocation already proven to work bare-metal. Flagged clearly for Adem: an actual docker compose up -build run on his own machine is the one verification step this couldn't do itself, and should happen before relying on it for the meeting.

Live status dot. The header's small red dot was purely decorative before (tied to the white/green/red palette, not to anything real). Now polls /api/health every 8s from both index.html and dashboard.html; green while the backend answers, red the moment it doesn't (page load with the server down, or the server dying mid-session). A try/catch around the fetch treats a network error the same as a non-200 response - either way, red. Small, but a real signal for a live demo: if something goes wrong, "is the server even alive" is answered before anyone has to ask.

LLM picker: default to Auto, show model names. The mode dropdown defaulted to "off" only because "No LLM" was listed first, not because anyone had chosen it as the default. Adem asked for it to default to Auto - not just cosmetic, since a demo audience will judge "the hybrid model" by what they see FIRST, and that should be the actual hybrid behavior he asked to build, not the disabled baseline.

Showing the resolved model name next to "Ollama"/"Groq" turned out to need swapping the native <select> for a small custom dropdown: an HTML <option> can only ever hold plain text, no nested styling, so "model name in dark grey small font, in front of local/API" is structurally impossible inside a real <select>. Built a minimal button + absolutely-positioned menu instead (.llm-picker-* in both CSS and JS) - each row is a real that can hold differently styled spans, closes on an outside click, and updates a plain JS variable (selectedLlmMode) the existing submit handler already reads from, so nothing about how /api/ask gets called had to change.

The model names themselves come from a new GET /api/llm/config endpoint that reads OllamaProvider().model / GroqProvider().model directly - not a second, hand-typed copy of "llama3.1" in the frontend that could drift the moment someone sets OLLAMA_MODEL in .env to something else. Confirmed the override actually flows through with a dedicated test (test_llm_config_respects_env_overrides) rather than assuming the existing provider classes handle it correctly just because they were designed to.

9 new tests in tests/test_backend.py (config endpoint, env-override respected, both pages' new markup present, Auto pre-selected in the served HTML) - all passing,

full existing suite re-run with no regressions, live-verified against the real running server (/api/ llm/config returns the real default model names; both pages' HTML contains the expected status-dot and picker markup).

Docker itself worked first try on Adem's machine (screenshot: app-1 and ollama-1 both up, healthcheck hitting /api/health repeatedly, Application startup complete) - confirms the sandbox-only validation (YAML parse + matching the proven uvicorn command) held up in practice, not just in theory.

One real usage snag, not a bug: a browser tab tried `http://0.0.0.0:8000` (visibly copied from the log line "Uvicorn running on http://0.0.0.0:8000") and got `ERR_ADDRESS_INVALID. 0.0.0.0` in that log line means "listening on every network interface inside the container" - it's what the server binds TO, not an address a browser can connect to. `http://localhost:8000` (or the container's mapped host port) is the actual address. Worth a one-line callout in `docs/docker_setup.md` since Uvicorn always prints it this way and it's an easy, common mix-up - not something to silently fix in code (it's log output being accurately literal, not wrong).

2026-08-06 (yet later) - Real DAM answer for out-of-scope references, not a generic dead end

Adem tested "who checks 2.312.2" and got "No one is recorded to check on 2.312.2 ... in the DAM" - technically consistent with existing design (2.312.2's references, 16.100-16.400, point to chapter 16, which was never part of this ~79-page export and correctly excluded per schema.py's own documented caution and the 2026-07-28 graph entry), but misleading: the DAM's actual text for 2.312.2 isn't silence, it's an explicit "See DAM 16.100, 16.200, 16.300, and 16.400" spanning the whole row (confirmed against Adem's screenshot of the real page, PSO 2.310). Saying "no one is recorded" reads as "the DAM has no answer" when the true situation is "the DAM's answer lives in a section this document doesn't include" - a meaningfully different and more honest thing to tell someone.

Fixed in `agent/qa.py`: `_format_reference_pointer()`: now tracks which of a node's references don't exist in nodes (`out_of_scope`) separately from ones that do but lead nowhere useful, and if a node's references are ALL out of scope, returns "2.312.2 (...) is governed by DAM section(s) 16.100, 16.200, 16.300, 16.400, which are outside the scope of this document." instead of falling through to the generic "no one is recorded" message. Deliberately narrow: only fires when literally none of the references resolve to anything - a node with a mix of a real, useful reference and an out-of-scope one still prefers the real, useful answer (existing behavior, unchanged).

1 new test (`test_reference_entirely_out_of_document_scope_says_so_honestly`), pinned to the exact reported case. Full existing suite (`test_agent.py`,

`test_generate.py`, `test_backend.py`) re-run with no regressions - the grounded-generation skip-when-no-roles fix from earlier today already covers this new message too (2.312.2 still resolves to `roles: []`, so an LLM mode still never touches it, correctly, for the same reason a redirect-to-a-process answer doesn't).

2026-08-06 (even later) - Abbreviations/acronyms glossary lookup, scoped to pages 2-7 only

Adem asked "wait the agent doesn't know the abbreviations and acronyms?" after testing "what does DDG mean" / "what does RDG stand for" and getting the generic "I couldn't find a task in the DAM matching that question." - a real, confirmed gap: the DAM's own front matter (pages 2-11, 0-indexed) defines every role acronym used throughout the document, but nothing in the pipeline ever parsed it.

Asked Adem whether abbreviation knowledge should show up as (a) a dedicated "what does X mean" lookup, (b) inline auto-expansion of acronyms inside normal answers, or (c) both - he chose both.

Scope decision: Abbreviations section only (pages 2-7), not Glossary (pages 8-11). Both sections are two-column term/definition lists with no font-weight distinction between the columns (checked directly via `extract_words(extra_attrs=['fontname', 'size'])` - both columns render in the same font/size, so column membership has to come from x-position, not styling). The Abbreviations section's definitions are short and mostly single-line, so a term and its definition stay row-aligned all the way down a page. The Glossary section's definitions are long and routinely wrap across several lines, which lets the two columns drift out of row-alignment over the course of a page - confirmed by checking that the Abbreviations pages' term-reading-order stays alphabetical throughout, while the same check on the Glossary pages does not (e.g. "Annual Programming for Bank Group Operations" reads between "ADF Charter" and "AfDB Charter" instead of after both under simple per-row pairing). Reconstructing a reliably-ordered Glossary would need real column text-flow analysis, not a per-row pairing - deferred as a known, documented limitation, since it isn't what was actually needed (defining role-code acronyms like DDG/RDG/RISM, which live entirely in the Abbreviations section). If the Glossary's longer concept definitions (e.g. "Accountability", "Appraisal") are ever needed, this is the specific harder problem to come back to.

Built `parsing/glossary.py: extract_abbreviations()` - clusters words into rows by top (with a small merge tolerance, same near-miss-alignment issue seen elsewhere in this project: a term and its definition sometimes render ~1pt apart in top, not perfectly aligned), splits each row into term-column / definition-column text by x-position, and classifies each row as: a new entry (term text AND definition

text both present), a continuation of the current entry's term (term-only, not colon-terminated - handles a term wrapping onto its own next line, e.g. "Concerned VP" / "/" Manager"), a section header to skip (term-only OR definition-only text ending in ":", e.g. "Committees of the Board of Directors:"), or a continuation of the current entry's definition (definition-only, no new term on the row).

Two real bugs found and fixed before trusting the output, both caught by manually inspecting the printed extraction against the real pages, not assumed correct:

1. Def-only continuation rows were wrongly starting a new, term-less entry.

The original logic treated "any row with definition-column text" as "start a new entry," so a definition that simply wrapped onto its own line (no new term on that row) got treated as a fresh entry with an empty term - which then silently vanished (filtered out for having no term), while the real term's wrapped continuation landed on the wrong, now-orphaned entry instead. Concretely: "Concerned VP" (term-only row) then "The Vice-President / Manager under whom..." (definition-only row) then "/" Manager" (term-only row) produced a dropped "Concerned VP" entry and a bogus "/" Manager" entry, instead of one correct "Concerned VP / Manager" entry. Fixed by only starting a new entry when a row has BOTH term and definition text; a definition-only row now correctly appends to the CURRENT entry's definition instead.

2. Page-footer page numbers leaking into the last definition on each page. Every one of pages 2-7 ends with its own lowercase roman-numeral page number (xiv, xv, xvi, xvii, xviii, xix - an unbroken sequence, confirming it's a footer artifact and not real content) landing close enough in top to the last real row that the row-merge tolerance folded it into that row's definition (e.g. "NSO and Private Sector Support Department xv"). Fixed with a narrow post-process step that strips a trailing lowercase word from a definition only if that word is itself a full, valid roman numeral AND the definition has at least one other word (so a definition that's legitimately a single roman-numeral-shaped word would be left alone, though no real case like that exists in this data).

Also confirmed, not fixed (correct behavior, not a bug): the source PDF itself lists 8 terms twice across these pages (EDCC, ECC, EMT, OCC, PEN, ECGF, PIVP, PINS), sometimes with slightly different wording between the two listings (e.g. PINS: "NSO and Private Sector Support Department" vs "NSO & Private Sector Support"). `build_abbreviation_glossary()` merges pages 2-7 into one {term: definition} dict and keeps the longer of the two definitions per term as a tie-break - both listings are correct, this is just a preference for the more detailed one.

170 entries saved to `data/reference/abbreviations.json` (one canonical source, same precedent as `authority_codes.json` - never regenerated inline at request time, so behavior can't silently drift between runs).

Agent wiring (`agent/glossary.py`, **new**), **both halves of Adem's "both" answer:**

- **Dedicated lookup:** `detect_glossary_query()` matches deliberately narrow trigger phrasing only ("what does X mean/stand for", "define X", "what is the meaning of X") - specifically NOT a bare "what is X", since that would collide with real DAM id lookups like "what is 2.120". Wired into `agent/qa.py: answer_question()` right after the small-talk check and before DAM node resolution. An unmatched term gets an honest "isn't in the DAM's Abbreviations and Acronyms list" message, not a fabricated guess or a silent fall-through to the generic "couldn't find a task" message.
- **Inline auto-expansion:** `expand_acronym_in_role_name()`, wired into `agent/qa.py: _format_role_list()`. Expands a role name that's itself a known acronym (e.g. "RDG" -> "RDG (Regional Director-General)"), or that contains one embedded in a composite role (e.g. "Country Manager / DDG" -> "Country Manager / DDG (DDG = Deputy Director-General)"). Expands each distinct acronym only once per answer (tracked per `_format_role_list()` call) so a long role list with the same acronym repeated doesn't get cluttered.

19 of the DAM's 132 distinct role names are themselves exact Abbreviations-list acronyms (CODE, CPO, CRC, DMT, ECVF, EDCC, FIVP, IDEV, OPSCOM, PIVP, PRC, PRST, PSEG, PSRC, RDG, RDVP, SMCC, TIC, TQAC -checked directly against the real graph before deciding this was worth building, not assumed).

28 new tests: `tests/test_glossary_parsing.py` (extraction against the real PDF pages - pins both bug fixes, the section-header skip, the duplicate-term merge, and the deliberate page-range scoping) and `tests/test_glossary_agent.py` (trigger-phrasing detection, the "never hijacks a real DAM question" guarantee, honest not-found handling, inline expansion including the once-per-answer rule, and full `answer_question()` integration). Full existing suite re-run file-by-file with no regressions.

Follow-up same day: Adem tested this live and hit two things - "what's DDG?" / "what's DDG" returned the generic "couldn't find a task" message (a real gap: those weren't among the explicit trigger phrasings), and the inline expansion wasn't showing up on "who approves 2.111" either. Checked both directly against the code: "what does DDG stand For?" and the 2.111 inline expansion ("Country Manager / DDG (DDG = Deputy Director-General)") both worked correctly when tested straight from this session's code - meaning the server Adem was looking at was still running the pre-glossary code (needs a restart, or a rebuild if running via Docker). The bare

"what's DDG" phrasing was a genuine, separate gap though, and a very natural one to miss, so fixed it too: added `_BARE_WHATS_PATTERN` in `agent/glossary.py`, handled deliberately separately from the explicit trigger patterns rather than merged into them, since "what's X" / "what is X" is inherently ambiguous with real DAM lookups ("what is 2.120"). Only treated as a glossary question when the term is a bare single token with no dots or spaces (a DAM id always has a dot; a task description is always multiple words) AND that token actually resolves in the glossary - if it doesn't resolve, returns no match at all rather than an honest "not found," since an unresolved bare "what's X" could just as easily be a mistyped task question. 6 new parametrized cases added to the existing trigger-phrasing tests, all passing.

Noticed in passing, not fixed (out of scope for this feature, flagged for a later pass): `tests/test_llm.py`'s `test_chat_without_api_key_raises_unavailable_no_network_call` fails when run in the same pytest session as `tests/test_backend.py`, because importing `webapp.backend` triggers `load_dotenv()`, which picks up Adem's real local `.env` (with a real `GROQ_API_KEY`) and leaks it into a test that assumes no key is set. Confirmed this is a pre-existing test-isolation gap, not something this change introduced passes in isolation, only fails when session-ordered after a module that imports the backend. Easy fix later: have that test explicitly clear `GROQ_API_KEY` from `os.environ` rather than relying on it being absent.

2026-08-06 (still later) - Frontend rewritten as React + GSAP, plus a new landing page

Adem asked for the frontend to become real React components, with GSAP animation, a loading screen, and a new landing page that lets the user choose chat or dashboard - same color palette (white / #228b22 / #c80815 accent).

Flagged the real trade-off before building anything. The frontend had been plain HTML/CSS/JS with zero build step - it just worked the moment the container started. A React setup needs a build pipeline (Vite + npm), which means a new Node toolchain in the Dockerfile, an extra build stage that has to succeed before the app can even start, and a new class of "works on my machine, not the professor's" risk, this close to a demo. Presented the choice explicitly (vanilla JS + GSAP, zero new infrastructure, vs. full React + Vite + GSAP, more "modern" but more that can break this week) and let Adem decide with that trade-off in view, per how this project's decisions get made -he chose full React + Vite.

Architecture: three independent React apps, not one SPA. Each page (landing, chat, dashboard) is its own Vite build entry (`webapp/frontend/{landing,chat,dashboard}.html`) mounting its own React root, rather than a

single-page app with client-side routing. Deliberate: the backend already serves each page as its own static file at its own FastAPI route (`/`, `/chat`, `/dashboard`), so this keeps that same simple model - no history-API fallback route needed in FastAPI, no risk of a hard refresh 404ing on a client-side-only route, smaller per-page JS bundles. `webapp/frontend/vite.config.js` builds straight into `webapp/static/` (now a build artifact, not committed -see `.gitignore` - `webapp/frontend/` is the real source from here on).

What got built, all reusing the same color palette (`webapp/frontend/src/styles/theme.css`):

- Landing (new page) - hero + two GSAP-animated cards routing to `/chat` and `/dashboard`, slow-drifting background accents, wrapped in the new `LoadingScreen` component gated on a real health-check call (not a fake timer - the splash means "is the backend actually reachable," not just padding for effect).
- Chat - ported every behavior from the old `index.html` faithfully: message list, low-confidence flagging, the LLM picker (off/ollama/ groq/auto, live model names from `/api/llm/config`), the "show structured (template) answer" toggle. New: each message animates in with GSAP on arrival instead of just appearing.
- Dashboard - ported the cards + all three Chart.js charts unchanged (same `ChartCanvas` wrapper around real Chart.js, not a React chart library swap - lowest-risk way to keep the exact working chart configs). Wrapped in `LoadingScreen` gated on `/api/dashboard/summary` actually resolving - the old "Loading summary..." text state became a real animated loading screen instead of disappearing outright.
- `LoadingScreen` (new, shared) - GSAP-crossfades from an animated ring into the real page content once a page's own ready condition is true. Deliberately NOT applied to Chat - that page has no real blocking data dependency (the old version was usable instantly, with model names filling in a moment later), so forcing a loading gate there would have been a pure regression dressed up as polish.
- `Header`, `StatusDot`, `LlmPicker` - componentized versions of the existing header/status-dot/picker, same markup/behavior, shared across pages instead of copy-pasted per HTML file like before.

Docker: Dockerfile is now a two-stage build - a `node:20-slim` stage runs `npm install && npm run build`, and only the built `webapp/static/` output gets copied into the final `python:3.11-slim` image; Node itself never ships in the image that actually runs. `docker compose up -build` needs no new steps from Adem - it just

takes a little longer on first build now. Bare-metal running needs a new one-time step (`cd webapp/frontend && npm install && npm run build`), documented in `docs/llm_setup.md` where the "start the server as usual" instruction lives.

Real environment problem hit while building this, worth recording: `webapp/frontend/` lives inside `damaiagent/`, which is synced via OneDrive - `npm install` there was consistently too slow to even finish (100+ packages, thousands of small files, over what turned out to be a FUSE-mounted sync layer, not local disk). Fixed by doing the actual `npm install / npm run build` in `/tmp` (a real local disk in this environment) and copying only the small built output (three HTML files + a handful of JS/CSS bundles, not `node_modules`) back into `webapp/static/` - fast, since it's a handful of files instead of thousands. Not a code change, just how this session did the build; Adem's own machine won't have this problem the same way, since running `npm install` directly on a local disk (not through a cloud-sync layer) is the normal case - but worth knowing if `npm install` ever seems to hang for him too, since OneDrive sync can behave the same way for any tool that writes many small files quickly.

Test coverage changed shape, on purpose, documented not hidden: the old `test_backend.py` checks for `id="status-dot"`, `id="llm-picker-menu"`, and `class="llm-picker-option selected" data-value="auto"` all asserted against raw server-rendered HTML - that markup no longer exists server-side at all now, it only exists after React runs in a browser. Replaced those assertions with route-level checks (the right pages serve, reference a built `/assets/` script) and added `tests/test_frontend_source.py`, which checks the same real guarantees (Auto is the default LLM mode, every page renders the status dot, the landing page links to both other pages) by reading the component source directly instead - cheap, no browser needed, but honestly weaker than the old check: it can't catch a runtime error, a build problem, or components wired together incorrectly, only a real browser check can. Attempted an automated headless-browser smoke test as a stronger substitute (Playwright) - blocked by this sandbox's network allowlist (couldn't download a Chromium binary), not a decision to skip it. Live-verified everything that curl-based checks over an unauthenticated HTTP API actually can check instead: all three routes return 200 with the right content-type and reference real built assets, and the `/api/ask / /api/dashboard/summary` endpoints still return correct data through the new frontend's exact request shape. Told Adem directly this gap exists and to do one real look-and-click pass himself before the demo - not something to quietly consider "done."

15/15 `test_backend.py` tests pass (updated), 7/7 new `test_frontend_source.py` tests pass. No changes to `agent/`, `modeling/`, `knowledge/`, `parsing/`, or `llm/` in this

entry - purely a frontend/Docker change, so the whole existing DAM-answering test suite is unaffected by construction, not just by re-running it.

2026-08-06 (later again) - Five real gaps found from Adem's first live look at the React build

Adem ran the rebuilt frontend and reported back with a real screenshot: two glossary queries failed ("what does RDG stands for?", "what does RDNG STANDS for?"), the chat felt too wide/sparse with no way to keep old conversations, and the agent read as flat/scripted. All five real, not cosmetic complaints - fixed in order of how they were found.

1. **"stands for" (not "stand for") never matched.** Adem's exact query used the natural, if grammatically off, third-person form -agent/glossary.py's trigger regex only had the literal substring "stand for", so "stands for" (an extra "s") silently missed it entirely. Widened the pattern to accept mean|means|stand for|stands for. 2 new parametrized cases pin the exact query Adem typed.
1. **"RDNG" isn't in the DAM's own Abbreviations list.** Checked directly: `responsible_roles()` shows real DAM role names using "RDNG" verbatim ("RDG / Director RDNG", "Country Manager / DDG RDG / Director RDNG"), but the Abbreviations pages (2-7) only ever define "RDG" - with a note that RDG "also covers the Director of the Nigeria Country Office," never spelling out "RDNG" as its own entry. Not a parsing bug (`abbreviations.json` correctly reflects what the PDF actually says) - a real gap between what the DAM *uses* and what its own front matter *defines*. Fixed by adding a small, clearly-separate `MANUAL_ALIASES` dict in `agent/glossary.py` rather than folding it into `abbreviations.json`, specifically so that file stays what it's documented to be: a pure, reproducible extraction of pages 2-7 you could regenerate from the PDF and get byte-identical - not something with silent hand edits mixed in.
1. **Chat page was full-width with a dead gray gap in the middle.** `#chat` had no max-width, so on anything wider than a laptop the message column stretched edge to edge instead of staying readable. Fixed with a `.chat-inner / .form-inner` wrapper (`max-width: 760px, centered`) - the same fix ended up doing double duty once the sidebar (next item) also gave the page real horizontal structure instead of one wide empty canvas.
1. **No conversation history.** Asked Adem to pick a scope, since this is genuinely two different amounts of work: persist the current chat across a reload, or a real multi-conversation history like ChatGPT's. He chose the bigger one. Built

webapp/frontend/src/hooks/useConversations.js (conversations array + active id, persisted to localStorage under dam-agent-conversations-v1, every read/write wrapped so a private-browsing block or full quota fails open into "just don't persist," never a crash) and ConversationSidebar (new chat, switch, delete -an empty "New chat" you're currently composing in doesn't get its own row, so clicking "+ New chat" repeatedly doesn't pile up blank entries). Conversation titles auto-derive from the first message, ChatGPT-style. One correctness detail worth recording: the in-flight question's target conversation id is captured before the await, not read again after - so if you switch conversations while a question is still resolving, the answer lands in the conversation that actually asked it, not whatever's on screen when it comes back. There's still no accounts/backend session layer behind this - it's genuinely per-browser, which is an honest limitation of a localStorage-only design, not a shortcut being hidden.

1. **"Doesn't feel alive" - no personality or tone.** Deliberately did NOT touch the deterministic template answers themselves ("For X, the following approve(s): ...") - that rigidity is a feature, not a flaw, it's the always-correct fallback and several tests pin its exact wording. Personality has to live in the parts that were always presentation, not fact-delivery:
 - agent/smalltalk.py: each canned-reply category now has 2-3 varied lines (picked via random.choice) instead of one fixed string repeated on every greeting - still says the same substantive thing every time, just doesn't read as a script. The tests pin required substrings ("Hello", "Goodbye", "welcome", "Delegation of Authority Matrix"), not exact strings, specifically so this kind of variation stays safe to add.
 - agent/generate.py's SYSTEM_PROMPT (the LLM-phrasing prompt, only used when an LLM mode is on): now explicitly asks for a warm, conversational tone with a named identity ("the DAM Agent")
 - but the grounding rules right below it are unchanged, and _mentions_expected_facts() (the actual enforcement, not just a prompt request) still requires every real role name verbatim in the output. Tone is a request; grounding is a check - only one of those is allowed to be optional.
 - New TypingIndicator component (Chat.jsx) - three animated dots shown while a question is in flight, instead of just a disabled Send button. Small, but a disabled button reads as "broken", three bouncing dots reads as "thinking."

Rebuilt via the same /tmp workaround as before (OneDrive-synced webapp/frontend/ is too slow for npm install/node_modules - see the 2026-08-06 "still

later" entry). All 61 backend-side tests (`test_backend.py`, `test_smalltalk.py`, `test_generate.py`, `test_glossary_agent.py`) plus the 7 `test_frontend_source.py` checks pass; 4 new tests added (2 for "stands for", 2 for RDNG). Live-verified against the rebuilt server: both glossary queries from Adem's screenshot now resolve correctly, and three repeated "hi" calls came back with different (but still substantively correct) greetings, confirming the variation actually reaches a real request and isn't just correct in isolation.

2026-08-06 (once more) - Deterministic typo tolerance, no LLM required

Adem asked for the agent to "understand human typos alone and fix and understand the prompt correctly on its own." Investigated where typos actually break the pipeline today before building anything - confirmed four real, concrete failures with actual queries: "who aproves 2.126" resolved the right node (id matching is a digit pattern, typo-immune) but silently lost the "approve" intent filter and dumped every responsibility on the task instead - looks like it worked, is actually quietly less useful than what was asked. "who aproves the qaurterly mision program" (no id at all) put more at risk: TF-IDF text search has zero inherent typo tolerance (a misspelled word is just an out-of-vocabulary token contributing nothing to the score). "helo" missed the anchored smalltalk regex entirely. "waht does DDG men" missed the glossary trigger phrasing entirely.

Design choice: deterministic correction, not an LLM. The LLM in this project has one job by design - rephrase already-retrieved facts, never touch retrieval or understanding (see the 2026-08-05 grounded-generation entry). Routing query understanding through an LLM instead would mean typo tolerance only works when an LLM mode is configured and reachable, adds real latency to every question, and makes "understanding" non-deterministic - a step backward from every design choice this project has made so far (TF-IDF over embeddings, grounded phrasing over free generation, honest failure over guessing). Built `knowledge/typo_correct.py`: `correct_words()` instead: offline, no network, no dependency beyond the `stdlib`'s `difflib` - works exactly the same whether Groq/Ollama are configured or not, same as everything else that actually has to be right.

How it works, and why it's conservative on purpose: replaces a word with its closest match in a supplied vocabulary only when it isn't already an exact match but clears a similarity ratio - a word with no close-enough match is left exactly as typed, not guessed at, the same "don't fabricate" discipline as the rest of this project's retrieval logic. Two guards found necessary, not assumed: words under 4 letters are skipped (edit-distance similarity is close to meaningless at 1-3 letters), and ALL-CAPS words are skipped entirely so a real DAM acronym (DDG, RDG, PGCL...)

never gets "corrected" into an unrelated common word just because it's short and out of vocabulary.

A real false positive found and fixed before trusting this at all: the first version (`min_ratio=0.82`, applied to the ~1400-word DAM title vocabulary for text search) broke an existing test `-test_unresolvable_free_text_query_is_honest_not_fabricated`, which asks about "random unrelated words banana spaceship" and expects an honest non-match. Turned out "unrelated" (correctly spelled, not a typo) was getting "corrected" into "related" (ratio 0.875 - they share a root, a textbook false positive for any edit-distance approach) purely because "related" happened to be the closest word in that vocabulary. Root cause: a large, organic-language vocabulary makes this kind of false positive likely, since real English words often resemble each other by coincidence. Fixed by raising the default threshold to 0.88 - empirically checked against every real typo case that motivated building this (0.889-0.941, all still pass) and against the false positive (0.875, now correctly excluded), not picked arbitrarily.

Wired in at four points, each reusing whatever vocabulary was already naturally available there rather than threading a new parameter through every function signature in the call chain:

- `agent/authority.py: detect_intent()` - corrects against the union of every intent's own keyword list (~35 words) before the existing substring-match logic. Only ever changes WHICH intent gets selected; never touches the actual action-code matching rules (A/A1/A2, C vs C1/C2 vs C3/C4) just below, so it can't reopen the "bare C is Check, not Consult" ambiguity that was already carefully fixed (confirmed with a dedicated regression test, not just assumed safe).
- `agent/smalltalk.py: detect_smalltalk()` - corrects against a small vocabulary of the words the anchored regex patterns actually use ("hello", "goodbye", "thanks", "help"...) before re-attempting the match.
- `agent/glossary.py: detect_glossary_query()` - corrects against the trigger phrasing's own words ("what", "does", "mean", "define"...)
- deliberately NOT including the term being asked about, so "what dose DDG mean" can't risk "correcting" DDG itself.
- `knowledge/search.py: search_by_text()` - corrects against the fitted TF-IDF vectorizer's own single-word vocabulary (its bigrams, e.g. "country strategy", are excluded - only single words are meaningful to correct an individual query word against).

Three of the four use a looser 0.75 threshold than the 0.88 default, not the same one everywhere - the 0.88 floor exists specifically to protect the *large, organic-language* vocabulary (knowledge/search.py's ~1400 DAM title words) from false positives like "unrelated"/"related". The other three vocabularies (intent keywords, smalltalk words, glossary triggers) are each small (16-35 words), curated, and semantically distinct from each other - checked directly against a batch of ~20 unrelated realistic words before lowering the threshold there (one weak, low-impact collision found: "form" -> "for" in the glossary vocabulary; zero collisions in the other two). The looser threshold is what lets short 4-letter trigger words survive a typo at all ("dose"/"does" and "waht"/"what" both only reach 0.75 - a short word's ratio ceiling under a one-character transposition is inherently lower than a longer word's).

Known, accepted limitation, not silently hidden: words under 4 letters are never corrected, by design (see above) - so "waht does DDG men" (typo'd "men" instead of "mean", 3 letters) still fails, even though "waht does DDG mean" and "what dose DDG mean" (single typos, "mean" spelled correctly) both work. Chasing every possible short-word typo combination wasn't judged worth the added false-positive risk this session - a real trade-off, made deliberately, not an oversight.

7 new tests in tests/test_typo_correct.py (the real typo cases, the "unrelated"/"related" false positive pinned as a regression guard, the short-word and ALL-CAPS guards, capitalization preservation), 4 new integration tests in tests/test_agent.py (typo'd intent keyword still resolves the right *and identical* answer as the clean spelling, typo'd free-text query with no id still resolves the right node, the consult/check action-code precision explicitly re-confirmed unaffected, the false-positive case re-confirmed honest), 3 new tests in tests/test_smalltalk.py. 144 tests pass across every file touched (test_agent.py, test_smalltalk.py, test_glossary_agent.py, test_typo_correct.py, test_backend.py, test_search.py, test_generate.py, test_dashboard_data.py) - zero regressions. Live-verified against the real running server: typo'd id query, typo'd free-text query, typo'd greeting, and single-typo glossary query all resolve correctly; the genuinely-unrelated query still answers honestly instead of guessing.

2026-08-06 (final for now) - Auto now prefers the API over local

Adem asked to reverse Auto's preference order: API (Groq) first, local (Ollama) as fallback, not the other way around. Changed llm/router.py: resolve_provider("auto") - checks GROQ_API_KEY first and returns GroqProvider() immediately if set, only falling back to checking OllamaProvider().is_available() when no key is configured. Updated the 3 router tests in tests/test_llm.py for the new order (one now asserts is_available() is never even called when a Groq key is

present, not just that Groq wins) and the frontend picker label (webapp/frontend/src/components/LlmPicker.jsx: "API first, local fallback"). Also fixed, not just documented this time, the recurring GROQ_API_KEY test-isolation leak flagged in the 2026-08-06 glossary entry (test_chat_without_api_key_raises_unavailable_no_network_call now explicitly clears the env var via monkeypatch instead of relying on it being absent) - it kept resurfacing every time this session ran the fuller suite, cheap enough to close for good rather than re-flag again.

30/30 relevant tests pass. Live-verified: Auto mode now resolves to llm_provider: "groq" immediately (confirmed via the real .env key already on this machine) rather than checking Ollama first - the actual Groq network call itself failed in this sandbox specifically (ProxyError: 403 Forbidden reaching api.groq.com - this environment's own network allowlist, not a code issue), which usefully double-confirmed the existing fallback-to-template safety net still works correctly under a real provider failure.

2026-08-06 (even later still) - Free hosting: Render, docs/hosting.md

Researched free container hosts for the deployed demo. Render is the recommendation: Docker-native (builds straight from the existing multi-stage Dockerfile, no changes needed beyond the port fix below), no credit card, 750 free hours/month, 512MB RAM - enough for FastAPI + the agent, no local model involved. The real trade-off is a 15-minute inactivity sleep with a 30-60s cold-start wake, worth pinging before a live demo. Alternatives checked and ruled out: Railway's free tier is gutted to ~\$1/month credit now, Fly.io has no free tier and requires a card, Hugging Face Spaces' Docker SDK is paid-only now, Back4App's 256MB is too tight, Google Cloud Run is capable but needs a card and more setup than this project warrants. Koyeb noted as a viable secondary option if Render's cold start is a problem for a specific demo window.

Decided to drop Ollama entirely from the hosted deployment - no free tier has the RAM for even a small local model. Not a real loss: Auto already prefers Groq first (see the entry above), and the deterministic template answer is always the fallback if Groq is unreachable, so the hosted app only needs the app service from docker-compose.yml, not ollama, with GROQ_API_KEY set as an environment variable in Render's dashboard rather than committed to the repo.

One code change made to support this: Dockerfile's CMD switched from the JSON-array form with a hardcoded -port 8000 to the shell form CMD uvicorn webapp.backend:app -host 0.0.0.0 -port \${PORT:-8000}, because most free hosts (Render included) inject their own PORT env var at deploy time and expect the container to bind to it. Falls back to 8000 when PORT is unset, which

keeps local docker run / docker-compose behavior unchanged - confirmed docker-compose.yml doesn't set PORT and its healthcheck + port mapping are both hardcoded to 8000 already.

Known gap, not silently hidden: this Dockerfile change hasn't been build-tested. No docker binary has been available in this sandbox for the whole project, so this was validated by review only (checking docker-compose.yml's behavior is unaffected, checking the shell-form CMD syntax is correct). Worth one real docker build on Adem's own machine, or trusting Render's build log on first deploy, before relying on it for the actual demo.

Full deploy steps, and the reasoning above, written up in docs/hosting.md for Adem to follow directly.

2026-08-06 (later yet again) - Render deploy OOM: cache parsed nodes at build time

First real Render deploy attempt failed: "Ran out of memory (used over 512MB)". Root cause, confirmed directly (not guessed): every webapp/backend.py startup event was calling build_nodes(PDF_PATH), which opens the raw PDF with pdfplumber and does word- AND character-level extraction across every page, including a character-level fallback reconstruction pass for corrupted titles -real, repeated work, and real memory (pdfplumber/pdfminer cache font and page resources as they go, never explicitly flushed). Measured it directly: build_nodes() alone takes ~12.7s on this machine, and tests/test_column_roles.py (which independently re-opens and re-parses the whole document three more times across different tests) was the single slowest file in the suite - useful confirmation, in the test suite itself, that repeating this parse is expensive. Locally that's a non-issue; on Render's 512MB free instance it was enough to OOM-kill the container before it ever opened a port - the Render log showed "No open ports detected, continuing to scan..." right before the OOM kill, meaning the process was still parsing when it got killed.

build_graph(nodes) and build_search_index(nodes) were checked and confirmed NOT the problem - both are cheap, pure functions of the already-parsed nodes (networkx edge-building, TF-IDF over a few hundred short titles). The fix only needed to touch the PDF-parsing step.

Fix: stop re-parsing the PDF at runtime at all. Added modeling/nodes_cache.py (save_nodes() / load_nodes(), using Node.model_dump() / Node(**data) since Node is already a pydantic model - computed fields has_children/actions are excluded from the dump since they're 100% derivable and recomputed for free on load) and scripts/build_nodes_cache.py (a one-shot script that runs build_nodes() once and writes data/processed/nodes.json). Dockerfile now runs that

script as a build step (`RUN python scripts/build_nodes_cache.py`, after `COPY . .` and `pip install`), baking the cache into the image - a build environment generally has more headroom than a constrained free runtime instance, and this only needs to run once per image build, not once per container boot. `webapp/backend.py`'s startup event now prefers loading that cache and only falls back to a live `build_nodes()` parse if the cache file doesn't exist (fresh checkout with no cache built yet) - never a hard dependency on the cache being present. `data/processed/nodes.json` added to `.gitignore`, same "derived build artifact, not source" treatment as `webapp/static/`.

Verified the fix is actually lossless before trusting it, not just plausible: live-parsed and cached-then-reloaded node sets compared directly - 0 mismatches across all 327 nodes (`model_dump()` equal field-for-field), identical `build_graph()` output on both (same node count, same edge count, same skipped-reference list). Added `tests/test_nodes_cache.py` (2 new tests: the round-trip losslessness check as a permanent regression guard, and a check that computed fields are genuinely absent from the serialized JSON). Full existing suite re-run in batches (this sandbox's tool has a 45s per-call cap, and this project's test suite already took over that just from `build_nodes()` being called fresh in ~8 separate module-scoped fixtures across different test files, unrelated to this change) - all passed, including `test_backend.py`'s `nodes_loaded == 327` health check now exercising the actual cache-loading path.

Known gap, not silently hidden: this has not been verified inside an actual memory-constrained container (no docker binary in this sandbox, as with the earlier \$PORT fix). The fix is verified correct (lossless data, identical downstream graph/search behavior) and verified to remove the actual heavy operation from the runtime request path entirely - but the real proof is Adem's next Render deploy attempt succeeding, not a claim made here.

2026-08-06 (yet later still) - Hosting platform churn: Vercel, Netlify/Pages, Cloudflare, Koyeb all ruled out

Adem tried several alternatives to Render in quick succession, live, mid-session. Each was checked against this app's actual architecture (one persistent FastAPI process, in-memory startup state, a Docker multi-stage build) rather than assumed to just work:

- **Vercel** ("FastAPI preset"): serverless functions, not a persistent container - ignores the repo's `Dockerfile` entirely, so neither the React build step nor the nodes-cache build step would run, and the app's single-process in-memory state dict has no equivalent in a cold-start-per-invocation model.

- **GitHub Pages:** static files only, by design - literally cannot execute Python. Not a preference, a hard wall.
- **Netlify:** same serverless/no-Dockerfile mismatch as Vercel.
- **Cloudflare:** the piece that would actually fit (Containers) requires the Workers Paid plan, \$5/month - no free tier covers it. Confirmed via direct search, not assumed.
- **Koyeb:** architecturally the right fit (Docker-native, same model as Render) and was set up as the new primary recommendation in docs/hosting.md - then Adem's own dashboard showed "Koyeb is joining Mistral! Stay tuned for a revamped Agentic experience," confirmed via search to be a real Feb 2026 acquisition by Mistral AI, platform being folded into "Mistral Compute." Not disqualifying on its own, but the self-serve dashboard visibly not showing a normal "create service" flow mid-acquisition-transition is a real risk against a graded deadline - not worth trusting for this.

Ended back at Render, which was the right call from the start: the actual OOM root cause was already fixed (previous entry) and had simply never been retried. docs/hosting.md updated to lead with Render again; the Koyeb section kept as a documented, ruled-out alternative rather than deleted, so the reasoning isn't lost if free hosting needs revisiting later.

2026-08-06 (later once more) - Chat follow-ups losing context: the 2.118/3.226 misfire

Adem's live test: "who approves of Communication with Co-Financiers of projects" correctly resolved 2.118. The natural chat follow-up "who are the informed parties for that activity?" landed on 3.226 - a completely different node that happens to have an almost identical title ("...Communication with Co-Financiers of projects and third parties"). Root cause confirmed directly, not guessed: the follow-up names no real subject of its own - "informed", "parties", "activity" aren't in 2.118's title at all. Its only real content word, "parties", happens to appear in 3.226's title instead ("...and third parties"), so TF-IDF scored 3.226 at a comfortably "confident" 0.42 - well above MIN_TEXT_SEARCH_SCORE (0.15). This isn't a threshold-tuning problem (0.42 already clears any reasonable bar) - the backend genuinely had zero legitimate signal for the true intended target, and happened to get unlucky on which unrelated node it coincidentally overlapped with instead. The deeper issue: nothing in this app ever carried conversation context between turns at all, despite the frontend already keeping a full per-conversation message history in localStorage (2026-08-06, multi-conversation sidebar entry) purely for display.

Fix, kept deterministic on purpose - same principle as `knowledge/typo_correct.py`, this project's LLM is scoped to only phrase already-retrieved facts and never touches retrieval/ understanding: added `_refers_to_previous_context()` to `agent/qa.py`, a small fixed set of anaphoric trigger phrases ("that activity", "this one", "for that", "about this", etc.) plus a standalone "it" pronoun check. `answer_question()` gained a `previous_node_id=None` parameter; when the query has no digit-shaped substring (an explicit id always still wins - checked first) AND matches the referential pattern AND a `previous_node_id` was supplied, that node is used directly as the answer's subject (method: `"context_carryover"`) instead of running `resolve_query()` at all - skipping the exact trap that caused the misfire, not just working around it after the fact.

Wired end to end: `webapp/backend.py`'s `Question` model gained an optional `previous_node_id` field, passed straight through to `answer_question()`. `webapp/frontend/src/api.js`'s `askQuestion()` now takes and sends it. `Chat.jsx` now stores `nodeId` on each agent message's meta (it wasn't being kept at all before, only used inline for one render check) and, before sending each new question, scans the active conversation's own message history backward for the most recent agent `node_id` - the exact same "read before mutating" pattern already used for `targetId`. The UI also labels these answers distinctly ("carried over from previous question" instead of "matched by text search") so it's visible, not silent, when this path fires.

Known, deliberate trade-off: a bare "it"/"that"/"this" pronoun always assumes the user means the previous subject once a `previous_node_id` exists - there's no way to distinguish "who checks it" (meaning the same task) from a new, self-contained question that just happens to contain the word "it" incidentally. Judged the right default for a chat UI (matches how people actually use pronouns in conversation) rather than something to solve with more machinery right now.

Regression tests added: `tests/test_agent.py` (3 new - the exact 2.118/3.226 reproduction with the real follow-up wording, an explicit-id follow-up correctly ignoring `previous_node_id`, and a pronoun follow-up with no `previous_node_id` falling through to normal resolution unaffected) and `tests/test_backend.py` (2 new - the same reproduction through the real `/api/ask` endpoint, and confirming omitting `previous_node_id` entirely still works). 43 tests across both files pass. Frontend rebuilt via the established `/tmp` workaround (copying only source files, not the stale `node_modules` already sitting in the OneDrive-synced `webapp/frontend/` from an earlier session - copying that over the slow FUSE mount was itself the first attempt's timeout) and re-verified against `test_backend.py + test_frontend_source.py` (24 passed) after the rebuild.

2026-08-06 (once more still) - The phrase-list fix above got defeated within one message

Adem tested the fix above live and immediately broke it with a second, differently-worded follow-up: "and who are the informed partie?" (a typo, no "that activity", no "it" - none of `_REFERENCE_TRIGGERS` matched at all). It landed on the exact same wrong node, 3.226, at the exact same 0.42 score as the original bug. Measured directly rather than patching the phrase list again: `resolve_query()` against both worded differently ("...for that activity?" and "and who are the informed partie?") returns the identical 0.4223 score for 3.226 either way - proof the anaphoric phrase was never actually the operative signal, the score was, and a fixed phrase list was always going to be whack-a-mole against real rephrasing.

Replaced the phrase-list heuristic entirely with a score-based one, measured the same way `knowledge/typo_correct.py`'s threshold was tuned - real cases, not a guess. Ran `resolve_query()` against several genuine, self-contained new questions with real DAM vocabulary ("quarterly mission program" -> 0.888, "loan grant processing" -> 0.714) versus the two confirmed coincidental-overlap failures (0.42, 0.39 for "who checks it") - a clean gap between roughly 0.4 and 0.7. `CONTEXT_OVERRIDE_MAX_SCORE = 0.5` sits in that gap. `answer_question()` now always runs `resolve_query()` first (same as before typo-correction-era code), then only overrides its result with `previous_node_id` when the fresh text-search score is below that bar (or there's no match at all) - an explicit id, or a fresh match that clears 0.5 on its own merits, always wins regardless of conversation history. This is strictly more robust than matching specific wording, since it judges the actual evidence quality instead of guessing at phrasing patterns.

Known, deliberate trade-off, sharper than the previous entry's: a genuinely new but weakly-worded question (measured: "who checks the annual budget submission" -> 0.263, "what about approvals" -> 0.296) will now also get swept into carryover whenever a `previous_node_id` exists, even though it's a real, different topic - there's no way to tell "this is a follow-up" apart from "this is a new question my corpus just doesn't have strong vocabulary overlap for" using lexical score alone. Accepted because these were already low-confidence, borderline answers before this change existed (both comfortably above the 0.15 honesty floor but far from a real distinctive match) - this doesn't make an already-uncertain case more wrong, it just resolves the uncertainty toward conversational continuity instead of a coincidental unrelated node.

Tests: `tests/test_agent.py` gained the literal "and who are the informed partie?" reproduction (pinned specifically because it broke attempt #1) and a test confirming a

genuinely strong fresh match ("quarterly mission program") still overrides `previous_node_id` rather than getting swept into carryover. 21 tests in that file pass, 17 in `test_backend.py` unaffected (no frontend or endpoint contract change this time - the fix is entirely inside `agent/qa.py`).

2026-08-06 (final for real this time) - Mobile: hamburger sidebar, responsive header/chat/dashboard

Adem's last frontend ask: the chat and dashboard should look good on a phone, and the always-visible 240px conversation column should become a hamburger-triggered drawer instead of permanently eating a third of a narrow screen.

`Header.jsx` gained an optional `onMenuClick` prop - only Chat passes it (Dashboard has no sidebar to toggle), and the hamburger button it renders is CSS-hidden by default, only shown below the 720px breakpoint (`shared.css`'s `.hamburger-btn`). Same file's header rules gained a `flex-wrap` fallback below 720px so the LLM picker + "Dashboard →" nav link wrap to their own right-aligned row instead of overflowing measured this was a real risk, not a guess: hamburger + status dot + title on one side plus the picker + nav link on the other adds up to roughly 400px of content, more than a 375px phone viewport, if forced onto one row. The long subtitle text is hidden entirely below that breakpoint rather than truncated - it's supporting copy, not something worth fighting for space against actual controls.

`ConversationSidebar.jsx` gained `open/onClose` props - the aside's class now includes `open` when applicable, and selecting a conversation or starting a new one also calls `onClose` (auto-closes the drawer after a choice is made, standard mobile drawer UX). `conversationSidebar.css`'s existing 720px breakpoint (previously just shrank the always-visible column) now instead makes the sidebar `position: fixed`, transformed off-screen by default and slid in via `.open`, with a `.sidebar-backdrop` overlay to close on tap-outside. Desktop never sees any of this - the `fixed/transform` rules only exist inside the media query, so `.open` toggling is inert above 720px by construction, not by an extra guard that could drift out of sync.

`Chat.jsx` added `sidebarOpen` state, wired to `Header`'s `onMenuClick` and the sidebar's `open/onClose/backdrop-click`. `chat.css` and `dashboard.css` both got a tightened mobile breakpoint (smaller padding, narrower message bubbles at 88% width instead of a fixed 640px, smaller dashboard cards) - the existing desktop layout logic (centered `chat-inner`, cards auto-fit grid) already degraded reasonably on its own, this just trims the padding that felt oversized on a small screen. `.chat-page`'s `height: 100vh` gained a `100dvh` companion line (ignored by browsers that don't support it, overrides it on ones that do) - accounts for mobile browser chrome (address bar) eat-

ing into 100vh and clipping the fixed input bar, a well-known mobile Safari/Chrome quirk.

3 new source-level tests in `tests/test_frontend_source.py` (hamburger button wiring, Chat's sidebar-toggle state, the sidebar's off-canvas CSS) - same "check the source since there's no browser in this sandbox" approach as every other frontend guarantee in this file, with the same documented gap (a real device/browser check is still worth doing before the demo). 10 tests in that file pass, 24 across `test_backend.py` + `test_frontend_source.py` combined after the rebuild.

2026-08-06 (professor feedback round) - Dashboard rework: chapter filter, 2 new KPIs, linked charts

Adem's professor asked for two things after a review: the dashboard needed real improvement (more dynamic/interactive, filters, relevant KPIs), and the chatbot demo needed to be re-run with in-context questions rather than the out-of-scope ones tried live (which correctly failed - that's the honest-refusal behavior working as designed, not a bug).

Dashboard. Chose a chapter filter as the primary interactive control because it's a real, existing dimension on every node (`chapter`), not something invented to look interactive. Backend (`webapp/dashboard_data.py`) factored the existing computation into `_scope_summary(nodes)`, called once for the whole DAM (kept flattened at the top level - fully backward compatible, nothing existing broke) and once per chapter into a new `by_chapter` dict, plus a top-level `chapters` list. `graph` (`total_graph_nodes/total_edges`) stays whole-DAM-only on purpose - role/reference edges routinely cross chapter boundaries, so a "chapter subgraph" would need its own edge-filtering semantics that aren't worth the scope here; documented directly in the function's docstring so the reasoning doesn't need rediscovering later.

Two new KPIs, both real data-quality signals that were already computable but never surfaced: `avg_responsibilities_per_node` (workload density - `responsibilities ÷ answerable nodes`) and `no_direct_responsibilities_rate` (answerable nodes with nothing recorded directly on them - live data shows ~29.6% DAM-wide, a real, worth-discussing number, not a bug). "Answerable" is deliberately scoped to `RESPONSIBILITY_BEARING_TYPES = {process, task, child_task, threshold_variant}` - chapter nodes are organizational and always empty by design, including them would have diluted both metrics meaninglessly.

Linked charts: the action-code distribution chart is now clickable -selecting a bar filters the roles chart to that action's top performers (e.g. "who are the top approvers" vs. "who's informed most often"), backed by a new `roles_by_action` breakdown computed once server-side (same exclude-unresolved logic as `top_roles`, kept in one

place rather than duplicated in JS). Selecting a different chapter clears any selected action, since a chapter-scoped action selection can be meaningless (or simply absent) once the scope changes underneath it.

Real bug caught along the way: `ChartCanvas.jsx` built its `Chart.js` instance once on mount with an empty effect dependency array - nothing before this rework ever changed a chart's data after first render, so this had never been exercised. Fixed to depend on `[type, data, options]` so filtering (chapter or action) actually updates the charts instead of silently freezing them at their first-render values - would have been a real, confusing bug to discover live in front of the professor instead of here.

Non-breaking on purpose: every existing top-level field (`total_nodes`, `node_counts_by_type`, etc.) is unchanged, so `test_dashboard_summary_reflects_real_graph` in `test_backend.py` needed zero changes. 15 tests in `test_dashboard_data.py` (6 new, covering the per-chapter sum-reconciles-to-whole-DAM invariant, shape parity between whole-DAM and per-chapter breakdowns, and that `roles_by_action` never includes the "unresolved" placeholder), 3 new in `test_frontend_source.py`. Live-verified against the real running server (not just unit tests): confirmed the actual response shape, real per-chapter numbers, and `roles_by_action["A"]` surfacing sensible real approvers (Sector Manager, BOARD OF DIRECTORS, PRC).

Chatbot demo. No code change - the out-of-context failures the professor saw are correct, intentional behavior (this project's core grounding principle: honest "I don't know" over a guessed answer). What was actually missing was a prepared set of in-context questions for the live demo instead of improvising - drafted separately as `docs/demo_script.md`.

2026-08-06 (professor feedback round, chatbot majors 1-2) - Vague questions + out-of-scope detection

Same professor review also asked for three chatbot majors: multi-language support, better handling of vague/no-id questions from a new employee, and graceful out-of-scope + tone-aware responses. Built in priority order (agreed with Adem): vague-question handling and out-of-scope detection first (safest, fastest, no new external dependency), multi-language next, tone detection last (see the entries below for those).

Vague/ambiguous questions. Measured the real failure directly before designing anything: "mission" alone scores 0.48/0.45/0.41 across THREE different real tasks (2.121, 2.124, 2.125) - genuinely ambiguous, not a confident match that just happens to have a modest score. "approve" alone scores 0.46 against a single task (2.513.3) it has

no real reason to specifically mean. The old behavior silently picked `resolve_query`'s top guess and answered as if certain in both cases.

Two independent, deterministic signals added to `agent/qa.py` (`_needs_clarification`): too few real content words (`_content_word_count` strips English stopwords - sklearn's own list - plus this app's own intent-verb vocabulary, so a bare verb or a subject-less question counts as under-specified regardless of what `resolve_query` scores it), or a close score gap to the runner-up (even a longer, well-formed query can genuinely name 2+ plausible targets - "mission"'s 6% gap is the clean example). `CLARIFICATION_MAX_SCORE` (0.6) and `CLARIFICATION_MIN_GAP` (0.15) reuse the same measured cluster `CONTEXT_OVERRIDE_MAX_SCORE` was calibrated against earlier this session (genuine matches score 0.65-0.89 with real separation) - same "measure, don't guess" approach throughout. New method `"needs_clarification"` returned with the top 3 real candidates listed, same honest-suggestion pattern the invalid-id path already used.

Also widened `agent/smalltalk.py`'s `_HELP` trigger to catch "how does this work", "I'm new here", "where do I start" - a brand-new employee wouldn't necessarily type the literal word "help" - and added a third help reply variant explicitly telling them they don't need to know any DAM codes to get started, with a real example question.

Out-of-scope detection. The existing "zero TF-IDF matches" branch used one generic message for two very different real causes: a query that's only generic/intent words with no real subject at all ("informed" alone - still on-topic, just under-specified) versus a query with real, substantive content words that share nothing with the DAM's vocabulary at all ("what's the weather today", "who won the world cup" - genuinely outside scope). Split them using the same `_content_word_count` signal rather than a topic keyword list - deliberately avoided a fixed list of "off-topic subjects" since a fixed-phrase heuristic is exactly what got defeated by rephrasing earlier this session (see the context-carryover entries above) and would just be a slower version of the same mistake. New method `"out_of_scope"` with an explicit "that's outside what I can help with" message instead of the vaguer "couldn't find a task", which read like a search miss rather than "this isn't what I'm for."

Known, honestly-documented limitation: this classification is best-effort, not real topic understanding. "tell me a joke" lands in `"needs_clarification"` instead of `"out_of_scope"` (only "joke" survives word-stripping, exactly 1 content word - under the threshold either way). "write me a poem about spring" gets real but irrelevant suggestions ("Write-off decisions", "WRITE-OFFS") because the word "write" happens to overlap with real DAM vocabulary. Neither case ever fabricates a false DAM answer - the honesty guarantee holds either way - it just sometimes picks the less-precise of two truthful refusal messages. Fixing this properly would need real

semantic topic classification (embeddings or an LLM call), a bigger investment than this pass's scope; not worth chasing for a handful of whimsical edge cases when the core new-employee cases (single DAM-domain words, bare intent verbs) and the core off-topic cases (weather, sports, general knowledge) both work exactly as intended.

Existing message-text assertions in `test_agent.py`, `test_backend.py` updated to match the new, more specific wording (`test_generate.py`'s `NO_MATCH` fixture didn't need to change - it only tests `humanize_answer`'s handling of the `node_id`: `None` shape, not the exact wording). 4 new tests in `test_agent.py`, 1 in `test_backend.py`. 92 tests pass across `test_agent.py` + `test_backend.py` + `test_generate.py` + `test_smalltalk.py`.

2026-08-06 (professor feedback round, chatbot major 3) - Multi-language support (FR/ES/PT/AR)

Third of the three chatbot majors from the same professor review. Chosen architecture: a translation shim wrapped around the existing English-only deterministic pipeline, not a rebuild of that pipeline per language. Every deterministic layer in this project (typo correction, intent detection, context-carryover, vague/out-of-scope detection) is built directly around the DAM's own English vocabulary and score-threshold tuning - rebuilding all of that four times over was never realistic in this pass's scope, and would have meant re-deriving every "measure, don't guess" threshold in this log per language. Instead: translate the incoming question to English before it reaches `answer_question()` (retrieval is completely untouched, still the same tested logic), then translate the final answer back afterward. This is a deliberate, and the first, expansion of the LLM's role in this project beyond pure fact-*phrasing* - it's now also doing the language conversion - which is why the grounding check below was the main thing worth getting right.

New file `llm/translate.py`. `looks_non_english()` is a cheap, regex/wordlist-only pre-filter (Arabic Unicode range, accented Latin characters, or a whole-word hit against small French/Spanish/ Portuguese function-word lists) - not a real language detector, just enough to decide whether it's worth even attempting the expensive path. Gates the entire rest of the flow: since the large majority of real traffic is plain English, this keeps the extra Groq round trip (one call to detect+translate the question in, a second to translate the answer back out) off of every English question, which is the whole reason this is a separate cheap check rather than always asking the LLM to identify the language itself.

Real gap found while testing this against a realistic short question: "qui approuve 3.111" (natural French for "who approves 3.111") has only ONE real French function

word ("qui") once the DAM verb ("approuve") is excluded - the first version of `looks_non_english()` required 2+ hits per language and missed it outright. Fixed by (a) removing "o" and "as" from the Portuguese word list (both collide with ordinary English - "as needed", "as approved" - and neither is distinctive enough to trust alone) and (b) lowering the bar to a single whole-word hit. Verified this is safe specifically because a false positive here costs nothing but one extra, self-correcting round trip (the model reports `LANGUAGE: en` and returns the text unchanged) - checked against 5 real non-English examples (all detected) and 10 English edge cases including "who approves as required" and "and who needs to sign off on this" (all correctly left alone).

`detect_and_translate_to_english()` sends the raw question to the LLM with an explicit instruction to copy any digit-and-period id (e.g. 2.126) verbatim rather than translating it, and to respond in a strict two-line `LANGUAGE: xx / TRANSLATED: ...` format that's parsed with a couple of small regexes. Falls back to treating the original text as English, unchanged, on any failure - no provider, a network error, or a response that doesn't parse - so the caller always has something safe to run the untouched pipeline on. `translate_text()` is the simpler counterpart used for static English messages (smalltalk, help, vague/out-of-scope refusals) that carry no DAM facts worth grounding-checking, just a straight translation with the same safe fallback.

Grounding check across languages (`agent/generate.py`). `build_grounding_prompt()` now takes `target_language` and appends one additional numbered rule to the existing system prompt when it isn't English: write the answer in that language, but never translate role names, footnote numbers, or DAM ids - copy those exactly, even inside an otherwise-French/Spanish/Portuguese/Arabic sentence. This matters because `_mentions_expected_facts()`, the existing check that a rephrased answer still contains every real role name verbatim, was NOT changed - it still checks for the literal English role-name strings regardless of the surrounding sentence's language. That's the actual safety property this feature depends on: a French answer that dropped or altered a role name still fails the same grounding check an English answer would, and falls back to the same safe deterministic template (in English - a known, accepted trade-off over the alternative of a translated-but-unverified answer, documented in `webapp/backend.py`'s docstring). Two new tests in `test_generate.py` pin this directly: a French answer that correctly preserved both role names passes, and one that dropped one still fails and falls back.

Backend wiring (`webapp/backend.py`). `/api/ask now:` resolves the LLM provider up front (previously resolved deeper in the handler), runs `looks_non_english()` on the raw question, and - only when it looks non-English AND a provider is available - translates to English before calling the unchanged `answer_question()`. Two

paths for translating the answer back: through `humanize_answer()`'s grounding-checked path when the resolved node has real facts to protect, or through the simpler `translate_text()` when it doesn't (smalltalk, vague, out-of-scope, invalid-id - nothing to fabricate). Two new response fields: `detected_language` (defaults to "en") and `translation_error` (set, honestly, when a non-English-looking question couldn't be translated - most notably when `llm=off`, since translation has no deterministic fallback the way retrieval does). That honesty was itself a deliberate choice over silently running non-English text through an English-only search index and most likely returning a confusing "out of scope" refusal with no explanation why.

Frontend. `Chat.jsx` surfaces `detected_language` as a small badge next to the existing confidence/LLM-provider meta line (mirrors the `llm-badge` pattern already there), and `translation_error` as a warning badge reusing the existing `llm-fallback` style - both were a deliberate requirement, not a nice-to-have: the translation happening silently with zero on-screen indication would look like the agent either mysteriously understanding French or mysteriously failing on it, with no way for Adem (or the professor, live) to tell which. Rebuilt via the established /tmp workaround (this rebuild also caught a copy-paste gap - only `index.html` had been copied, not the project's actual three multi-page entry points `landing.html` / `chat.html` / `dashboard.html` - fixed before the build would even resolve its entry modules).

Tests. New `tests/test_translate.py` (24 tests) covering `looks_non_english()`'s true/false/tricky-false-positive cases and both translation functions' success/fallback paths. 3 new integration tests in `test_backend.py`, including one that mocks `GroqProvider.chat` directly (not `requests.post - chat()` raises before ever reaching the HTTP layer when `GROQ_API_KEY` is unset, which is the case in this sandbox, so an HTTP-layer mock would silently never fire) with two canned responses in sequence for the detect+translate-in and translate-out round trip. 2 new tests in `test_generate.py` for the cross-language grounding check. 1 new source-level test in `test_frontend_source.py` pinning the language badge's presence. Live-verified end to end via `TestClient` against the real running app: a mocked "qui approuve 3.111" round trip returns `detected_language: "fr"` and a correctly-grounded French answer. 114 tests pass across `test_frontend_source.py` + `test_backend.py` + `test_generate.py` + `test_translate.py` + `test_agent.py` + `test_dashboard_data.py`.

2026-09-03 - Live bug: Groq 404 on the deployed instance, llama-3.3-70b-versatile deprecated

Adem hit a real error on the live Render deployment - a tooltip in the chat UI reading Groq request failed (model=llama-3.3-70b-versatile): 404 Client Error: Not Found for url: <https://api.groq.com/openai/v1/chat/completions>. Confirmed against Groq's own current docs (console.groq.com/docs/models, fetched live, not assumed from training knowledge - the same discipline used for the original model name back on 2026-08-05) rather than guessed: Groq deprecated llama-3.3-70b-versatile for standard/developer API keys on 2026-08-16, moving it to Enterprise-only, contact-sales pricing. A regular key gets a clean 404 - the model id still exists, it's just no longer reachable on this account's plan. This is an external-service change, not a bug in this project's own code, but it broke the LLM path on the deployed instance regardless.

Fix: moved DEFAULT_MODEL in `llm/groq_provider.py` from `llama-3.3-70b-versatile` to `openai/gpt-oss-120b` - currently Groq's flagship model on the standard production tier (per the same docs fetch): similar speed (~500 t/s vs. the old model's 280 t/s), 131K context, and strong enough instruction-following for this app's two strict-format prompts (the grounded-answer system prompt in `agent/generate.py` and the two-line LANGUAGE: /TRANSLATED: format in `llm/translate.py`) - both depend on the model reliably following exact formatting, so a real quality/instruction-following floor mattered here, not just "cheapest available."

Also updated every place the old model name was hardcoded or documented, so nothing in the repo could regress back to it silently: `.env.example`, the actual local `.env` (which still had `GROQ_MODEL=llama-3.3-70b-versatile` pinned, overriding the code default entirely - the real reason the fresh code default alone wouldn't have fixed the local reproduction), `docs/llm_setup.md`, `tests/test_backend.py`'s `test_llm_config_reports_the_real_resolved_model_names`, and the example model names in `webapp/backend.py`'s `llm_config()` docstring. `tests/test_llm.py`'s explicit test-only model string was also updated for consistency, though it wasn't actually load-bearing (mocked, never a real call).

Not independently live-verified against the real Groq API from this environment - the sandbox's outbound network is proxy-restricted and blocks `api.groq.com` (confirmed: a real call attempt failed with a 407/403 proxy tunnel error, not a Groq-side error), so this fix rests on Groq's own current docs listing `openai/gpt-oss-120b` as a live production model with standard pricing, not on a real round trip from this session. Verify with a real question against the actual deployment after the next deploy.

Action still needed on Render (outside this session's reach): if Render's own environment variables have `GROQ_MODEL` set explicitly (separate from what's in this repo's `.env/.env.example`, which Render never reads), that value overrides the new code default the same way the local `.env` did - check Render's dashboard and update or remove it if present.

2026-09-03 (later) - Explicit language picker, not just auto-detect

Adem tested the live deployment and reported the multi-language feature from earlier this session had no visible way to use it - no button or control anywhere on the page, and he wanted users to be able to actively pick a language and have it "translate all the data to better understand the answers." Real gap in the original design: the 2026-08-06 multi-language work only auto-detected the language FROM the question text itself - there was never a way to ask an English question and get a non-English answer, or vice versa, and nothing in the UI signaled the capability existed at all before you'd already typed a non-English question and gotten a translated reply back.

Two design forks worth deciding deliberately rather than guessing, so these were put to Adem directly before building: (1) should a selected language be an **explicit override** (always wins, regardless of what language the question itself is typed in) or just a **default/fallback** underneath the existing per-message auto-detect - chose explicit override, since that's what "select and change it" actually implies and it's the simpler mental model; (2) should the surrounding UI chrome (Send button, input placeholder, meta labels) switch language too, or just the agent's answers - chose full UI translation.

Backend: two independent language concerns, deliberately decoupled (`webapp/backend.py`). The existing auto-detect logic answers "what language do we need to translate the QUESTION *from*, to run it through the English-only retrieval pipeline" - that's unchanged and still runs regardless of the picker. A new, separate `target_language` field on the Question model answers a different question entirely: "what language should the ANSWER be phrased *in*." `answer_language = target_language` when it's set to anything other than `None` / "auto", else falls back to whatever the question was detected as (the original, pre-picker behavior - fully backward compatible, the existing French-question auto-detect test needed no changes). This split is what lets someone type an English question with the picker set to French and get a French answer, or type a French question with the picker forced to English and get an English one - two real test cases, both passing (`test_ask_explicit_target_language_overrides_detected_language`, `test_ask_explicit_english_target_overrides_a_french_question`).

Also extended the existing honesty guarantee: if a non-English `answer_language` is requested (explicitly or via detection) but no LLM provider is available (`llm=off`), `translation_error` is now set even when the question itself was plain English (previously that field only ever got set when the `*question*` needed translating and couldn't be) - same "never silently substitute a language you can't actually produce" principle as the rest of this feature.

Frontend chrome translation - a second, separate mechanism, on purpose (`webapp/frontend/src/i18n.js`). This is NOT routed through the LLM translation shim in `llm/translate.py` - that path exists for arbitrary DAM answer text where the exact wording can't be known ahead of time and needs a grounding check. UI chrome is the opposite case: a small, fixed set of ~15 strings (subtitle, placeholder, send button, meta labels, empty state, error message) that are identical on every page load. Translating those via an LLM call would be slower, cost money per page view, and could phrase the same button differently between reloads - exactly wrong for interface text that should feel stable. Hand-translated once into a plain dictionary instead (`UI_STRINGS`, keyed by language code), the same "deterministic where the content is fixed, LLM only where it has to be" principle used throughout this project (typo correction, intent detection, glossary lookup are all in this same category). Scoped to the Chat page only, not the conversation sidebar or Dashboard - a reasonable, explicitly documented boundary for this pass, not an oversight.

New LanguagePicker component (`webapp/frontend/src/components/LanguagePicker.jsx`), styled to match the existing `LlmPicker` pattern exactly (same dropdown structure, `languagePicker.css` mirrors `llmPicker.css`) so it reads as "another mode selector" rather than a bolted-on control. Sits next to the LLM picker in the header. Six options: Auto (the original detect-from-question behavior) plus the five supported languages, each self-labeled in its own language (Français, Español, Português, Arabic (native-script label)) rather than translated from English - a picker option should say what it is in the language it represents.

Arabic is RTL - a real layout correctness issue, not just a translation one. Chat now sets `dir="rtl"` on the page root whenever Arabic is selected (`RTL_LANGUAGES` set in `i18n.js`), so text alignment and reading order flip correctly instead of rendering Arabic text left-aligned in an otherwise LTR layout, which would have looked visibly broken in a live demo.

Badge logic tightened. The per-message language badge on each answer now reads `meta.answerLanguage` together with `meta.usedLlm` -deliberately not `answerLanguage` alone, since that field reflects what was `*requested*`, not what was `*achieved*` (see the backend section above - a failed/unavailable LLM still reports the requested `answer_language`). Showing a "answered in French" badge on an answer

that actually fell back to English would be a real, avoidable dishonesty bug; the existing fallback-warning badge now also covers this specific "language requested but not delivered" case, reusing `llm_error/translation_error` for the tooltip.

Tests. 4 new backend tests in `test_backend.py` covering the explicit-override, `llm-off-honesty`, `reverse-direction`, and `"auto"-still-behaves-as-before` cases. 2 new source-level tests in `test_frontend_source.py` pinning the picker's existence, its six option values, and that `api.js` actually sends `target_language`. Rebuilt via the established `/tmp` workaround and live-verified end to end via `TestClient`: an English question with `target_language: "fr"` came back in French with `detected_language: "en" / answer_language: "fr"`; the same request with `llm: "off"` correctly fell back to the English deterministic answer with an honest `translation_error`. 135 tests pass across `test_frontend_source.py + test_backend.py + test_generate.py + test_translate.py + test_agent.py + test_dashboard_data.py + test_llm.py`.

Known, honestly-documented limitation: the five UI-chrome dictionaries were hand-translated by the agent, not reviewed by a native speaker of each language - reasonable for interface labels ("Send", "Dashboard", a placeholder) where a slightly stiff phrasing is a minor cosmetic issue, not a correctness one, but worth a native read-through before this is shown to French/Spanish/Portuguese/Arabic speakers in a formal setting. The actual DAM facts in every answer are never at risk either way - those still go through the grounding check in `agent/generate.py`, unchanged by this feature.

2026-09-03 (later still) - Domain rule: Check/Verify is mandatory, always surfaced

New domain knowledge from Adem, from the actual internship context rather than anything derivable from the DAM PDF's formatting alone: when a task's DAM row records a Check/Verify (C/C1/C2) entity, that step is a mandatory part of the process - not optional supplementary information. The agent's existing behavior was to answer only the specific action asked about ("who approves X" showed only the A-coded role, nothing else), which meant a real, mandatory obligation on the task could be completely invisible to someone who happened to ask a narrower question. Adem also asked for the informed-party ((i)) role to get the same treatment "when it feels needed."

Two design questions put to Adem directly before writing any code, since both changed what every intent-specific answer contains: (1) should this note apply to every intent asked about the task (approve, review, initiate...) or only to approve-specific questions - chose "every intent," since the underlying rule is a property

of the task, not of the specific question; (2) should the informed-party mention be deterministic-always-when-present (matching the Check/Verify rule exactly) or left out of this pass, since "when it feels needed" isn't buildable as a reliable rule without either an LLM judgment call or an undertested heuristic - chose "always," for the same consistency and testability reasons the rest of this project has favored deterministic rules over LLM guesses throughout (typo correction, intent detection, vague/out-of-scope detection are all in this category).

Implementation (agent/qa.py). Two small filters added `_check_verify_roles()` (action in C/C1/C2, deliberately excluding C3/C4 consult - the same split agent/authority.py's "check" vs "consult" intents already draw) and `_informed_roles()` (action (i)). A new `_format_mandatory_notes(roles, intent_name)` builds up to two trailing sentences - "This task must also be checked/ verified by X." and "Y must also be informed." - skipping whichever one matches the intent actually asked about, so "who checks 2.126" doesn't redundantly repeat its own main answer as a note. `_format_intent_answer()` appends this to both of its existing outcomes (a normal role-list answer, and the "no one is recorded to X" case - a task can have no approver recorded but still have a real, mandatory check/verify step, so the note has to survive that branch too).

Grounding check extended to cover the new facts. The roles list returned by `answer_question()` (consumed both directly by the frontend and as the "verified facts" an LLM rephrasing must preserve, per agent/generate.py's `_mentions_expected_facts`) previously only contained the roles matching the asked intent. Now it also includes the check/verify and informed roles whenever they're part of the answer text - otherwise an LLM-phrased rephrasing could silently drop the new mandatory notes without the grounding check ever catching it, which would have made this feature's core promise (mandatory information is never invisible) untrustworthy the moment Groq/Ollama phrasing was turned on.

Verified against real data before writing tests, same discipline as everywhere else in this project: queried the actual built graph for a node with both a Check role and other action types (found 2.126, "Quarterly Mission program" - already used elsewhere in this project's tests and demo script) and confirmed the real generated answer text before pinning it. New tests in `test_agent.py`: the check/informed notes appear on an approve question and are included in roles (grounding-check-safe); a check-intent question doesn't redundantly repeat its own note; an informed-intent question doesn't redundantly repeat its own note either. 4 existing `test_backend.py` tests needed their canned/mock LLM replies updated to also mention 2.126/3.111's real informed-party role names, since the grounding check now correctly requires those too - this is the intended behavior showing up as expected, not a regression to work

around. 81 tests pass across `test_agent.py` + `test_backend.py` + `test_generate.py` + `test_dashboard_data.py`.

Demo script updated (`docs/demo_script.md`, section 1) - the existing "who approves 2.126" question already used in the live-demo script now also shows this feature in the same answer, so a short explanation of the new behavior was added right there rather than as a separate section.

2026-09-03 (once more) - Real live bug: action-code questions swallowed by context-carryover

Adem hit this live: asked "tell me about 2.515.2", then as a natural follow-up "what's I, A and (i)?" - expecting an explanation of what those action-code letters mean. Instead the agent just re-answered 2.515.2's full breakdown again, unchanged, labelled "carried over from previous question."

Root cause, found by tracing the actual code path rather than guessing: `_content_word_count()` (the signal `_needs_clarification/context-carryover` both key off) strips any word under 3 characters, which means bare letters like "I", "A", "C" - and stopwords like "what's" - all get stripped, leaving a content-word count of 0. That's indistinguishable, to the existing heuristic, from a genuinely vague follow-up like "and that one?" - so it fell straight into the context-carryover path and just re-answered the open task.

The actual right fix wasn't a tweak to the vague-question threshold - it was that this class of question needed its own detection path entirely, run BEFORE context-carryover ever gets a chance to fire, same as smalltalk and glossary questions already do. And the data to answer it correctly already existed: `data/reference/authority_codes.json`, with the DAM's own action-code legend text extracted (role of I/C/C1-C4/R/A/A1-A3/(i), levels, colors) - carried over from v1's work, and `schema/schema.py`'s own `AuthorityCode` model was literally designed for this back on 2026-07-21 ("needed as reference data for the agent's `explain_authority / glossary-lookup` capability") - but nothing in the v2 agent had ever actually read it. This was unfinished planned scope, not new scope.

New `agent/action_codes.py`, same shape as `agent/glossary.py`: `detect_action_code_query()` + `format_action_code_answer()`. The detection is deliberately conservative about bare single-letter codes - "I" in particular collides constantly with the pronoun - so a bare code only counts as a real mention when it appears alongside a second code (a list, like the real "I, A and (i)" case) or is itself unambiguous on its own ((i), or a letter+digit combination like "A2"/"C1" that isn't a plausible English word). Checked directly against a batch of ordinary questions before trusting this ("what is 2.120", "I approve this", "A2 is fine", "C is for cookie" - all correctly

non-matches) - same verify-before-trust discipline used for every other regex-based detector in this project. A specific code gets its full DAM-sourced definition; a generic "explain the action codes" with no code named gets the six top-level codes (I, C, C3, R, A, (i)) with a pointer to ask about a specific numbered level for more detail.

Wired into agent/qa.py's `answer_question()` right after glossary detection, before `resolve_query()/context-carryover` ever run - the same position in the pipeline glossary and smalltalk already occupy, for the same reason: these are all "answer this without touching node resolution at all" cases.

Verified against the exact real failure, live, via `TestClient`: asked "tell me about 2.515.2" then "what's I, A and (i)?" with `previous_node_id` set to 2.515.2 - now returns `method: "action_code_legend", node_id: None`, and the real three definitions, instead of re-answering 2.515.2. New tests/`test_action_codes.py` (8 tests: the real screenshot query, five more real trigger phrasings, ten deliberately-tricky non-matches including the "I" pronoun collision risk, and the two "unambiguous alone" cases for suffixed codes and (i)). New regression test in `test_agent.py` pinning the exact end-to-end scenario (open a task, then ask the action-code question, confirm it does NOT carry over). 102 tests pass across `test_agent.py + test_action_codes.py + test_backend.py + test_generate.py + test_dashboard_data.py`.

2026-09-03 (yet again) - Real live bug: "LLM unavailable" fallback firing too often

Adem reported the Groq-phrased answers were falling back to the deterministic template too frequently - specifically flagged via a screenshot showing "LLM output failed the grounding check (missing/ altered role names)" on an otherwise-ordinary full-breakdown answer. Investigated two candidate causes; found and fixed one real bug along the way, chased and correctly abandoned a second one.

Candidate 1 (chased, then abandoned): the "Task Manager1" garbled role name. The screenshot's node (2.224.1) has a role literally stored as "Task Manager1 & Project Team Members" - no space before the 1. Scanned the entire dataset first (not just this one node): exactly one distinct garbled role name across all 327 nodes (also affects 2.325.2, same shared role text), so this is narrow, not systemic. Traced it to the actual character geometry on the source PDF page via `pdfplumber` (not guessed): the raw character stream has . . . r (the last letter of "Manager") immediately followed by a shrunk 1 (size 3.24 vs. neighboring letters' 3.99-4.5) with NO space character between them anywhere in the extracted stream, while real spaces DO exist immediately before and after that 1 elsewhere in the same run. This looks like a mid-string shrunk footnote-style digit that `parsing/column_roles.py`'s `_strip_trailing_footnote_digit` doesn't catch, because that function - correctly, given

every other known real case - only strips a shrunk digit from the very END of a run, and this one sits mid-run (before "& Project Team Members" continues after it).

First attempted fix: always insert a space at the header-line merge boundary in `extract_column_headers` (reasoning: `COLUMN_MERGE_GAP` merges wrapped header-line runs, and `" ".join(group_text.split())` already collapses any resulting double space, so this looked safe). Rebuilt and checked against the real 2.224.1 data: **the "Task Manager1" text was completely unchanged** - confirming this merge boundary was never where the actual bug lives (the missing space is inside a single character run, not between two merged runs). Worse, running the full test suite turned up a real regression this "fix" introduced elsewhere: "Sector Manager (HQ-based / Region-based)" on node 2.126 became "Sector Manager (HQ- based / Region-based)" - a DIFFERENT merge boundary that happens to fall right after a hyphen, which correctly needs NO space, and this change forced one in anyway. **Reverted the change entirely** rather than keep a "fix" that didn't fix its target and broke a previously-correct case -verified via `test_column_roles.py`'s full 8-test suite passing again after the revert. The narrow mid-run shrunk-digit case is left as a known, honestly-documented data-quality limitation (affects 2 of 327 nodes) rather than risking another blind fix to geometry code that several other real, hard-won bug fixes already depend on staying correct.

Candidate 2 (the real fix): fact count outgrew the sentence budget. The mandatory Check/Verify + informed-party notes feature (shipped earlier the same day) routinely pushes a real answer's fact count from 1-2 roles (just the specific action asked about) to 4-6 roles. `agent/ generate.py`'s system prompt rule 5 ("Answer in 1-3 sentences") was never revisited when that feature was built - and squeezing 4-6 role names into 3 sentences while ALSO keeping every one of them verbatim intact (rule 4) is a genuinely hard constraint to satisfy at once. The model's honest ways to resolve that pressure are to drop a role, paraphrase/shorten a name, or run longer than 3 sentences - only the third doesn't silently fail the grounding check.

Fix: `MANY_FACTS_THRESHOLD = 4` - when `structured_result["roles"]` has 4 or more entries, `build_grounding_prompt()` swaps rule 5 for a variant that explicitly lifts the sentence cap ("use as many sentences as you need... a longer, complete answer is much better than a short one that drops or blends any of them") instead of leaving the model to guess how to trade off length against accuracy. Rule 4 (verbatim accuracy) is completely untouched - this only removes brevity pressure, never relaxes what has to survive.

Also fixed alongside it, a smaller but real gap: `_mentions_ expected_facts()` compared the LLM's text against each role name with a raw, case-sensitive, whitespace-sensitive substring check. A model introducing purely incidental for-

matting noise - a double space, or different capitalization mid-sentence - would fail the grounding check over nothing but that noise, not an actual missing or altered fact. Added `_normalize_for_match()` (collapse whitespace, lowercase) applied to both sides of the comparison before checking. Deliberately does NOT weaken the actual guarantee: the same words, in the same order, still have to be present - checked directly with a test that a genuinely different/wrong role name still correctly fails after normalization, not just the incidental-formatting case.

Tests. 6 new tests in `test_generate.py`: few-facts keeps the short cap, many-facts switches to the uncapped rule, a real many-facts answer that faithfully states every role now succeeds, a many-facts answer that drops a role still correctly fails, whitespace/case noise is tolerated, and a genuinely wrong name still fails. 101 tests pass across `test_generate.py` + `test_backend.py` + `test_agent.py` + `test_action_codes.py` + `test_column_roles.py`.

Not independently verified against a real Groq call - same sandbox network restriction as the earlier Groq-model fix (outbound requests to `api.groq.com` are proxy-blocked here) - so the actual real-world reduction in fallback frequency rests on the reasoning above (fewer facts-vs-brevity conflicts, fewer false rejections from formatting noise) rather than a measured before/after rate. Worth Adem watching whether "LLM unavailable" shows up less often in practice after this deploys.

2026-09-03 - Chatbot major 4/4: LLM-based tone detection

Last of the four chatbot majors Adem asked for this session (multi-language, mandatory Check/Verify notes, action-code fix, and this). Request: detect when a user sounds frustrated or confused and respond more warmly - "let's go with the tone detection," confirmed earlier in the session as LLM-based (over a keyword-only heuristic) since tone is exactly the kind of thing a small classification call handles far more reliably than regex.

Two scoping questions asked via `AskUserQuestion` before building:

1. **Scope: every response type, not just grounded DAM answers.** Adem's choice. This matters architecturally - it rules out folding tone into `agent/generate.py`'s `humanize_answer()` prompt, since that path only runs when there's a real node/roles to ground an answer on. A frustrated "why won't this work AGAIN" asked as an out-of-scope question, or a confused "what does C1 mean" glossary lookup, still needed the same warmer opening. So tone is applied as a separate, final step in `webapp/backend.py`'s `ask()`, after `result["answer"]` is already fully built by whichever path produced it (grounded/LLM-phrased, translated-only, or fully deterministic).

1. **Cheap pre-filter before the LLM call.** Same cost-avoidance shape as `looks_non_english()` already established for translation: `llm/tone.py`'s `looks_emotional()` is a deterministic regex check (repeated `!!/??`, ALL-CAPS words, and phrase patterns like "still not working" / "I don't understand") that has to return True before `detect_tone()` ever spends a Groq call. An ordinary "who approves 3.111" never pays for tone classification at all.

Architecture, deliberately two separate concerns:

- **Detection** (`detect_tone()`): one cheap LLM call, `temperature=0`, `max_tokens=8`, asks for exactly one word (frustrated/confused/ neutral). Always defaults to "neutral" on any failure - no provider, network error (`LLMUnavailableError`), or an unparseable reply - the same "never guess wrong, fail safe" principle as `detect_and_translate_to_english()`. Tolerates incidental formatting noise in the reply ("Frustrated." still parses as frustrated).
- **Response adaptation** (`apply_tone_prefix()`): NOT a second LLM call. A small, hand-translated, deterministic prefix (`EMPATHY_PREFIXES`, keyed by tone + language) prepended to whatever answer text already exists. Same reasoning as `i18n.js`'s UI strings and the note templates in `agent/qa.py`: this is a tiny, fixed set of strings, not dynamic content, so hand-translating once beats an LLM round trip on every use. Covers all 5 supported languages (en/fr/es/pt/ar), matching the language picker's own set.

Detected from `payload.question` (what the user actually typed), never the English-translated pipeline text - tone is about how someone expressed themselves, not the retrieval-internal text. And the prefix itself is applied in `answer_language` (the same explicit-override-or-detected language the rest of the answer is already in), not the question's own language - so a French answer to a frustrated English question gets the French prefix, matching what the user actually sees.

`deterministic_answer` (the structured/template answer, used for the "show without LLM" toggle) deliberately never gets the prefix - it's a warmth/UX layer on top of an already-decided answer, not part of the trusted facts, and mixing it into the fact-bearing field would blur that boundary for no reason.

Tests. `tests/test_tone.py` (25 tests): the pre-filter's true/false cases against realistic phrasing, `detect_tone()`'s parsing/fallback behavior (including the no-provider and unparseable-reply paths), and `apply_tone_prefix()` across all 5 languages plus the neutral no-op and unsupported-language fallback. `tests/test_backend.py` (4 new integration tests): a frustrated grounded answer gets prefixed (with the two canned Groq replies queued in the actual call order - phrasing happens before tone

classification in `ask()`, a real ordering dependency worth keeping documented in the test itself, not just assumed); a confused *out-of-scope* question also gets prefixed, specifically to pin the "every response type" requirement since that path never touches `humanize_answer()`; a plain neutral question confirmed to never even call the LLM (`mock_chat.assert_not_called()`); and an LLM-off frustrated question stays neutral rather than fabricating a prefix with no real classification behind it. Full suite (all 23 test files) re-run after this feature: all pass, no regressions - the only surprise was that running every file back-to-back can exceed a 120s wall-clock budget purely from cumulative PDF-parsing time in the ~8 heaviest files (`test_build_nodes.py`, `test_graph.py`, etc., each ~12s alone), not from any actual hang; confirmed by isolating and timing each file individually.

Not independently verified against a real Groq call - same sandbox network restriction noted in every LLM-dependent feature this session. Deployed behavior (does "frustrated" actually show up, does the model respect the exactly-one-word format reliably) is worth Adem watching for after this ships, same as the grounding-fallback-frequency fix above.

No frontend changes needed. The prefix is baked directly into `result["answer"]`'s text server-side, not a separate UI element or badge - so despite covering "every response type," this shipped as a backend-only change (`llm/tone.py` + `webapp/backend.py`), with `Chat.jsx` untouched and no rebuild required.

Appendix G

Key Code Excerpts

This annex reproduces the actual source code of the ten functions this report leans on most heavily when describing how a specific guarantee is implemented — so that a claim like "the grounding check normalizes whitespace before comparing" can be checked against the real sixty lines of Python that implement it, not a paraphrase. Each excerpt is preceded by a one- or two-sentence pointer back to the report section that discusses it, and reproduces the function's own in-code documentation and comments unchanged, since those comments are themselves part of this project's decision-logging discipline (Section 3.4) and often cite the exact real numbers or real failure cases that motivated the code.

G.1 The grounding check — `agent/generate.py`

Discussed in Section 8.4 and Section 9.4. `_mentions_expected_facts()` is the function that decides whether an LLM's rephrased answer is trustworthy enough to show a user, and `_normalize_for_match()` is the whitespace/case tolerance fix described in Section 9.4.

```
def _normalize_for_match(s):
    # Collapses whitespace and lowercases before comparing - makes the
    # grounding check robust to trivial formatting noise a model might
    # introduce (extra/missing spaces, different casing) WITHOUT
    # weakening what it actually verifies: the exact same words, in the
    # exact same order, still have to be present. Added 2026-09-03 (see
    # docs/decisions.md) after finding this was rejecting some
    # otherwise-faithful rephrasings over nothing but incidental
    # whitespace/case differences.
    return " ".join(s.lower().split())

def _mentions_expected_facts(text, structured_result):
    """
    Grounding check, not just a hopeful prompt: every role name in the
    facts that back this answer must still be present, verbatim (modulo
    whitespace/case - see _normalize_for_match), in the LLM's
    rephrasing. Guards against the model quietly dropping, merging, or
```

```

renaming a role while still sounding fluent - the exact failure
mode that makes free-form LLM output risky for a compliance
document, even under a strict system prompt.
"""
roles = structured_result.get("roles")
if not roles:
    return True
normalized_text = _normalize_for_match(text)
return all(_normalize_for_match(r["role"]) in normalized_text for r in roles)

```

Listing G.1: agent/generate.py — normalization and the grounding check

G.2 Sentence-cap scaling for many-fact answers — agent/generate.py

Discussed in Section 9.4 as the actual fix for the "LLM unavailable" fallback firing too often — relaxing brevity pressure without ever touching the accuracy requirement above.

```

_SHORT_ANSWER_RULE = (
    "5. Answer in 1-3 sentences. No bullet points, no headers, no "
    "markdown, no emoji."
)

_LONG_ANSWER_RULE = (
    "5. Use as many sentences as you need to naturally state every fact "
    "below - there are several this time. A longer, complete answer "
    "that keeps every fact intact is much better than a short one that "
    "drops or blends any of them. Still no bullet points, no headers, "
    "no markdown, no emoji."
)

MANY_FACTS_THRESHOLD = 4

def build_grounding_prompt(question, structured_result, target_language="en"):
    roles = structured_result.get("roles") or []
    system = SYSTEM_PROMPT
    if len(roles) >= MANY_FACTS_THRESHOLD:
        system = system.replace(_SHORT_ANSWER_RULE, _LONG_ANSWER_RULE)
    # ... target-language rule 6 and the final prompt assembly follow;
    # see the full file for the complete function.
    return system, user

```

Listing G.2: agent/generate.py — build_grounding_prompt, abbreviated

G.3 Deterministic typo correction — knowledge/typo_correct.py

Discussed in Section 10.11. Note the code comment's exact, measured similarity scores for both the real typos this was built to catch and the false positive that calibrated the final threshold.

```
import difflib
import re

_WORD_PATTERN = re.compile(r"[A-Za-z']+")

# Tuned against a real false-positive, not picked arbitrarily: 0.82
# was loose enough to "correct" the genuinely-different, correctly-
# spelled word "unrelated" into "related" (ratio 0.875 - they share a
# root, textbook false positive for any edit-distance approach) purely
# because "related" happened to be the closest word in a narrow
# ~1400-word DAM-title vocabulary. Every real typo this feature was
# built for - "aproves"/"approves" (0.933), "chek"/"check" (0.889),
# "intiates"/"initiates" (0.941), "qaurterly"/"quarterly" (0.889),
# "mision"/"mission" (0.923), "helo"/"hello" (0.889) - still clears
# 0.88 with room to spare, so raising the floor to 0.88 closes that
# false-positive gap without losing any of the cases that motivated
# building this in the first place.
DEFAULT_MIN_RATIO = 0.88
DEFAULT_MIN_WORD_LENGTH = 4

def correct_words(text, vocabulary, min_ratio=DEFAULT_MIN_RATIO, min_word_length=
    DEFAULT_MIN_WORD_LENGTH):
    """
    Deterministic, offline typo correction - no LLM, no network.
    Replaces each alphabetic word in 'text' with its closest match in
    'vocabulary' when it isn't already an exact match (case-
    insensitive) but is close enough (difflib SequenceMatcher ratio
    >= min_ratio). Conservative on purpose - a word with no
    sufficiently close match is left exactly as typed.
    """
    lower_vocab = {w.lower() for w in vocabulary}

    def replace(match):
        word = match.group()
        if len(word) < min_word_length or word.isupper():
            return word
        lowered = word.lower()
```

```

        if lowered in lower_vocab:
            return word
        candidates = difflib.get_close_matches(lowered, lower_vocab, n=1, cutoff=
min_ratio)
        if not candidates:
            return word
        corrected = candidates[0]
        return corrected.capitalize() if word[0].isupper() else corrected

return _WORD_PATTERN.sub(replace, text)

```

Listing G.3: knowledge/typo_correct.py

G.4 Action-code legend detection — agent/action_codes.py

Discussed in Section 10.6. The conservative rule for treating a bare letter like "I" as a real code mention (rather than the pronoun) is the core of the fix for the context-carryover bug.

```

_BARE_CODE_TOKEN = re.compile(r"\b([ICRA][1-4]?)\b")
_SUFFIXED_CODE = re.compile(r"^[ICRA][1-4]$")

def detect_action_code_query(query):
    """
    Returns {"codes": [...]} or {"codes": None} (generic legend
    question) if this looks like a question about the DAM's action-code
    legend, else None. Deliberately conservative about single bare
    letters (I/C/R/A) by themselves - "I" in particular collides with
    the pronoun far too often to trust alone. Only treated as a real
    code mention when EITHER: two or more code-shaped tokens appear
    together (a list, like the real "I, A and (i)" case this was built
    from), or at least one token is unambiguous on its own ("(i)", or a
    letter+digit combination like "A2"/"C1" that is not a plausible
    English word).
    """
    if not _EXPLAIN_TRIGGER.search(query):
        return None

    codes = _extract_code_tokens(query)
    has_unambiguous = any(c == "( i )" or _SUFFIXED_CODE.match(c) for c in codes)

    if codes and (len(codes) >= 2 or has_unambiguous):

```

```

        valid = [c for c in codes if c in _CODES]
        if valid:
            return {"codes": valid}

    if _GENERIC_LEGEND_TRIGGER.search(query):
        return {"codes": None}

    return None

```

Listing G.4: agent/action_codes.py — detect_action_code_query

G.5 Tone pre-filter and empathetic prefix — llm/tone.py

Discussed in Section 8.6 and Section 10.6. `looks_emotional()` is the cheap, deterministic gate that decides whether the one paid LLM classification call is even worth attempting; `apply_tone_prefix()` is the deliberately non-LLM response-adaptation step that follows it.

```

_REPEATED_PUNCTUATION = re.compile(r"[!?]{2,}")
_ALL_CAPS_WORD = re.compile(r"\b[A-Z]{4,}\b")

_FRUSTRATION_PATTERNS = re.compile(
    r"\b(useless|terrible|annoying|frustrat\w*|ridiculous|broken|"
    r"stupid|worst|hate|sucks?|ugh+|wtf|ffs|seriously)\b|"
    r"\b(still (not|isn't|doesn't)|not working|doesn't work|"
    r"come on|waste of time|fix this|this again)\b",
    re.IGNORECASE,
)

_CONFUSION_PATTERNS = re.compile(
    r"\b(confus\w*|clueless|lost|huh)\b|"
    r"\b(don't (get|understand)|doesn't make sense|no idea what|"
    r"not sure what|what does that mean|i'm (so )?lost)\b",
    re.IGNORECASE,
)

def looks_emotional(text):
    """
    Cheap, deterministic pre-filter - decides whether it's even worth
    ATTEMPTING the Groq tone classification, never trusted for the
    actual tone itself.
    """
    if _REPEATED_PUNCTUATION.search(text):
        return True

```

```

if _ALL_CAPS_WORD.search(text):
    return True
if _FRUSTRATION_PATTERNS.search(text):
    return True
if _CONFUSION_PATTERNS.search(text):
    return True
return False

def apply_tone_prefix(answer, tone, language="en"):
    """
    Prepends a short, tone-appropriate opener to 'answer' when 'tone'
    is "frustrated" or "confused" - a no-op for "neutral". Kept
    entirely separate from 'deterministic_answer' in the caller - this
    is a warmth/UX layer on top of the facts, never part of the
    trusted facts themselves.
    """
    prefixes = EMPATHY_PREFIXES.get(language, EMPATHY_PREFIXES["en"])
    prefix = prefixes.get(tone)
    if not prefix:
        return answer
    return f"{prefix} {answer}"

```

Listing G.5: llm/tone.py — the pre-filter and the deterministic prefix

G.6 Multi-language pre-filter — llm/translate.py

Discussed in Section 8.6 and Section 10.4. Mirrors the same cost-avoidance pattern as G.5's tone pre-filter — a cheap heuristic gates an expensive LLM round trip.

```

_ARABIC_SCRIPT = re.compile(r"[\u0600-\u06FF]")
_ACCENTED_LATIN = re.compile(r"[âãäéêëïîôöùüçñãõ]", re.IGNORECASE)
_WORD_RE = re.compile(r"[a-zâ-ÿ']+")

_FUNCTION_WORDS = {
    "fr": {"le", "la", "les", "des", "une", "qui", "que", "pour", "dans",
          "est", "du", "avec", "sont", "quels", "quelles"},
    "es": {"el", "los", "las", "que", "es", "una", "por", "para", "con",
          "quien", "quienes", "cuales", "cual"},
    "pt": {"os", "que", "uma", "por", "para", "com", "quem", "quais", "qual"},
}

def looks_non_english(text):
    """

```

```

Best-effort guess, not a real language detector - just cheap
enough to gate the expensive path. A single whole-word hit is
enough - measured directly against a realistic short question
("qui approuve 3.111") that only contains ONE real function word
("qui") once the DAM-specific verb is excluded; a 2-hit
requirement (an earlier version of this check) missed it entirely.
"""

if _ARABIC_SCRIPT.search(text):
    return True
if _ACCENTED_LATIN.search(text):
    return True
words = set(_WORD_RE.findall(text.lower()))
return any(words & lang_words for lang_words in _FUNCTION_WORDS.values())

```

Listing G.6: llm/translate.py — looks_non_english

G.7 Lossless node caching — modeling/nodes_cache.py

Discussed in Section 10.1, the fix for the Render out-of-memory failure. Reproduced in full since it is short — the entire mechanism that lets the backend skip a 12.7-second PDF re-parse on every process startup.

```

import json
from pathlib import Path

from schema.schema import Node

# has_children/actions are @computed_field on Node - always derivable
# from children/responsibilities, so caching them would just be dead
# weight on disk and Node recomputes them for free the moment
# load_nodes() reconstructs each one.
_COMPUTED_FIELDS = {"has_children", "actions"}

def save_nodes(nodes, cache_path):
    """Serializes {id: Node} to a single JSON file at cache_path."""
    cache_path = Path(cache_path)
    cache_path.parent.mkdir(parents=True, exist_ok=True)
    data = {
        node_id: node.model_dump(exclude=_COMPUTED_FIELDS)
        for node_id, node in nodes.items()
    }
    cache_path.write_text(json.dumps(data))

```

```
def load_nodes(cache_path):
    """Reconstructs {id: Node} from a file written by save_nodes()."""
    data = json.loads(Path(cache_path).read_text())
    return {node_id: Node(**node_data) for node_id, node_data in data.items()}
```

Listing G.7: modeling/nodes_cache.py — complete file

G.8 The /api/ask request/response contract — webapp/backend.py

Discussed throughout Chapter 10. The Question model is the complete contract between the frontend and the agent, including the explicit language-override field added in September 2026.

```
class Question(BaseModel):
    question: str
    llm: Optional[str] = None
    # The node_id this same chat thread last resolved to, if any - the
    # frontend tracks this per conversation and sends it back so
    # pronoun-style follow-ups have a real anchor instead of
    # resolve_query guessing off incidental word overlap.
    previous_node_id: Optional[str] = None
    # Explicit language picker override for the ANSWER's language -
    # independent of whatever language the question text itself is
    # in. None/"auto"/omitted keeps the original auto-detect-from-the-
    # question behavior. Any other supported code (en/fr/es/pt/ar)
    # always wins.
    target_language: Optional[str] = None
```

Listing G.8: webapp/backend.py — the Question model

G.9 Tone detection wired into the request handler — webapp/backend.py

Discussed in Section 8.6. This is the final step of the /api/ask handler — the point where the prefix from G.5 is actually applied to whatever answer the rest of the pipeline already produced, regardless of which path produced it.

```
detected_tone = "neutral"
if provider is not None and looks_emotional(payload.question):
    tone_result = detect_tone(payload.question, provider)
    detected_tone = tone_result["tone"]
```

```
result["detected_tone"] = detected_tone
result["answer"] = apply_tone_prefix(result["answer"], detected_tone,
answer_language)

return result
```

Listing G.9: webapp/backend.py — end of ask(), tone detection

G.10 Reproducibility of the knowledge-graph query used in Figure 3

Discussed in Section 8.1. This illustrates the exact graph query whose output was checked, role-for-role and footnote-for-footnote, against a real screenshot of the source document.

```
# responsible_roles(graph, node_id, action=None) walks every
# 'responsible_for' edge out of the given DAM node, optionally
# filtered to one authority-code action, and returns the connected
# role nodes with their edge attributes (level, footnote_refs).
#
# responsible_roles(graph, "3.111", action="A") returned, verified
# against the real printed table on page 58 of the source PDF:
# [{"role": "Origination Sector Manager", "level": 3, "footnote_refs": [3]},
#  {"role": "Supporting Dept. Division Manager", "level": 4, "footnote_refs":
#    [4]}]
```

Listing G.10: conceptual reconstruction of the validated query, modeling/graph.py

Appendix H

API Reference

Complete reference for the four HTTP endpoints exposed by `webapp/backend.py`, referenced from Section 11.7.

H.1 POST `/api/ask`

The system's core endpoint. Accepts a question and returns a fully-resolved answer along with the metadata needed to render confidence, language, and LLM-usage information in the UI.

Request body:

```
{
  "question": "who approves 2.126",
  "llm": "groq",
  "previous_node_id": null,
  "target_language": "auto"
}
```

Listing H.1: example request

Response body (fields, not exhaustive of every possible key):

Field	Type	Meaning
<code>answer</code>	string	The final answer text shown to the user, after any LLM phrasing, tone prefix, and translation
<code>deterministic_answer</code>	string	The original template-based answer, always present, never LLM-phrased
<code>node_id</code>	string or null	The resolved DAM activity identifier, if any

Field	Type	Meaning
method	string	How the question was resolved: id, text_search, smalltalk, glossary, action_code_legend, invalid_id, needs_clarification, out_of_scope, or a context-carryover variant
score	number or null	TF-IDF confidence score when method is text_search
roles	array	The structured facts (role, action, level, footnote_refs) backing this answer
used_llm	boolean	Whether the shown answer was produced by an LLM rephrasing that passed the grounding check
llm_provider	string or null	Which provider actually produced the phrasing (groq, ollama), if any
llm_error	string or null	The reason LLM phrasing was not used, if applicable
detected_language	string	The detected language of the question text (en/fr/es/pt/ar)
answer_language	string	The language the answer was actually phrased in

Field	Type	Meaning
translation_error	string or null	Set when translation was needed but no LLM provider was available
detected_tone	string	neutral, frustrated, or confused

H.2 GET /api/health

Used for both the frontend's live status indicator and Render's platform health check.

```
{"status": "ok", "nodes_loaded": 327}
```

Listing H.2: example response

H.3 GET /api/llm/config

Reports the actual model names each LLM mode would currently use, read directly from the provider classes so the UI can never drift from what the backend actually does.

```
{"ollama_model": "llama3.1", "groq_model": "openai/gpt-oss-120b"}
```

Listing H.3: example response

H.4 GET /api/dashboard/summary

Backs the analytics dashboard. Returns node-type counts, total responsibilities, the authority-code distribution, the top roles by responsibility count, unresolved-role rate, per-chapter breakdowns, and knowledge-graph totals — the exact figures cited in Section 9.3 are read live from this endpoint's own output, not hand-copied.

```
{
  "node_counts": {"chapter": 3, "process": 35, "task": 138, "child_task": 102, "
    threshold_variant": 49},
  "total_responsibilities": 1776,
  "unresolved_count": 10,
  "unresolved_rate": 0.0056,
  "distinct_roles": 131,
  "avg_responsibilities_per_node": 5.48,
```

```
"no_direct_responsibilities_rate": 0.2963,  
"graph": {"total_graph_nodes": 459, "total_edges": 2108}  
}
```

Listing H.4: example response, abbreviated

Table of Contents

Acknowledgments	ii
Contents	v
List of Figures	vii
List of Tables	viii
List of Abbreviations	x
Abstract	xii
General Introduction	2
Context of the Internship	2
Report Structure and How to Read It	2
A Note on Authorship and Process	3
1 Host Institution: Polytech Intl	4
1.1 Presentation	4
1.2 Academic Supervision	4
1.3 Why This Arrangement	4
2 Client Organization: African Development Bank Group	5
2.1 The African Development Bank Group	5
2.2 Project Context and Working Model	5
2.3 Tools and Technologies	6
2.4 Data Confidentiality	6
2.5 Project Governance: The Decision Log	6
3 Literature Review	7
3.1 CRISP-DM as a process model	7
3.2 Knowledge graphs as an enterprise data structure	8
3.3 Lexical versus semantic text retrieval	8
3.4 PDF-based information extraction	8
3.5 Retrieval-augmented generation and the hallucination problem	9

4	Business Understanding	10
4.1	Business context	10
4.2	Objective	10
4.3	Success criteria	11
4.4	Constraints	11
5	Requirements Specification	12
5.1	Functional requirements	12
5.2	Non-functional requirements	13
5.3	Use cases	14
6	Data Understanding	16
6.1	The source document	16
6.2	Initial data exploration: what pdfplumber sees	16
6.3	Data quality issues catalogued before parsing began	17
6.4	Scope decisions grounded in real data	18
6.5	Two deep-dive investigations worth narrating in full	19
7	Data Preparation	21
7.1	Parsing pipeline overview	21
7.2	Title and boundary detection	21
7.3	Reference and footnote cleaning	21
7.4	Action, level, and footnote geometry	23
7.5	Role/column attribution	23
7.6	Node assembly and validation	24
7.7	A chain of six real bugs from one scoping question	25
8	Modeling	28
8.1	Knowledge graph	28
8.2	Natural-language understanding	29
8.3	Why TF-IDF over trained embeddings	30
8.4	Grounded generation: how the LLM is allowed to touch the answer	31
8.5	Hybrid local/cloud LLM layer	33
8.6	Recent modeling extensions: multi-language, mandatory notes, tone	33
9	Evaluation	34
9.1	Evaluation philosophy	34
9.2	Automated test suite	34
9.3	Data-model evaluation metrics	35
9.4	Correctness under generation: the grounding check in practice	35

9.5	Live verification against the real system	36
9.6	Recurring bug pattern: found by real testing, not by inspection	36
10	Deployment	38
10.1	Hosting	38
10.2	Containerization	39
10.3	Web application	39
10.4	Multi-language support	39
10.5	Operational incident: a live production bug and its resolution	40
10.6	Chatbot feature iteration following professor and user feedback	41
10.7	Live status and reliability signaling	42
10.8	The hosting-platform search, narrated in full	42
10.9	Glossary and abbreviation lookup	43
10.10	The React frontend rewrite, and a real sandbox environment limitation . . .	44
10.11	Deterministic typo tolerance	45
10.12	Conversational context carryover: a two-round fix	45
10.13	Mobile responsiveness and the dashboard rework	46
11	User Manual and Deployment Guide	47
11.1	Running the system with no setup at all	47
11.2	Local setup (bare metal)	47
11.3	Local setup (Docker)	47
11.4	Environment variables reference	48
11.5	Choosing an LLM mode	48
11.6	Deploying to Render (free tier)	48
11.7	API reference	49
11.8	Verifying an LLM mode is actually working	49
12	Project Management and Process	50
12.1	Timeline	50
12.2	Decision-logging discipline	50
12.3	Working method	50
13	Lessons Learned and Limitations	52
13.1	Technical lessons	52
13.2	Known limitations	52
	General Conclusion	54
	Web References	55

A	Automated Test Suite Breakdown	57
B	Project Directory Structure	59
C	Sample Live-Demo Transcript	60
D	Selected Decision-Log Excerpts	61
E	Glossary of Terms Used in This Report	62
F	Complete Project Decision Log (Verbatim)	64
G	Key Code Excerpts	124
G.1	The grounding check — agent/generate.py	124
G.2	Sentence-cap scaling for many-fact answers — agent/generate.py . . .	125
G.3	Deterministic typo correction — knowledge/typo_correct.py	126
G.4	Action-code legend detection — agent/action_codes.py	127
G.5	Tone pre-filter and empathetic prefix — llm/tone.py	128
G.6	Multi-language pre-filter — llm/translate.py	129
G.7	Lossless node caching — modeling/nodes_cache.py	130
G.8	The /api/ask request/response contract — webapp/backend.py	131
G.9	Tone detection wired into the request handler — webapp/backend.py . .	131
G.10	Reproducibility of the knowledge-graph query used in Figure 3	132
H	API Reference	133
H.1	POST /api/ask	133
H.2	GET /api/health	135
H.3	GET /api/llm/config	135
H.4	GET /api/dashboard/summary	135